University of Basilicata

Department of Basic and Applied Sciences - DISBA

Numerical Analysis

A First Course

Student:
Donato Martinelli

Academic Year 2025/2026

Contents

0.1 Propagation of Errors Introduced in Initial Data . . . 24

1 Resolution of a Square Linear System 31
1.1 The Problem of Solving a Linear System . . . 31
1.2 Diagonal and Triangular Systems . . . 40
1.3 Gaussian Elimination Method . . . 47
1.4 Pivoting Strategy and LU Factorization . . . 51
1.5 Cholesky Method . . . 60
1.6 Stability of Gauss Methods . . . 63
1.7 Exercises . . . 69

2 Numerical Methods for Solving a Rectangular Linear System in the Least Squares Sense 80
2.1 Exercises . . . 85

3 Numerical Methods for Solving Non-Linear Equations 88
3.1 Solving Non-Linear Equations . . . 88
3.2 Solution of Algebraic Equations . . . 97
3.3 Exercises . . . 108

Introduction

The Role of Mathematics and Modeling Mathematics is an indispensable tool for interpreting and predicting real-world phenomena, serving as the foundation for all sciences. Scientists seek to derive a mathematical model (variables, parameters, relations) that is rigorous and consistent with the phenomenon. The complexity of the phenomenon determines the amount of data required and the number of variables in the model. Unknown properties are deduced by solving such a model.

Necessity of the Numerical Approach Often a solution is not available in explicit form or is not analytically calculable, making a numerical approximation method necessary. Even when an analytical resolution is possible, it may turn out to be too computationally expensive as the size of the problem increases.

Esempio 0.1. To clarify the necessity of approximation, let us consider some examples:

- **Quadratic equation:** Given the equation $ax^2 + bx + c = 0$, the solutions are $x_{1,2} = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$. If $a = 1, b = 1, c = -1$, the solutions are $\frac{-1 \pm \sqrt{5}}{2}$. To determine the numerical value, it is necessary to approximate $\sqrt{5}$.
- **Exponential equation:** The solution to $2^x = 100$ is $x = \log_2(100)$. Here too, the numerical answer requires an approximation.
- **Transcendent equation:** The equation $2^x + x^2 - 3 = 0$ is not analytically solvable (it cannot be made explicit with respect to x). Although it can be studied graphically, the calculation of the solutions strictly requires a numerical method.

Numerical Method

Definizione 0.1. A numerical method is a process implementable on a computer that provides a numerical solution in a finite number of steps.

There exist mathematically valid methods that are, however, not implementable on a computer due to execution times.

Case Study: Linear Systems (Cramer vs. Gauss) A crucial example concerns the resolution of linear systems, a central problem in modeling.

- **Cramer's Rule (Unimplementable):** If we wanted to solve a 20×20 system ($n = 20$) with Cramer, we would have to calculate 21 determinants of order 20. With Laplace's rule, the operations are approximately $21 \cdot 20! \approx 5.2 \cdot 10^{19}$. On a 3GHz processor ($3 \cdot 10^9$ op/sec), the time required would be more than 5 centuries. It is a procedure with prohibitive execution time.
- **Gaussian Elimination Method (Efficient):** This method requires approximately $\frac{n^3}{3}$ operations. For $n = 20$, only 2667 operations are needed ($0.88 \cdot 10^{-6}$ seconds). For $n = 1000$, 334 million operations are needed, executable in a tenth of a second.

Selection Criteria and Errors The choice of method is based on two fundamental criteria:

1. **Efficiency:** fewer required operations.
2. **Effectiveness:** greater calculation precision.

Despite the power of modern computers, mathematicians still face difficulties. Since numbers are represented in binary on a finite memory, every storage introduces an error.

Error Propagation A central theme of Scientific Computing is the awareness of error and its propagation. Some methods propagate errors (introduced in the initial data) to the point of providing completely wrong solutions. The stability of algorithms with respect to these errors is therefore studied.

Representation of Numbers in a Computer

Numbering Systems A number is a concept that exists independently of the set of symbols or signs used to represent it. A numbering system is nothing more than a scheme that allows "ideal" numbers to be represented through a set of symbols for which manipulation techniques (arithmetic operations) are defined. It is certain that all peoples learned to count, driven by the needs of daily life, to quantify a set of elements. Primitive men limited their knowledge to the first natural numbers, to indicate which they resorted to the names of concrete objects, "I" to indicate 1, "wings" to indicate 2, "hand" to indicate 5. Later they began to represent numbers with vertical bars. Every integer was represented with a number of bars equal to the units contained within it. This system, although the simplest possible, has the disadvantage of assigning each number a representation whose length is proportional to the number itself. Thus, very soon, the need arose to represent the infinity of numbers with a limited number of particular signs, called digits, and to fix some fundamental numbers that served as references. Thus the idea of what is called a numbering base was born. Almost all peoples chose 10 as the numbering base. A choice certainly due to the number of fingers on the hands. The numbering systems of known civilizations, starting from primitive populations, are classified into:

- additive numbering system
- positional numbering system

An additive numbering system is a numbering system based on an additive law applied to certain fundamental numerical symbols. Every number is represented through a sequence of such symbols and its value is given by the sum of the values attributed to each of them. In additive systems, a symbol for zero is not needed. Among the most famous additive systems, in addition to that of bars, are the Roman and Egyptian systems.

Only around the 10th century did the additive numbering system begin to be supplanted by the positional numbering system introduced by Indian and Arab mathematicians. A numbering system is called positional if the symbols used to write the numbers assume different values depending on the position they occupy in the notation.

Decimal system The numbering system commonly used by us to represent numbers is the decimal one. It is a positional numbering system based on the use of 10 digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, and was invented by the Indians. The introduction of zero is credited to the Indians, who called it sunya (nothing), although some scholars state that the Chinese already knew the positional method from ancient times. This numbering system was introduced to the West by the Arabs, probably in the 12th century when *Algoritmi de numero Indorum* by the great Arab mathematician al-Khuwarizmi (9th century) was translated. The base of the decimal system is 10 and every number is represented as

$$a = \pm a_m a_{m-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-M}$$

with

$$0 \leq a_i \leq 9$$

The system is positional because the value of each digit varies according to its position in the decimal representation of the number

$$z = \pm a_m 10^m + a_{m-1} 10^{m-1} + \dots + a_1 10^1 + a_0 10^0 + a_{-1} 10^{-1} + a_{-2} 10^{-2} + \dots$$

Esempio 0.2.

- $931.57 = 9 \times 10^2 + 3 \times 10^1 + 1 \times 10^0 + 5 \times 10^{-1} + 7 \times 10^{-2}$
- $34.002 = 3 \times 10^1 + 4 \times 10^0 + 0 \times 10^{-1} + 0 \times 10^{-2} + 2 \times 10^{-3}$
- $12600.09 = 1 \times 10^4 + 2 \times 10^3 + 6 \times 10^2 + 0 \times 10^1 + 0 \times 10^0 + 0 \times 10^{-1} + 9 \times 10^{-2}$

The decimal representation of every real number is unique, except when the fractional part contains a sequence of consecutive 9s.

Esempio 0.3.

- 0.319999...9... = 0.32
- 10.99999...9... = 11
- 199.99999...9... = 200

Base N System Any integer $N > 1$ can be chosen as a base and every real number a can be written in the form

$$z = \pm a_m N^m + a_{m-1} N^{m-1} + \cdots + a_1 N^1 + a_0 N^0 + a_{-1} N^{-1} + a_{-2} N^{-2} + \dots$$

that is

$$a = \pm a_m a_{m-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-M}$$

with $0 \leq a_i \leq N - 1$. The representation of every real number in base N is unique, except when the fractional part contains a sequence of consecutive $a_k = N - 1$ digits. The smaller the base chosen, the longer the string of characters necessary to represent the same number.

Binary system The base of the binary system is 2. The digits used by this system are 0 and 1 and are called bits, from *binary digit*. Every real number a is represented as a sequence of 0s and 1s, that is

$$a = \pm a_m 2^m + a_{m-1} 2^{m-1} + \cdots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + a_{-2} 2^{-2} + \dots$$

with $0 \leq a_i \leq 1$. This system is particularly interesting because it can be realized with any object capable of assuming two different states, one for the digit 0 and the other for the digit 1.

Esempio 0.4.

- State of magnetization and non-magnetization of a ferrite core;
- conductivity and non-conductivity of a diode.

Due to these characteristics, it was adopted for the representation of data and, specifically, numbers in a computer.

Numerical representation in a computer Numbers are represented in a computer according to the binary system and, therefore, as a sequence of bits. For obvious reasons, a finite space of memory is reserved for each real number, capable of containing a finite number of bits called a **word**. Consequently, not all real numbers can be represented exactly. Let l be the length of the word; only those numbers whose binary representation consists of a number of bits less than or equal to l can be represented exactly. These are called **machine numbers**. All operations between machine numbers are performed using binary arithmetic.

Integers With words of length l, it is possible to represent all integers belonging to the interval

$$\left[-\frac{2^l}{2}, \frac{2^l}{2} - 1 \right]$$

Example: If $l = 16$, all integers belonging to the interval $[-32768, 32767]$ are representable.

Floating-point representation

Definizione 0.2. Every real number a can be written in the form

$$a = pN^q$$

where p is a real number called the **mantissa** of a, N is the base of the numbering system, and q is a

positive or negative integer called the **exponent** of a. The representation of a is said to be **normalized** when

$$N^{-1} \leq |p| < 1$$

The digits of p are called **significant digits**. Once the base N is fixed, every real number a is uniquely defined by the pair

$$a = (p, q)$$

This representation, called floating-point, is not unique; indeed

$$321.25 = 32.125 \times 10^1 = 0.32125 \times 10^3$$

Esempio 0.5.

- The normalized representation of $a = 92.25$ is $a = 0.9225 \times 10^2$;
- the normalized representation of $a = 0.000718$ is $a = 0.718000 \times 10^{-3}$;
- the normalized representation of $a = 4152.0002156$ is $a = 0.41520002156 \times 10^4$;
- the normalized representation of $a = 0.0215600000$ is $a = 0.2156 \times 10^{-1}$.

Floating-point real numbers are represented in a computer in normalized form according to the IEEE 754 Standard (Institute of Electrical and Electronics Engineers).

IEEE 754 Standard: Single Precision

Definizione 0.3. The structure is defined by the following parameters:

$$s = 1 \text{ bit}$$

$$p = 23 \text{ bits}$$

$$q = 8 \text{ bits}$$

It is possible to represent all real numbers of the following form:

$$\pm 0.d_1 d_2 d_3 d_4 d_5 d_6 \times 10^q$$

where:

$$0 < d_1 \leq 9, \quad 0 \leq d_i \leq 9, \quad i \in \{2, \ldots, 6\}$$

and

$$-38 \leq q \leq 38$$

There are **6 significant digits**.

IEEE 754 Standard: Double Precision

Definizione 0.4. The structure is defined by the following parameters:

$$s = 1 \text{ bit}$$

$$p = 52 \text{ bits}$$

$$q = 11 \text{ bits}$$

It is possible to represent all real numbers of the following form:

$$\pm 0.d_1 d_2 \ldots d_{16} \times 10^q$$

where:

$$0 < d_1 \leq 9, \quad 0 \leq d_i \leq 9, \quad i \in \{2, \dots, 16\}$$

and

$$-308 \leq q \leq 308$$

There are **16 significant digits**.

Nota 1. In the Double Precision IEEE 754 standard, although the space reserved for the mantissa of a real number is 52 bits, an extra bit is recovered by not representing the first bit, which is always equal to 1. Therefore, the mantissas p of real numbers are represented in double precision with 53 bits.

Assuming the use of a computer working in double precision, given a non-zero real number:

$$a = \pm 0.d_1 d_2 \dots 10^q, \quad d_1 \neq 0$$

the following cases may occur:

1. The exponent q is such that $-308 \leq q \leq 308$ and the digits after the 16th are all zero, i.e., $d_i = 0$ for every $i > 16$, so
$$a = \pm 0.d_1 d_2 \dots d_{16} 10^q$$
Then a is **exactly representable**.

2. The exponent q does not belong to the interval $[-308, 308]$. If $q < -308$, 0 is associated with a and the computer signals **underflow**. If $q > 308$, a is not represented and the computer signals **overflow**.

Nota 2. In the latest generation of computers, it is also possible to represent numbers whose exponent q is such that

$$-324 < q < -308$$

These numbers fill the interval between zero and the smallest representable normalized number

$$\text{realmin} = 2^{-1022} \sim 2.22 \times 10^{-308}$$

and are called **denormalized** (or subnormal). The largest normalized number representable in double precision is

$$\text{realmax} \sim 1.79 \times 10^{308}$$

It is one bit less than 2^{1024}.

3. The exponent q belongs to the interval $[-308, 308]$ but the digits d_i, $i > 16$, are not all zero. In this case, it is possible to associate a machine number $fl(a)$ with the number a following two different criteria:
   - truncation
   - rounding

Truncation rule To truncate a number

$$a = 0.d_1 d_2 \dots d_{t-1} d_t d_{t+1} d_{t+2} \dots N^q$$

to t significant digits, all digits $d_{t+1} d_{t+2} \dots$ to the right of the t-th digit are eliminated. Thus

$$fl(a) = trn(a) = 0.d_1 d_2 \dots d_{t-1} d_t N^q.$$

Rounding rule To round a number

$$a = 0.d_1 d_2 \dots d_{t-1} d_t d_{t+1} d_{t+2} \dots N^q$$

to t significant digits, all digits $d_{t+1} d_{t+2} \dots$ to the right of the t-th digit are eliminated and d_t is replaced with the digit c, where

$$\begin{cases} c = d_t & \text{if } 0 \le d_{t+1} \le 4 \\ c = d_t + 1 & \text{if } 5 \le d_{t+1} \le 9 \end{cases}$$

Thus

$$fl(a) = arr(a) = 0.d_1 d_2 \dots d_{t-1} c N^q.$$

Esempio 0.6.

- Consider the number $\pi = 3.1415926536 \cdots = 0.31415926536 \dots 10^1$.
  - $t = 6$: $trn(\pi) = 0.314159 \cdot 10^1$, $arr(\pi) = 0.314159 \cdot 10^1$
  - $t = 5$: $trn(\pi) = 0.31415 \cdot 10^1$, $arr(\pi) = 0.31416 \cdot 10^1$
  - $t = 4$: $trn(\pi) = 0.3141 \cdot 10^1$, $arr(\pi) = 0.3142 \cdot 10^1$
- Consider the number $a = 2\sqrt{2} = 2.828427124746190 \cdots = 0.2828427124746190 \dots 10^1$.
  - $t = 6$: $trn(a) = 0.282842 \cdot 10^1$, $arr(a) = 0.282843 \cdot 10^1$
  - $t = 5$: $trn(a) = 0.28284 \cdot 10^1$, $arr(a) = 0.28284 \cdot 10^1$
  - $t = 4$: $trn(a) = 0.2828 \cdot 10^1$, $arr(a) = 0.2828 \cdot 10^1$

Advantages of normalized representation Consider the number

$$a = \frac{1}{7000}$$

Let

$$\overline{a} = 0.000142857142857 \dots \quad \text{(non-normalized representation)}$$
$$\overline{\overline{a}} = 0.142857142857142 \dots 10^{-3} \quad \text{(normalized representation)}$$

Applying truncation and rounding rules to $\overline{a}$:

- $t = 6$: $trn(\overline{a}) = 0.000142$, $arr(\overline{a}) = 0.000143$
- $t = 3$: $trn(\overline{a}) = 0$, $arr(\overline{a}) = 0$

Applying truncation and rounding rules to $\overline{\overline{a}}$:

- $t = 6$: $trn(\overline{\overline{a}}) = 0.142857 \cdot 10^{-3}$, $arr(\overline{\overline{a}}) = 0.142857 \cdot 10^{-3}$
- $t = 3$: $trn(\overline{\overline{a}}) = 0.142 \cdot 10^{-3}$, $arr(\overline{\overline{a}}) = 0.143 \cdot 10^{-3}$

Therefore, normalization allows the reduction of representation error due to truncation or rounding to a finite number of digits. Furthermore, the information contained in the zeros after the decimal point can be stored, with less memory waste, in terms of the exponent.

Consequences of representing numbers as fixed-length words

First consequence: Finiteness of the set of representable numbers The set $\mathbb{R}$ of real numbers is infinite, but only a subset of $\mathbb{R}$, bounded below and above, is representable in a computer. Hereafter, we will denote by $\mathbb{F}$ the subset of real numbers representable in a computer.

$$\mathbb{F} = \{0\} \cup \{\pm 0.d_1 d_2 \dots d_{16} \times 10^q \in \mathbb{R} : -308 \le q \le 308\}$$

$$0 < d_1 \le 9, \quad 0 \le d_i \le 9, \quad i \in \{2, \dots, 16\}$$

There is an interval containing representable values between -realmax and -realmin, zero, and between realmin and realmax. Outside this interval, overflow or underflow occurs.

Second consequence: Representable set is "punctured" The set $\mathbb{R}$ of real numbers is dense, meaning between any two real numbers r_1 and r_2 there always exists a real number r_3 such that

$$r_1 < r_3 < r_2$$

In contrast, the subset $\mathbb{F}$, besides being bounded below and above, is also punctured (it has gaps).

Esempio 0.7. Using double precision representation, the machine numbers

$$a = 1.123456789123456$$

and

$$b = 1.123456789123457$$

are consecutive and no other machine number exists between them. All real numbers between a and b are approximated as a or b.

Exercises in MATLAB

- Type the number 0.958425678941526789541263 54 and verify that the program rounds it to 16 significant digits, providing

$$9.584256789415268e - 001 = 0.9584256789415268$$

```
>> 0.958425678941526789541263 54
ans =
     9.584256789415268e-001
```

- Type the number 0.0000005213565836512485898 5 and verify that the program rounds it to 16 significant digits, providing

$$5.21356583651249e - 007 = 0.521356583651249e - 006$$

```
>> 0.0000005213565836512485898 5
ans =
     5.213565836512486e-007
```

- Type the number 12356.00056256984154646 and verify that the program rounds it to 16 significant digits, providing

$$1.23560005625698e + 004 = 0.123560005625698e + 005$$

```
>> 12356.00056256984154646
ans =
     1.235600056256984e+004
```

- Calculate `realmin` $= 2^{-1022} \sim 2.225073858507201e - 308$

```
>> 2^(-1022)
ans =
     2.225073858507201e-308
```

- Calculate the values 2^{-k} with $k = 1040, 1050, 1060, 1070$ and observe that the program can represent numbers with an exponent between -308 and -324 (denormalized numbers).

```
>> k = 1040;
>> 2^(-k)
ans =
     8.487983163861089e-314
>> k = 1050;
>> 2^(-k)
ans =
     8.289046058458095e-317
>> k = 1060;
```

```
>> 2^(-k)
ans =
     8.094771541462983e-320
>> k = 1070;
>> 2^(-k)
ans =
     7.90505033459945e-323
```

- Verify that $2^{-1074} \sim 4.94065645841247e-324$ and that by calculating 2^{-1075} the program returns 0 as the value, therefore it cannot represent numbers with an exponent smaller than -324.

```
>> k = 1074;
>> 2^(-k)
ans =
     4.940656458412465e-324
>> k = 1075;
>> 2^(-k)
ans =
     0
```

- Verify that $2^{1023} \sim 8.988465674311580e+307$ and that by calculating 2^{1024} the program returns Inf, where Inf denotes overflow.

```
>> k = 1023;
>> 2^k
ans =
     8.988465674311580e+307
>> k = 1024;
>> 2^k
ans =
     Inf
```

Errors

Main causes of errors in solving mathematical problems with a computer When solving a mathematical problem with a computer, the sources of error can, in general, be divided into two groups.

1. **Data representation or rounding errors**
   We have seen that not all real numbers can be represented in a computer. If the computer uses a representation with t digits for the mantissa, all real numbers included between the lower bound and the upper bound of $\mathbb{F}$ whose mantissa has a number of digits greater than t must be rounded or truncated to t digits.

2. **Errors in calculations**
   By performing calculations between approximate machine numbers, the representation errors of the initial data propagate to some extent to the results of such calculations. The magnitude of the propagation also depends on the algorithm used.

Data representation errors In representing a real number $a \neq 0$ with a corresponding machine number $fl(a)$, it is necessary to evaluate the error committed, which will obviously be zero if a is itself a machine number, i.e., $a = fl(a)$. For this purpose, given an exact real value a and its approximation A, we provide some definitions.

Absolute Error

Definizione 0.5. The quantity

$$\Delta a = |a - A|$$

is called the absolute error.

```
function e = errAbs(valExact, valApprox)
% ERRABS Calculates the absolute error (Euclidean norm of the difference).
%   E = ERRABS(VALEXACT, VALAPPROX) returns \|valExact - valApprox\|_2.
%   Supports scalars, vectors, and multidimensional matrices.

    %% 1. Input Validation
    arguments
        valExact  double {mustBeNumeric}
        valApprox double {mustBeNumeric}
    end

    %% 2. Dimensions Check
    % isequal is the standard method for comparing array/matrix dimensions
    if ~isequal(size(valExact), size(valApprox))
        error("errAbs:DimensionMismatch", "Incompatible dimensions (%d and %d).", size(valExact), size(valApprox));
    end

    %% 3. Absolute Error Calculation
    e = norm(valExact - valApprox);
end
```

Function errAbs

Relative Error The absolute error fails to characterize what can intuitively be considered a measure of the precision of the approximate value. To measure the quality of an approximation A of a, it is necessary to compare the absolute error with the order of magnitude a of the data to be approximated. That is, calculating the ratio between the absolute error and the absolute value of the exact data.

Definizione 0.6. The quantity

$$\delta a = \frac{|a - A|}{|a|}$$

is called the relative error.

```
function e = errRel(valExact, valApprox, tol)
% ERRREL Calculates the relative error in norm 2.
%   E = ERRREL(VALEXACT, VALAPPROX, TOL) calculates \|valExact- valApprox\| / \|valExact\|.
%
%   NOTE: If \|valExact\| < tol, the function returns the absolute error
%   to avoid division by zero (NaN/Inf).

    %% 1. Input Validation
    arguments
        valExact  double {mustBeNumeric}
        valApprox double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Numerator Calculation (Absolute Error)
    num = errAbs(valExact, valApprox);

    %% 3. Denominator Calculation and Singularity Management
    den = norm(valExact);

    if den < tol
        % Critical case: The exact solution is zero.
        % The relative error is not defined. We return the absolute.
        warning("errRel:ZeroNorm", "The norm of the exact solution is less than the tolerance. Returning Absolute Error.");
        e = num;
    else
```

```
        e = num / den;
    end
end
```

Function errRel

Correct Digits Let

$$a = pN^q \quad \text{and} \quad A = \overline{p}N^q$$

with

$$N^{-1} \leq |p|, |\overline{p}| < 1$$

Correct Decimal Digits

Definizione 0.7. If

$$\Delta a \leq \frac{1}{2}N^{-t}, \quad t \geq 1,$$

it is said that A has at least t correct decimal digits in base N.

Starting from the inequality that relates the absolute error Δa to the number of decimal digits t:

$$\begin{aligned} \Delta a &\leq \frac{1}{2} \cdot 10^{-t} \\ 2 \cdot \Delta a &\leq 10^{-t} \\ \log_{10}(2 \cdot \Delta a) &\leq -t \\ t &\leq -\log_{10}(2 \cdot \Delta a) \end{aligned}$$

Since t must be an integer, we take the maximum value satisfying the condition (floor):

$$t = \lfloor -\log_{10}(2 \cdot \Delta a) \rfloor .$$

```
function n = cifDecCorr(valExact, valApprox)
% CIFDECCORR Calculates the number of correct decimal digits.
%   n = CIFDECCORR(valExact, valApprox) calculates the number of
%   correct decimal digits based on the absolute error.
%
%   Input:
%     valExact  - Exact value (numeric: scalar, vector or matrix)
%     valApprox - Approximate value (numeric: same dimensions as valExact)
%
%   Output:
%     n - Integer number of correct decimal digits.
%       Returns Inf if valExact and valApprox are identical (errAbs 0).

    %% 1. Input Validation
    arguments
        valExact  {mustBeNumeric}
        valApprox {mustBeNumeric}
    end

    %% 2. Dimensions Check
    if ~isequal(size(valExact), size(valApprox))
        error('cifDecCorr:SizeMismatch', 'The inputs must have the same dimension.');
    end

    %% 3. Correct decimal digits calculation
    err = errAbs(valExact, valApprox); % Calculate absolute error
    n = floor(-log10(2 * err));
end
```

Function cifDecCorr

Correct Significant Digits

Definizione 0.8. If

$$\delta a \leq \frac{1}{2} N^{-t+1}, \quad t \geq 1,$$

it is said that A has at least t correct significant digits in base N.

Starting from the inequality that relates the relative error δa to the number of significant digits t:

$$\begin{aligned} \delta a &\leq \frac{1}{2} \cdot 10^{1-t} \\ 2 \cdot \delta a &\leq 10^{1-t} \\ \log_{10}(2 \cdot \delta a) &\leq 1 - t \\ t &\leq 1 - \log_{10}(2 \cdot \delta a) \end{aligned}$$

Since t must be an integer, we take the maximum value satisfying the condition (floor):

$$t = \lfloor 1 - \log_{10}(2 \cdot \delta a) \rfloor .$$

```
function n = cifSigCorr(valExact, valApprox)
% CIFSIGCORR Calculates the number of correct significant digits.
%   n = CIFSIGCORR(valExact, valApprox) calculates the number of
%   significant digits correct based on the relative error.
%
%   Input:
%     valExact  - Exact value (numeric: scalar, vector or matrix)
%     valApprox - Approximate value (numeric: same dimensions as valExact)
%
%   Output:
%     n - Integer number of correct significant digits.

    %% 1. Input Validation
    arguments
        valExact  {mustBeNumeric}
        valApprox {mustBeNumeric}
    end

    %% 2. Dimensions Check
    if ~isequal(size(valExact), size(valApprox))
        error('cifSigCorr:SizeMismatch', 'The inputs must have the same dimension.');
    end

    %% 3. Correct significant digits calculation
    err = errRel(valExact, valApprox); % Calculate relative error
    n = floor(1-log10(2 * err));
end
```

Function cifSigCorr

Nota 3. The significant digits of the number $a = pN^q$ (written in normalized form) are the correct decimal digits of its mantissa p. In fact, assuming that the mantissa $\overline{p}$ of A approximates the mantissa p of a with t correct decimal digits, and considering $N^{-1} \leq |p|, |\overline{p}| < 1$, we have:

$$\delta a = \frac{|p - \overline{p}| N^q}{|p| N^q} = \frac{|p - \overline{p}|}{|p|} \leq \frac{\frac{1}{2} N^{-t}}{N^{-1}} = \frac{1}{2} N^{-t+1}$$

Esempio 0.8. 1. $a = 15.2000$, $A = 15.1997$, $N = 10$

$$\Delta a = 0.3 \ldots 10^{-3} < \frac{1}{2} 10^{-3}$$

$$\delta a = 0.197 \ldots 10^{-4} < \frac{1}{2} 10^{-4}$$

Thus A has at least 3 correct decimal digits and at least 5 correct significant digits.

```
a = 15.2000;    % Exact Value
A = 15.1997;    % Approximate Value

d = cifDecCorr(a, A);
s = cifSigCorr(a, A);

fprintf('The value %g, approximation of %g, has at least %d correct decimal digits.\n', A
    , a, d);
fprintf('The value %g, approximation of %g, has at least %d correct significant digits.\n
    ', A, a, s);
```

```
The value 15.1997, approximation of 15.2, has at least 3 correct decimal digits.
The value 15.1997, approximation of 15.2, has at least 5 correct significant digits.
```

2. $a = 199.2000$, $A = 199.1997$, $N = 10$

$$\Delta a = 0.3\ldots 10^{-3} < \frac{1}{2}10^{-3}$$

$$\delta a = 0.15\ldots 10^{-5} < \frac{1}{2}10^{-5}$$

Thus A has at least 3 correct decimal digits and at least 6 correct significant digits. *Note:* The absolute errors of examples 1 and 2 are equal!

```
a = 199.2000;    % Exact Value
A = 199.1997;    % Approximate Value

d = cifDecCorr(a, A);
s = cifSigCorr(a, A);

fprintf('The value %g, approximation of %g, has at least %d correct decimal digits.\n', A
    , a, d);
fprintf('The value %g, approximation of %g, has at least %d correct significant digits.\n
    ', A, a, s);
```

```
The value 199.1997, approximation of 199.2, has at least 3 correct decimal digits.
The value 199.1997, approximation of 199.2, has at least 6 correct significant digits.
```

3. $a = 1$, $A = 0.9999$, $N = 10$

$$\Delta a = 10^{-4} = 0.1 \cdot 10^{-3} < \frac{1}{2}10^{-3}$$

$$\delta a < \frac{1}{2}10^{-3}$$

Thus A has at least 3 correct decimal digits and at least 4 correct significant digits.

```
a = 1;    % Exact Value
A = 0.9999;    % Approximate Value

d = cifDecCorr(a, A);
s = cifSigCorr(a, A);

fprintf('The value %g, approximation of %g, has at least %d correct decimal digits.\n', A
    , a, d);
fprintf('The value %g, approximation of %g, has at least %d correct significant digits.\n
    ', A, a, s);
```

```
The value 0.9999, approximation of 1, has at least 3 correct decimal digits.
The value 0.9999, approximation of 1, has at least 4 correct significant digits.
```

4. $a = 332.122$, $A = 332.129$, $N = 10$

$$\Delta a = 0.7\ldots 10^{-2} = \frac{1}{2}0.14\ldots 10^{-1} < \frac{1}{2}10^{-1}$$

$$\delta a = 0.2\ldots 10^{-4} < \frac{1}{2}10^{-4}$$

Thus A has at least 1 correct decimal digit and at least 5 correct significant digits.

```
a = 332.122;    % Exact Value
A = 332.129;    % Approximate Value

d = cifDecCorr(a, A);
s = cifSigCorr(a, A);

fprintf('The value %g, approximation of %g, has at least %d correct decimal digits.\n', A
    , a, d);
fprintf('The value %g, approximation of %g, has at least %d correct significant digits.\n
    ', A, a, s);
```

```
The value 332.129, approximation of 332.122, has at least 1 correct decimal digits.
The value 332.129, approximation of 332.122, has at least 5 correct significant digits.
```

Roundoff Unit

Teorema 0.1. *Denoting by $A = fl(a)$ the machine number obtained by rounding the real number a to t significant digits in base N, we have*

$$\delta a \leq u$$

where

$$u = \frac{1}{2} N^{1-t}$$

The quantity u is called the ***roundoff unit****.*

The preceding theorem tells us that every number can be represented on a computer with an error that does not exceed u. In fact, setting $\delta = \frac{fl(a)-a}{a}$, we have

$$fl(a) = a(1+\delta), \quad |\delta| \leq u$$

The quantity u is a constant inherent to each floating-point arithmetic and represents the maximum calculation precision achievable on a computer on which such arithmetic is implemented.

Machine Epsilon The distance between two consecutive machine numbers is instead called **machine precision** or **machine epsilon**. It is possible to demonstrate that machine epsilon is exactly double the roundoff unit, i.e., $\varepsilon = 2u$. Therefore, we will refer indifferently to the roundoff unit or machine precision. In the case of the set $\mathbb{F}$ (double precision IEEE 754 standard, with 53 bits of mantissa including the hidden bit), the roundoff unit is:

$$u = \frac{1}{2} 2^{1-53} = 2^{-53} \approx 1.110223024625157e-016$$

and therefore

$$\varepsilon = 2^{-52} \approx 2.220446049250313e-016$$

Machine precision ε represents the width of the gap existing between consecutive machine numbers:

$$1.000000000000000$$

and

$$1.000000000000001$$

MATLAB Exercises Given the real number a and its approximation A, calculate the absolute error Δa and the relative error δa, then determine the number of correct decimal digits and correct significant digits of A.

Esempio 0.9.

$$a = 123.1256 \quad \text{and} \quad A = 123.12555551$$

```
>> a=123.1256; A=123.12555551;
>> absErr=abs(a-A)
absErr =
     4.449000000761316e-005
>> relErr=absErr/abs(a)
relErr =
     3.613383407480911e-007
```

Using normalized representation:

$$\text{absErr} = 4.449\ldots 10^{-5} = 0.4449\ldots 10^{-4}$$

$$\text{relErr} = 3.613\ldots 10^{-7} = 0.3613\ldots 10^{-6}$$

Therefore, A has at least 4 correct decimal digits and at least 7 correct significant digits. In reality, it has exactly 4 correct decimal digits and 7 correct significant digits.

Esempio 0.10. $a = 0.11$ and $A = 0.11111112$

```
>> a=0.11; A=0.11111112;
>> absErr=abs(a-A)
absErr =
     1.111119999999993e-003
>> relErr=absErr/abs(a)
relErr =
     1.010109090909085e-002
```

Using normalized representation:

$$\text{absErr} = 1.111\ldots 10^{-3} = 0.1111\ldots 10^{-2}$$

$$\text{relErr} = 1.010\ldots 10^{-2} = 0.1010\ldots 10^{-1}$$

Therefore, A has at least 2 correct decimal digits and at least 2 correct significant digits. In reality, it has exactly 2 correct decimal digits and 2 correct significant digits.

Esempio 0.11. $a = 1000.212324$ and $A = 1000.2123231$

```
>> a=1000.212324; A=1000.2123231;
>> absErr=abs(a-A)
absErr =
     8.999999181469320e-007
>> relErr=absErr/abs(a)
relErr =
     8.998088671290278e-010
```

Using normalized representation:

$$\text{absErr} = 8.999\ldots 10^{-7} = \frac{1}{2}0.1799\ldots 10^{-5} < \frac{1}{2}10^{-5}$$

$$\text{relErr} = 8.998\ldots 10^{-10} = \frac{1}{2}0.1799\ldots 10^{-8} < \frac{1}{2}10^{-8}$$

Therefore, A has at least 5 correct decimal digits and at least 9 correct significant digits. In fact, it has exactly 5 correct decimal digits and 9 correct significant digits.

The `eps` variable in MATLAB In MATLAB, the `eps` variable contains the value:

$$\varepsilon = 2^{-52} = 2.22044604925031e - 016$$

```
>> eps
ans =
     2.22044604925031e-016
```

We can verify that ε is the "gap" between the machine numbers 1.000000000000000 and 1.000000000000001 using its characterization (remembering the program returns 1 for true and 0 for false).

```
>> 1+eps>1
ans =
     1
>> 1+eps/2>1
ans =
     0
```

However, note that, contrary to what we might expect:

```
>> 1+eps
ans =
     1.00000000000000e+000
```

This is due to a display error for the number $1 + eps$. Instead, meeting our expectations:

```
>> 1+eps/2
ans =
     1
```

In MATLAB, there is a routine that allows calculating the maximum relative error made when approximating any machine number A.

```
>> x=10.23;
>> eps(x)
ans =
     1.77635683940025e-015
>> x+eps(x)>x
ans =
     1
>> x+eps(x)/2>x
ans =
     0
```

Obviously:

```
>> eps(1)
ans =
     2.22044604925031e-016
```

meaning `eps(1) = eps`.

Machine Operations and Numerical Cancellation

Third consequence of fixed-length word representation - Calculation errors An operation between two or more machine numbers does not always produce a result that is itself a machine number. Situations of overflow or underflow may occur. Furthermore, the normalized form of the result might have more decimal digits than the precision allows. Therefore, it is impossible to implement exact arithmetic operations on a computer. We must settle for so-called machine operations: two machine numbers are associated with a third machine number obtained by rounding the exact result of the arithmetic operation.

Let a and b be two real numbers and $A = fl(a)$ and $B = fl(b)$ be the corresponding machine numbers. Denoting $\oplus, \ominus, \otimes, \oslash$ as the machine operations corresponding to $+, -, \times, /$, we have:

$$A \oplus B = fl(A + B) = (A + B)(1 + \delta_1)$$

$$A \ominus B = fl(A - B) = (A - B)(1 + \delta_2)$$
$$A \otimes B = fl(A \times B) = (A \times B)(1 + \delta_3)$$
$$A \oslash B = fl(A/B) = (A/B)(1 + \delta_4)$$

with

$$|\delta_i| \leq u \leq \varepsilon, \quad i \in \{1, 2, 3, 4\}$$

Thus, when performing a machine operation, an error of the order of the machine precision ε is introduced.

Nota 4. The previous results hold for the IEEE 754 standard, which uses so-called guard bits; to perform operations, machine numbers are stored in specific registers with additional bits to reduce the effects of round-off errors. In such a system, we also have:

$$fl(\sqrt{A}) = \sqrt{A}(1 + \delta), \quad |\delta| \leq \varepsilon.$$

Using these results, it is possible to theoretically estimate errors in all expressions. For example, if X, Y, and Z are machine numbers:

$$\begin{aligned} X \otimes (Y \oplus Z) = X \otimes fl(Y + Z) &= fl(X \times fl(Y + Z)) \\ &= (X \times fl(Y + Z)) \times (1 + \delta_1) \\ &= (X \times (Y + Z) \times (1 + \delta_2)) \times (1 + \delta_1) \\ &\approx X \times (Y + Z) \times (1 + \delta_1 + \delta_2) \end{aligned}$$

neglecting the product $\delta_1\delta_2$ as it is negligible. Thus:

$$X \otimes (Y \oplus Z) \approx X \times (Y + Z)(1 + \delta_3), \quad \delta_3 = \delta_1 + \delta_2, \quad |\delta_3| \leq \varepsilon$$

Known properties (commutative, associative, distributive, etc.) of arithmetic operations do not always hold for machine operations. The commutative property remains:

$$A \oplus B = B \oplus A \quad \text{and} \quad A \otimes B = B \otimes A$$

while, in general:

$$(A \oplus B) \oplus C \neq A \oplus (B \oplus C)$$
$$(A \otimes B) \otimes C \neq A \otimes (B \otimes C)$$
$$A \otimes (B \oplus C) \neq (A \otimes B) \oplus (A \otimes C)$$

Another anomalous relationship is:

$$A \oplus B = A, \quad \text{if } |B| \ll |A|$$

therefore, the additive identity is not unique.

Esempio 0.12. $N = 10, t = 16$

$$a = \exp(18)$$
$$b = \exp(-20)$$
$$A = fl(a) = 0.6565996913733051 \cdot 10^8$$
$$B = fl(b) = 0.2061153622438558 \cdot 10^{-8}$$
$$A \oplus B = fl(A + B) = 0.6565996913733051 \cdot 10^8 = A$$

This happens because when summing machine numbers with different exponents, the computer follows these steps:

1. identify the number with the largest exponent $\overline{q}$;
2. store all other numbers using their representation with exponent $\overline{q}$;
3. perform the sum.

In the previous example:

$$\begin{aligned} B = fl(b) &= 0.2061153622438558 \cdot 10^{-8} \\ &= 0.00000000000000002061153622438558 \cdot 10^8 \end{aligned}$$

and, rounding to 16 digits:

$$B = fl(b) = 0$$

Summation of multiple numbers Due to alignment errors, when summing multiple positive numbers of different orders of magnitude, it is advisable to proceed in ascending order (from smallest to largest) to ensure smaller values contribute to the final sum.

Esempio 0.13. Given the numbers in base 10:

0.8999e+4	0.7889e+3	0.7767e+3
0.7555e+2	0.6266e+2	0.4298e+1
0.2581e+1	0.2653e+0	0.1580e+0

Assuming floating-point arithmetic with $t = 4$, sum the numbers in ascending and descending order and compare the results with the exact value $s = 0.107101123e + 5$. First, sum in descending order:

$$s1 = fl(0.8999e + 4 + 0.7889e + 3) = 0.9788e + 4$$

$$s1 = fl(s1 + 0.7767e + 3) = 0.1056e + 5$$

$$s1 = fl(s1 + 0.7555e + 2) = 0.1064e + 5$$

$$s1 = fl(s1 + 0.6266e + 2) = 0.1070e + 5$$

$$s1 = fl(s1 + 0.4298e + 1) = 0.1070e + 5$$

$$s1 = fl(s1 + 0.2653e + 0) = 0.1070e + 5$$

$$s1 = fl(s1 + 0.2581e + 1) = 0.1070e + 5$$

$$s1 = fl(s1 + 0.1580e + 0) = 0.1070e + 5$$

Only the first 5 numbers contribute to the sum. Now, sum in ascending order:

$$s2 = fl(0.1580e + 0 + 0.2653e + 0) = 0.4233e + 0$$

$$s2 = fl(s2 + 0.2581e + 1) = 0.3004e + 1$$

$$s2 = fl(s2 + 0.4298e + 1) = 0.7302e + 1$$

$$s2 = fl(s2 + 0.6266e + 2) = 0.6996e + 2$$

$$s2 = fl(s2 + 0.7555e + 2) = 0.1455e + 3$$

$$s2 = fl(s2 + 0.7767e + 3) = 0.9222e + 3$$

$$s2 = fl(s2 + 0.7889e + 3) = 0.1711e + 4$$

$$s2 = fl(s2 + 0.8999e + 4) = 0.1071e + 5$$

All numbers contribute. Calculating relative errors:

$$\frac{|s - s1|}{|s|} = 0.944 \cdot 10^{-3} \quad \text{(descending sum)}$$

$$\frac{|s - s2|}{|s|} = 0.105 \cdot 10^{-4} \quad \text{(ascending sum)}$$

Descending sum yielded 3 correct significant digits, while ascending sum yielded 5.

Numerical Cancellation This phenomenon occurs during the subtraction of two nearly equal numbers. Let x_1 and x_2 be two real numbers. If $x = x_1 - x_2$ is very small, the relative error:

$$\delta x = \left| \frac{fl(x_1 - x_2) - x}{x} \right|$$

can be very large, leading to a loss of significant digits in calculating $fl(x_1 - x_2)$.

It is always preferable to avoid subtracting nearly equal machine numbers by using alternative expressions where possible.

For example:

1. $$\sqrt{x+\delta} - \sqrt{x} = \frac{\delta}{\sqrt{x+\delta} + \sqrt{x}}$$

2. $$\cos(x+\delta) - \cos(x) = -2\sin\left(\frac{\delta}{2}\right)\sin\left(x + \frac{\delta}{2}\right)$$

Generally, if one must compute $f(x+\delta) - f(x)$ with $|\delta| \ll |x|$, it may be better to use the Taylor expansion:

$$f(x+\delta) - f(x) = f'(x)\delta + f(x)\frac{\delta}{2} + \dots$$

Cancellation can also occur in the quadratic formula $ax^2 + bx + c = 0$:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

where it is better to use the alternative:

$$x_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}$$

$$x_2 = \frac{c}{ax_1}$$

Examples in MATLAB

Esempio 0.14. Verify that a machine operation does not always result in a machine number.

```
a = 1.000345566e+150;
b = 0.15646464e+200;
disp(a*b) % Overflow

a = 1.000345566e-25;
b = 1.56464646e-300;
disp(a*b) % Underflow
```

```
Inf

  0
```

From this last example, it is also evident that the product of two machine numbers can be zero even if both numbers are non-zero; that is, the **zero-product property** does not hold.

Esempio 0.15. Let $a = 0.23371258 \cdot 10^{-4}$, $b = 0.33678429 \cdot 10^{2}$, and $c = -0.33677811 \cdot 10^{2}$; verify that $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$.

```
format long e;

% Data
a = 0.23371258e-4;
b = 0.33678429e+2;
c = -0.33677811e+2;

% Output: [a, b, c]
fprintf('a: %.15e\nb: %.15e\nc: %.15e\n', a, b, c);

% Calculations
s1 = (a + b) + c;
s2 = a + (b + c);

% Output: [s1, s2, difference]
fprintf('(a + b) + c: %.15e\na + (b + c): %.15e\nDifference between the two sums: %.15e\n', s1
    , s2, s1-s2);
```

```
a: 2.337125800000000e-05
b: 3.367842900000000e+01
c: -3.367781100000000e+01
(a + b) + c: 6.413712580055630e-04
a + (b + c): 6.413712580028940e-04
Difference between the two sums: 2.669088908224815e-15
```

Esempio 0.16. Let $a = 0.1234567$, $b = 6666.325$, and $c = -6666.325$; verify that $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$.

```
format long e;

% Data
a = 0.1234567;
b = 6666.325;
c = -6666.325;

% Output: [a, b, c]
fprintf('a: %.15e\nb: %.15e\nc: %.15e\n', a, b, c);

% Calculations
s1 = (a + b) + c;
s2 = a + (b + c);

% Output: [s1, s2, difference]
fprintf('(a + b) + c: %.15e\na + (b + c): %.15e\nDifference between the two sums: %.15e\n', s1
    , s2, s1-s2);
```

```
a: 1.234567000000000e-01
b: 6.666325000000000e+03
c: -6.666325000000000e+03
(a + b) + c: 1.234567000001334e-01
a + (b + c): 1.234567000000000e-01
Difference between the two sums: 1.334210519843282e-13
```

Esempio 0.17. Let $a = 0.8e{+}9$, $b = 0.500009e{+}4$, and $c = -0.500008e{+}4$; verify that $a \otimes (b \oplus c) \neq a \otimes b \oplus a \otimes c$.

```
format long e;

% Data
a = 0.8e+9;
b = 0.500009e+4;
c = -0.500008e+4;

% Output: [a, b, c]
fprintf('a: %.15e\nb: %.15e\nc: %.15e\n', a, b, c);

% Calculations
s1 = a * (b + c);
s2 = a * b + a * c;
```

```
% Output: [s1, s2, difference]
fprintf('a * (b + c): %.15e\na * b + a * c: %.15e\nDifference between the two operations: %.15e\n', s1, s2, s1-s2);
```

```
a: 8.000000000000000e+08
b: 5.000090000000000e+03
c: -5.000080000000000e+03
a * (b + c): 8.000000000174623e+06
a * b + a * c: 8.000000000000000e+06
Difference between the two operations: 1.746229827404022e-04
```

Indeed:

```
format long e;

% Data
a = 0.8e+9;
b = 0.500009e+4;
c = -0.500008e+4;

% Calculations
s1 = a * b;
s2 = a * c;
s3 = a * b + a * c;

% Output: [s1, s2, s3]
fprintf('a * b: %.15e\na * c: %.15e\na * b + a * c: %.15e\n', s1, s2, s3);
```

```
a * b: 4.000072000000000e+12
a * c: -4.000064000000000e+12
a * b + a * c: 8.000000000000000e+06
```

Esempio 0.18. Let $a = 7.45700244034177e-001$, $b = 3.77400852642836e-001$, and $c = 7.62332001118890e-001$; verify that $(a \otimes b) \otimes c \neq a \otimes (b \otimes c)$.

```
format long e;

% Data
a = 7.45700244034177e-001;
b = 3.77400852642836e-001;
c = 7.62332001118890e-001;

% Output: [a, b, c]
fprintf('a: %.15e\nb: %.15e\nc: %.15e\n', a, b, c);

% Calculations
p1 = (a * b) * c;
p2 = a * (b * c);

% Output: [s1, s2, difference]
fprintf('(a * b) * c: %.15e\na * (b * c): %.15e\nDifference between the two products: %.15e\n', p1, p2, p1-p2);
```

```
a: 7.457002440341770e-01
b: 3.774008526428360e-01
c: 7.623320011188900e-01
(a * b) * c: 2.145415002111401e-01
a * (b * c): 2.145415002111401e-01
Difference between the two products: 2.775557561562891e-17
```

We observe that the values $p1$ and $p2$ result in being equal even if the corresponding machine numbers are not (their difference is non-zero). This occurs due to a display error or a binary-to-decimal conversion error.

Esempio 0.19. Let us verify that given $c = 0.164646415647 \cdot 10^{+9}$ and $d = 0.38731646 \cdot 10^{-7}$, we have $c \oplus d = fl(c + d) = fl(c)$.

```
>> c = 0.164646415647e+9;
>> d = 0.38731646e-7;
>> c+d
ans =
     1.646464156470000e+008
```

Esempio 0.20. Verify that the equality

$$\sqrt{x+\delta} - \sqrt{x} = \frac{\delta}{\sqrt{x+\delta} + \sqrt{x}}$$

is not true when working in finite arithmetic. Assume $x = 4$ and $\delta = 0.002$.

```
>> a = sqrt(4.002)
a =
     2.000499937515620e+000
>> b = sqrt(4)
b =
     2
>> a-b
ans =
     4.999375156202746e-004
>> 0.002/(a+b)
ans =
     4.999375156201189e-004
```

With the first formula, 4 correct significant digits are lost.

Esempio 0.21. Verify that the equality

$$\cos(x+\delta) - \cos(x) = -2\sin\left(\frac{\delta}{2}\right)\sin\left(x + \frac{\delta}{2}\right)$$

is not true when working in finite arithmetic. Assume $x = \pi$ and $\delta = 0.0001$.

```
>> cos(pi+0.0001)-cos(pi)
ans =
     4.99999969612645e-009
>> -2*sin(0.0001/2)*sin(pi+0.0001/2)
ans =
     4.999999995809435e-009
```

With the first formula, 8 correct significant digits are lost.

Esempio 0.22. Given the quadratic equation

$$x^2 - 0.4002x + 0.8e - 4 = 0,$$

calculate the roots using the formulas:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a},$$

and

$$x_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{c}{ax_1}$$

and compare the results obtained with the exact values $x_1 = 0.4$ and $x_2 = 0.0002$. First, observe that if we calculate the value of the quadratic polynomial at its roots, we do not obtain 0 but one of the infinite machine zeros.

```
>> x = 0.4;
>> x^2 - 0.4002*x + 0.8e-4
ans =
     3.102173466024150e-017
>> x = 0.0002;
>> x^2 - 0.4002*x + 0.8e-4
ans =
     1.355252715606881e-020
```

Thus, the equation is numerically verified within an error of the order of machine epsilon.

```
>> a = 1; b = -0.4002; c = 0.8e-4;
>> x1 = (-b + sqrt(b^2 - 4*a*c))/(2*a)
x1 =
     4.000000000000000e-001
>> x2 = (-b - sqrt(b^2 - 4*a*c))/(2*a)
x2 =
     1.999999999999780e-004
>> x22 = c/x1
x22 =
     2.000000000000000e-004
```

It is evident that with the first formula, 3 correct significant digits are lost.

Esempio 0.23. Evaluate the analytically equivalent expressions

$$y_1 = \frac{1-\cos(t)}{t^2} \quad \text{and} \quad y_2 = \frac{1}{2}\frac{\sin^2(t/2)}{(t/2)^2}$$

at the point $t = 1.2e-5$.

```
>> t = 1.2e-5;
>> y1 = (1-cos(t))/t^2
y1 =
     4.999997329749008e-001
>> y2 = 0.5*(sin(t/2)/(t/2))^2
y2 =
     4.999999999940000e-001
```

The difference in the results obtained is due to the phenomenon of numerical cancellation (catastrophic cancellation) occurring in the calculation of $1-\cos(t)$, because, with the chosen t, 1 and $\cos(t)$ are nearly equal. Since the expression y_2 is defined by operations that do not introduce any loss of precision, we can assume that the value y_2 has 15 correct significant digits. Therefore, by comparing y_1 with y_2, we deduce that only 6 correct significant digits are obtained with expression y_1.

Esempio 0.24. Evaluate the analytically equivalent expressions

$$y_1 = (1-x)^6 \quad \text{and} \quad y_2 = x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1$$

at 100 equidistant points in the interval $[1-\delta, 1+\delta]$ for $\delta = 0.1, 0.01, 0.005, 0.0025$. Graphically represent y_1 and y_2 for each assigned value of δ.

```
>> delta = 0.1;
>> x = linspace(1-delta, 1+delta);
>> y1 = (1-x).^6;
>> y2 = x.^6 - 6*x.^5 + 15*x.^4 - 20*x.^3 + 15*x.^2 - 6*x + 1;
>> figure(1)
>> plot(x, y1, x, y2, 'r')
```

The plot for $\delta = 0.1$ is obtained.

```
>> delta = 0.01;
>> x = linspace(1-delta, 1+delta);
>> y1 = (1-x).^6;
>> y2 = x.^6 - 6*x.^5 + 15*x.^4 - 20*x.^3 + 15*x.^2 - 6*x + 1;
>> figure(2)
>> plot(x, y1, x, y2, 'r')
```

The plot for $\delta = 0.01$ is obtained.

```
>> delta = 0.005;
>> x = linspace(1-delta, 1+delta);
>> y1 = (1-x).^6;
>> y2 = x.^6 - 6*x.^5 + 15*x.^4 - 20*x.^3 + 15*x.^2 - 6*x + 1;
>> figure(3)
>> plot(x, y1, x, y2, 'r')
```

The plot for $\delta = 0.005$ is obtained.

```
>> delta = 0.0025;
>> x = linspace(1-delta, 1+delta);
>> y1 = (1-x).^6;
>> y2 = x.^6 - 6*x.^5 + 15*x.^4 - 20*x.^3 + 15*x.^2 - 6*x + 1;
>> figure(4)
>> plot(x, y1, x, y2, 'r')
```

The plot for $\delta = 0.0025$ is obtained.

Observing the previous plots, we deduce that the graphical representations obtained using the expressions y_1 and y_2 deviate from each other increasingly as δ decreases. In fact, while y_1 remains faithful to the real behavior of the polynomial, y_2 shows an increasingly oscillating behavior (as if it had many zeros!).

These oscillations are due to the phenomenon of numerical cancellation (catastrophic cancellation) that occurs when, in y_2, the sum between

$$x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x \quad \text{and} \quad 1$$

is performed for values very close to 1 ($x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x < 0$ for every $x \in [-1, 1]$). We observe that the aforementioned phenomenon, although not graphically evident, also occurs when the subtraction between 1 and x is performed in y_1 for values of x very close to 1. However, since the quantities

$$(1 - x)^6 \ll (1 - x)$$

the quantities

$$x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x \quad \text{and} \quad 1$$

have more common digits than the quantities 1 and x. Consequently, the evaluation of y_2 results in a greater loss of precision compared to y_1.

0.1 Propagation of Errors Introduced in Initial Data

Conditioning of a problem The numerical cancellation example introduces a very significant concept in Scientific Computing: the conditioning of a problem. In solving a mathematical problem, it is in many cases useful to have a measure of how sensitive the results are to small changes in the initial data. In general, if small changes in the initial data correspond to large changes in the final results, we say that the problem is **ill-conditioned**. Otherwise, we say it is **well-conditioned**. The quantity responsible for the amplification of errors in the initial data is called the **condition number**. Conditioning is an inherent characteristic of the mathematical problem itself and has no link to machine operation round-off errors or to the algorithms chosen to solve it. Determining the condition number of a problem is not always easy.

Let us consider the simple case of the numerical problem of summation. Let x, y, and z be three real numbers with

$$z = x + y$$

Suppose we perturb x by Δx and y by Δy, and let

$$\overline{z} = (x + \Delta x) + (y + \Delta y)$$

Let us calculate the relative error committed by approximating z with $\overline{z}$:

$$\begin{aligned}
\delta z = \frac{|z - \overline{z}|}{|z|} &= \frac{|(x+y) - (x + \Delta x + y + \Delta y)|}{|x+y|} = \frac{|-\Delta x - \Delta y|}{|x+y|} = \frac{|\Delta x + \Delta y|}{|x+y|} \\
&\leq \frac{|\Delta x| + |\Delta y|}{|x+y|} = \frac{1}{|x+y|}\left(|\Delta x| \cdot \frac{|x|}{|x|} + |\Delta y| \cdot \frac{|y|}{|y|}\right) \\
&= \frac{1}{|x+y|}\left(|x| \underbrace{\frac{|\Delta x|}{|x|}}_{\delta x} + |y| \underbrace{\frac{|\Delta y|}{|y|}}_{\delta y}\right) = \frac{1}{|x+y|}(|x|\delta x + |y|\delta y) \\
&\leq \frac{1}{|x+y|}\left(\max\{|x|, |y|\}\delta x + \max\{|x|, |y|\}\delta y\right) = \frac{\max\{|x|, |y|\}}{|x+y|}(\delta x + \delta y)
\end{aligned}$$

The quantity

$$\frac{\max\{|x|, |y|\}}{|x+y|}$$

represents the **condition number**. This means that if we sum two numbers that are very close and of opposite sign, even if the relative errors δx and δy are small, the value $\overline{z}$ will approximate z with a large relative error; i.e., the summation operation becomes ill-conditioned and numerical cancellation occurs. In general, denoted by $f(x)$ a generic numerical problem having x as initial data, we seek to determine a relationship of the type

$$\frac{|f(x) - f(\overline{x})|}{|f(x)|} \leq K \frac{|\Delta x|}{|x|}$$

where K represents the condition number of the problem f. The smaller K is, the more well-conditioned the problem is.

Esempio 0.25. Consider the linear system of two equations in two unknowns

$$\begin{cases} x_1 - x_2 = 1 \\ x_1 - 1.00001x_2 = 0 \end{cases}$$

whose exact solutions are

$$x_1 = 100001, \quad x_2 = 100000$$

We perturb by a small amount equal to $0.2 \cdot 10^{-4}$ the coefficient of the unknown x_2 in the second equation. Its new coefficient will be $-1.00001 + 0.2 \cdot 10^{-4} = -0.99999$. The new system becomes

$$\begin{cases} y_1 - y_2 = 1 \\ y_1 - 0.99999y_2 = 0 \end{cases}$$

and its solutions are

$$y_1 = -99999, \quad y_2 = -100000$$

Observations

- Attempting to solve the previous systems with Cramer's rule or the substitution method results in solutions different from those indicated. To verify that the indicated solutions are correct, simply substitute them into each row of the system.
- Since the solutions were derived working in infinite arithmetic, the only reason why such different results were obtained following the small perturbation is that the resolution of the starting system is an ill-conditioned problem.

Stability of an algorithm

Definizione 0.9. An algorithm, for a numerical problem, is a finite sequence of unambiguous operations that transforms input data and provides one or more output values. We say an algorithm is **unstable** if it amplifies round-off errors during the computation of a solution for a well-conditioned problem. Therefore, the concept of algorithm stability is strictly connected to finite calculation precision and the operations through which the algorithm itself is resolved.

Esempio 0.26. Now consider the following linear system of two equations in two unknowns

$$\begin{cases} \alpha x_1 + x_2 = 1 + \alpha \\ x_1 = 1 \end{cases}$$

whose exact solution is

$$x_1 = x_2 = 1$$

It can be shown that the problem of its resolution is well-conditioned. Despite the immediacy of the solution, let us follow the calculation scheme of the Gaussian elimination method. Multiplying both sides of the first equation by $-\frac{1}{\alpha}$, we obtain

$$-\frac{1}{\alpha}(\alpha x_1 + x_2) = -\frac{1}{\alpha}(1 + \alpha),$$

i.e.,

$$-x_1 - \frac{1}{\alpha}x_2 = -\frac{1+\alpha}{\alpha}$$

Adding the previous equation to the second equation ($x_1 = 1$), we obtain

$$-x_1 - \frac{1}{\alpha}x_2 + x_1 = 1 - \frac{1+\alpha}{\alpha}$$

$$-\frac{1}{\alpha}x_2 = 1 - \frac{1+\alpha}{\alpha}$$

Replacing the second equation with the one obtained, we get the following system equivalent to the previous one:

$$\begin{cases} \alpha x_1 + x_2 = 1 + \alpha \\ -\frac{1}{\alpha}x_2 = 1 - \frac{1+\alpha}{\alpha} \end{cases}.$$

Let us compute the system coefficients for $\alpha = 0.5 \times 10^{-11}$. We have

$$0.5 \times 10^{-11} x_1 + x_2 = 1 + 0.5 \times 10^{-11}$$

$$-2 \times 10^{11} x_2 = 1 - \frac{1 + 0.5 \times 10^{-11}}{0.5 \times 10^{-11}}$$

and, thus, performing the computation based on arithmetic operator precedence:

$$\begin{cases} x_2 = \frac{1}{-2\times 10^{11}}\left(1 - \frac{(1+0.5\times 10^{-11})}{0.5\times 10^{-11}}\right) = 1 \\ x_1 = \frac{(1+0.5\times 10^{-11}-1)}{0.5\times 10^{-11}} = 1.000000082740371 \end{cases}$$

Now we swap the equations (pivoting):

$$\begin{cases} x_1 = 1 \\ \alpha x_1 + x_2 = 1 + \alpha \end{cases}$$

and repeat the procedure to eliminate the unknown x_1 from the second equation. Multiplying both sides of the first equation by $-\alpha$, we obtain

$$-\alpha x_1 = -\alpha$$

Adding the previous equation to the second equation, we obtain

$$-\alpha x_1 + \alpha x_1 + x_2 = -\alpha + 1 + \alpha$$

$$x_2 = 1$$

Thus

$$\begin{cases} x_1 = 1 \\ x_2 = 1 \end{cases}.$$

This procedure which, under appropriate conditions, involves swapping the rows of a linear system is called **pivoting**. The cause of this bizarre behavior is to be found in the sequence of operations that lead us from initial data to results, i.e., in the algorithm. The Gaussian elimination method is generally not stable, but if combined with pivoting, it is always stable.

We stated that the problem of summation between two very close numbers is ill-conditioned and generates numerical cancellation. For this type of problem, it is even more important to choose a stable algorithm.

If we consider the problem of the difference between two radicals, we have seen that

$$\sqrt{x+\delta} - \sqrt{x} = \frac{\delta}{\sqrt{x+\delta} + \sqrt{x}}$$

As we have already verified with numerical tests, the algorithm implementing the formula on the left is unstable while the one implementing the formula on the right is always stable.

A similar argument can be made for:

- the calculation of the difference between two cosines using the equivalent formulas:

  $$\cos(x+\delta) - \cos(x)$$

  and

  $$-2\sin\left(\frac{\delta}{2}\right)\sin\left(x+\frac{\delta}{2}\right)$$

- the calculation of the roots of the equation $ax^2 + bx + c = 0$ using the equivalent formulas:

  $$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

  and

  $$x_1 = \frac{-b - \mathrm{sign}(b)\sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{c}{ax_1}$$

We have already verified through numerical tests that for both, the algorithm implementing the first formula is unstable while the one implementing the second is always stable.

Therefore, error control essentially passes through two phases:

1. study of the mathematical problem's conditioning;
2. use of a stable algorithm for solving the problem.

Conclusions

- Any algorithm applied to an ill-conditioned problem will generate amplification of data representation errors.
- Round-off error propagation will occur when applying an unstable algorithm to a well-conditioned problem.
- Maximum calculation precision is achieved when a stable algorithm is applied to a well-conditioned problem.
- Sometimes it may happen that a problem is well-conditioned for certain initial data and ill-conditioned for others (see the summation operation).

The goal of mathematicians dealing with Scientific Computing is to solve numerical problems, whose conditioning they have studied a priori, using algorithms that are as stable as possible.

MATLAB Exercises

Esempio 0.27. Let us solve the system

$$\begin{cases} x_1 - x_2 = 1 \\ x_1 - 1.00001x_2 = 0 \end{cases}$$

We define the system matrix:

```
>> A = [1, -1; 1, -1.00001]
A =
   1.000000000000000e+000  -1.000000000000000e+000
   1.000000000000000e+000  -1.000010000000000e+000
```

We define the vector of known terms:

```
>> b = [1; 0]
b =
     1
     0
```

Let us verify that the vector $[100001, 100000]'$ is the exact solution of the system:

```
>> A * [100001; 100000]
ans =
     1
     0
```

We solve the system $Ax = b$ using the MATLAB backslash function:

```
>> x = A\b
x =
   1.00009999993449e+005
   9.99999999934487e+004
```

Let us calculate the relative errors:

```
>> err1 = abs(100001 - 1.000009999993449e+005)/abs(100001)
err1 =
   6.550915688349306e-12
>> err2 = abs(100000 - 9.999999999934487e+004)/abs(100000)
err2 =
   6.551272235810757e-12
```

We have

$$\text{err1} = 0.655\ldots 10^{-11} < \frac{1}{2}10^{-10}$$

$$\text{err2} = 0.655\ldots 10^{-11} < \frac{1}{2}10^{-10}$$

that is, both solutions have been calculated with only 11 correct significant digits. The MATLAB backslash function solves linear systems using a stable algorithm that implements Gaussian elimination with pivoting. Therefore, the loss of significant digits is due to the ill-conditioning of the system matrix. If A denotes the coefficient matrix of a linear system, the command `cond(A,inf)` in MATLAB allows calculating the condition number of the system resolution problem.

```
>> C = cond(A,inf)
C =
   4.000040000073795e+005
```

Since the perturbations we induce on the initial data A and $\mathbf{b}$ of the problem are of the order of machine epsilon, by multiplying the obtained condition number by the MATLAB variable `eps`, we obtain an estimate of the error with which we will calculate the solution x.

```
>> deltax = C*eps
deltax =
   8.881873015007080e-011
```

that is

$$\text{deltax} = 0.888\ldots 10^{-10} < \frac{1}{2}10^{-9}$$

The previous estimate tells us that when solving a linear system having A as the coefficient matrix with a stable algorithm, we can obtain at least 10 correct significant digits. This estimate is confirmed by the numerical tests; in fact, we have 11 digits.

Esempio 0.28. Let us solve the system

$$\begin{cases} y_1 - y_2 = 1 \\ y_1 - 0.99999y_2 = 0 \end{cases}$$

We define the system matrix:

```
>> A = [1, -1; 1, -0.99999]
A =
   1.000000000000000e+000  -1.000000000000000e+000
   1.000000000000000e+000  -9.999900000000000e-001
```

We define the vector of known terms:

```
>> b = [1; 0]
```

Let us verify that the vector $[-99999, -100000]'$ is the exact solution of the system:

```
>> A * [-99999; -100000]
ans =
     1
     0
```

We solve the system $Ax = b$ using the MATLAB backslash function:

```
>> x = A\b
x =
  -9.999900000045510e+004
  -1.000000000004551e+005
```

Let us calculate the relative errors:

```
>> err1 = abs(-99999 + 9.999900000045510e+004)/abs(-99999)
err1 =
   4.551011478634235e-012
>> err2 = abs(-100000 + 1.000000000004551e+005)/abs(-100000)
err2 =
   4.550965968519449e-12
```

We have

$$\text{err1} < \frac{1}{2}10^{-11}, \quad \text{err2} < \frac{1}{2}10^{-11}$$

that is, both solutions have been calculated with only 12 correct significant digits.

Let us calculate the condition number of the problem.

```
>> C = cond(A,inf)
C =
   4.000000000018204e+005
```

Error estimate:

```
>> deltax = C*eps
deltax =
   8.881784197041673e-011
```

that is deltax $< \frac{1}{2}10^{-9}$. Thus, when solving a linear system having A as the coefficient matrix with a stable algorithm, we can obtain at least 10 correct significant digits. We obtained 12.

Esempio 0.29. Let us solve the following linear system with $\alpha = 0.5 \times 10^{-11}$

$$\begin{cases} \alpha x_1 + x_2 = 1 + \alpha \\ x_1 = 1 \end{cases}$$

Its exact solution is $x_1 = x_2 = 1$ for every choice of α. We define the system matrix and the vector of known terms:

```
>> al = 0.5e-11;
>> A = [al, 1; 1, 0];
>> b = [1+al; 1];
```

Let us calculate the condition number of the problem:

```
>> C = cond(A,inf)
C =
   1.000000000010000e+000
>> deltax = C*eps
deltax =
   2.220446049272518e-016
```

From which we deduce that the system resolution problem is well-conditioned, and if we solve it with a stable algorithm, we can obtain a solution with 16 correct significant digits.

We have seen that if we solve it applying the Gauss method (unstable in this case), we obtain:

```
>> x1 = (1 + 0.5e-11 - 1)/0.5e-11
x1 =
   1.000000082740371e+000
>> x2 = (1 - (1 + 0.5e-11)/0.5e-11)/(-2.e+11)
x2 =
     1
```

Thus, if a well-conditioned problem is solved with an unstable algorithm, poorly accurate solutions are obtained.

If, instead, we solve it with the MATLAB backslash function, which implements Gaussian elimination with pivoting (stable), we obtain:

```
>> A\b
ans =
     1
     1
```

Thus, if a well-conditioned problem is solved with a stable algorithm, highly accurate solutions are obtained.

Esempio 0.30. Let $a = 1.4e154$ and $b = 1.3e154$. Calculate the equivalent expressions

$$(a^2 - b^2) \quad \text{and} \quad (a-b)(a+b)$$

and compare the results.

```
>> a = 1.4e154; b = 1.3e154;
>> (a^2 - b^2)
ans =
   Inf
>> (a-b)*(a+b)
ans =
   2.700000000000000e+307
```

It can be seen that with the first algorithm, a result is obtained that is not a machine number (overflow). This occurs because

```
>> a^2
ans =
   Inf
```

We can conclude that the second algorithm is more stable than the first.

1 Resolution of a Square Linear System

1.1 The Problem of Solving a Linear System

Generalities on Linear Systems Many problems in engineering, physics, chemistry, computer science, and economics are modeled using linear equations, i.e., equations in which the unknowns appear with an exponent equal to one and are linked together by linear relations (multiplication by scalars and algebraic sums). A system of n linear equations in n unknowns is defined as follows:

$$\sum_{j=1}^{n} a_{i,j} x_j = b_i \quad i \in \{1, \dots, n\}$$

or:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n = b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n = b_2 \\ \vdots \\ a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,n}x_n = b_n \end{cases}$$

We will assume that all quantities involved are real numbers.

Matrix Form Letting A be the coefficient matrix, $\mathbf{b}$ the vector of known terms, and $\mathbf{x}$ the vector of solutions:

$$A = \begin{pmatrix} a_{1,1} & \dots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \dots & a_{n,n} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$$

the system can be rewritten as:

$$A\mathbf{x} = \mathbf{b}$$

where $A \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$, and $\mathbf{x} \in \mathbb{R}^n$.

Existence and Uniqueness of the Solution It is known that the system admits a unique solution if and only if the matrix A is non-singular, or equivalently if $\det(A) \neq 0$. In this case, denoting by A^{-1} the inverse matrix of A, we have:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

The inverse matrix is indeed the unique matrix for which $A^{-1}A = AA^{-1} = I$, where I is the identity matrix.

Cramer's Method Linear Algebra theory proposes Cramer's method as a solution method:

$$x_i = \frac{\det(A_i)}{\det(A)}, \quad i \in \{1, \dots, n\}$$

where A_i denotes the matrix obtained from A by replacing the i-th column with the vector $\mathbf{b}$. The determinants can be calculated using Laplace's rule:

$$\det(A) = \sum_{j=1}^{n} (-1)^{j+1} a_{1,j} \det(A_{1,j})$$

where $A_{1,j}$ is the matrix of order $n-1$ obtained from A by eliminating the first row and the j-th column.

Computational Cost and the Need for Numerical Methods Unfortunately, Cramer's method has a very high computational cost, equal to approximately $[(n+1)(n-1)n]!$ elementary operations (flops). This translates to unacceptable actual computation times. Assuming, for example, the capability to perform 10^6 flops per second, solving a system of 20 equations would take about $3 \cdot 10^7$ years (300,000 centuries). Even classical methods for calculating the inverse A^{-1} have a factorial computational cost. Since real-world problems can involve systems with hundreds or thousands of equations, efficient numerical methods are needed that take into account the dimension and properties of the matrix.

Dense Matrix

Definizione 1.1. A matrix $A \in \mathbb{R}^{n \times n}$ is called **dense** if most of its elements are non-zero.

Sparse Matrix

Definizione 1.2. A matrix $A \in \mathbb{R}^{n \times n}$ is called **sparse** if it possesses a number of non-zero elements of the order of n. To store a sparse matrix, it is possible to use only three vectors: one for the non-zero elements, one for the row indices, and one for the column indices.

Structured Matrices

Definizione 1.3. A matrix $A \in \mathbb{R}^{n \times n}$ is called **structured** if its elements are arranged according to a known rule.

- **Diagonal Matrix:** $a_{i,j} = 0$ for $i \neq j$.

$$\begin{pmatrix} a_{1,1} & 0 & \dots & 0 \\ 0 & a_{2,2} & \dots & \vdots \\ \vdots & \vdots & \ddots & 0 \\ 0 & \dots & 0 & a_{n,n} \end{pmatrix}$$

- **Lower Triangular Matrix:** $a_{i,j} = 0$ for $i < j$.

$$\begin{pmatrix} a_{1,1} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \dots & a_{n,n} \end{pmatrix}$$

- **Upper Triangular Matrix:** $a_{i,j} = 0$ for $i > j$.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ 0 & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{n,n} \end{pmatrix}$$

- **Tridiagonal Matrix:** non-zero elements only on the main diagonal, the super-diagonal, and the sub-diagonal.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \dots & 0 & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

- **Lower Hessenberg:** $a_{i,j} = 0$ for $j > i + 1$ (zero elements above the first super-diagonal).

$$\begin{pmatrix} a_{1,1} & a_{1,2} & 0 & \dots & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ a_{n-1,1} & a_{n-1,2} & \dots & a_{n-1,n-1} & a_{n-1,n} \\ a_{n,1} & a_{n,2} & \dots & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

- **Upper Hessenberg:** $a_{i,j} = 0$ for $i > j + 1$ (zero elements below the first sub-diagonal).

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n-1} & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n-1} & a_{2,n} \\ 0 & a_{3,2} & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & a_{n-1,n-1} & a_{n-1,n} \\ 0 & \dots & 0 & a_{n,n-1} & a_{n,n} \end{pmatrix}$$

Symmetric Matrix

Definizione 1.4. A matrix A is called **symmetric** if it coincides with its transpose, i.e., $A = {}^tA$.

```
function isSym = isSymm(A, tol)
%ISSYMM Checks if a matrix is symmetric within a given tolerance.
%  isSym = ISSYMM(A) returns true if A is symmetric (within 1e-10).
%  isSym = ISSYMM(A, tol) allows specifying the tolerance.
%
%  Input:
%    A   - Square numerical matrix (double)
%    tol - (Optional) Tolerance threshold. Default: 1e-10
%
%  Output:
%    isSym - Logical true (1) if symmetric, false (0) otherwise

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
        tol (1,1) double {mustBeNonnegative} = 1e-10
    end

    %% 2. Check Dimensions
    % Extract matrix dimensions
    [r, c] = size(A);

    % Verify the matrix is square
    if r ~= c
        error('isSymm:DimError', 'The matrix must be square (Input size: %dx%d).', r, c);
    end

    %% 3. Symmetry Check
    % Calculate transpose once
    AT = A';

    % Vectorized comparison using tolerance
    isSym = all( abs(A(:) - AT(:)) <= tol );

end
```

Function isSymm

Positive Definite Matrix

Definizione 1.5. A symmetric matrix A is called **positive definite** if

$$\forall\, \mathbf{y} \in \mathbb{R}^n \ (\mathbf{y} \neq \mathbf{0}), \quad {}^t\mathbf{y}A\mathbf{y} > 0.$$

Sylvester's Criterion

Proposizione 1.1. *A symmetric matrix $A \in \mathbb{R}^{n\times n}$ is positive definite if and only if:*

$$\det(A_k) > 0, \quad k \in \{1, \dots, n\}$$

where $\det(A_k)$ is the determinant of the leading principal submatrix of order k.

```
function isPositive = isDefPos(A)
% ISDEFPOS Determines if a matrix is positive definite.
%  isPositive = ISDEFPOS(A) returns true if matrix A is positive definite
%  using Sylvester's criterion (all leading principal minors must have
%  strictly positive determinant).
%
%  Input:
%    A - Square numerical matrix (real and symmetric).
%
%  Output:
%    isPositive - Logical true (1) if symmetric, false (0) otherwise.

    %% 1 - Input Validation
    arguments
        A (:,:) {mustBeNumeric, mustBeReal}
    end

    %% 2 - Dimensions and Symmetry Check
    % Extract matrix dimensions
    [r,c] = size(A);

    % Verify matrix is square
    if r ~= c
        error('isDefPos:NonSquareMatrix', 'Input matrix must be square (%dx%d).', rows, cols);
    end

    % Symmetry check (necessary condition for real pos. def.)
    if ~isSymm(A)
        error('isDefPos:NonSymmetricMatrix', 'Matrix must be symmetric to apply the criterion.
    ');
    end

    %% 3 - Positive Definiteness Check
    % Initialization
    isPositive = true;

    % Sylvester's criterion with early exit
    for k = 1:r
        minorDet = det(A(1:k, 1:k));

        if minorDet <= 0
            isPositive = false;
            return; % Stop calculation if a minor does not satisfy the condition
        end
    end

end
```

Function isDefPos

Diagonally Dominant Matrix

Definizione 1.6. A matrix A is called **row diagonally dominant** if:

$$|a_{i,i}| > \sum_{j=1, j\neq i}^{n} |a_{i,j}|, \quad i \in \{1, \dots, n\}.$$

It is called **column diagonally dominant** if:

$$|a_{j,j}| > \sum_{i=1, i\neq j}^{n} |a_{i,j}|, \quad j \in \{1, \ldots, n\}.$$

```
function isDom = diagDomRow(A)
% DIAGDOMROW Checks if a matrix is row diagonally dominant.
%   isDom = DIAGDOMROW(A) returns true if matrix A is row diagonally dominant.
%
%   Input:
%       A - Square numerical matrix.
%
%   Output:
%       isDom - Logical true (1) if row dominant, false (0) otherwise.

    %% 1. Input Validation
    arguments
        A (:,:) {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [m, n] = size(A);
    if m ~= n
        error('diagDomRow:NonSquareMatrix', 'Matrix must be square (Input %dx%d).', m, n);
    end

    %% 3. Verify Property
    % Calculate absolute values
    absA = abs(A);

    % Extract diagonal
    d_diag = diag(absA);

    % Calculate sum of each row
    sum_rows = sum(absA,2);

    % Calculate sum of off-diagonal elements
    sum_off_diag = sum_rows - d_diag;

    % Verify condition
    isDom = true;
    for i = 1:m
        if d_diag(i) <= sum_off_diag(i)
            isDom = false;
            return; % Stop as soon as a row violating the condition is found
        end
    end

end
```

Function diagDomRow

```
function isDom = diagDomCol(A)
% DIAGDOMCOL Checks if a matrix is column diagonally dominant.
%   isDom = DIAGDOMCOL(A) returns true if matrix A is column diagonally dominant.
%
%   Input:
%       A - Square numerical matrix.
%
%   Output:
%       isDom - Logical true (1) if column dominant, false (0) otherwise.

    %% 1. Input Validation
    arguments
        A (:,:) {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [m, n] = size(A);
```

```
    if m ~= n
        error('diagDomCol:NonSquareMatrix', 'Matrix must be square (Input %dx%d).', m, n);
    end

    %% 3. Verify Property
    % Calculate absolute values
    absA = abs(A);

    % Extract diagonal
    d_diag = diag(absA);

    % Calculate sum of each column
    sum_cols = sum(absA).';

    % Calculate sum of off-diagonal elements
    sum_off_diag = sum_cols - d_diag;

    % Verify condition
    isDom = true;
    for j = 1:n
        if d_diag(j) <= sum_off_diag(j)
            isDom = false;
            return; % Stop as soon as a column violating the condition is found
        end
    end

end
```

Function diagDomCol

Vector Norms

Definizione 1.7. A **vector norm** is a function that associates to a vector $\mathbf{x} \in \mathbb{R}^n$ a real number $\|\mathbf{x}\|$ with the following properties:

1. $\|\mathbf{x}\| > 0, \forall \mathbf{x} \neq \mathbf{0}$ and $\|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0}$;
2. $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|, \forall c \in \mathbb{R}$;
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|, \forall \mathbf{y} \in \mathbb{R}^n$.

The most used norms are:

- **Infinity norm:**
$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|;$$
- **1-norm:**
$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|;$$
- **Euclidean norm (or 2-norm):**
$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} |x_i|^2} = \sqrt{{}^t\mathbf{x}\mathbf{x}}.$$

Esempio 1.1. Consider $\mathbf{x} = {}^t(1, -2, 3)$.

- $\|\mathbf{x}\|_\infty = \max\{1, 2, 3\} = 3$;
- $\|\mathbf{x}\|_1 = 1 + 2 + 3 = 6$;
- $\|\mathbf{x}\|_2 = \sqrt{1 + 4 + 9} = \sqrt{14}$.

Matrix Norms

Definizione 1.8. A **matrix norm** associates to a matrix A a real number $\|A\|$ satisfying properties analogous to vector ones, with the addition of the sub-multiplicativity property:

- $\|AB\| \leq \|A\|\|B\|, \forall B \in \mathbb{R}^{n\times n}$.

The most used matrix norms are:

- **Infinity norm:**
$$\|A\|_\infty = \max_{1\leq i\leq n} \sum_{j=1}^{n} |a_{i,j}|$$
(maximum row sum).
- **1-norm:**
$$\|A\|_1 = \max_{1\leq j\leq n} \sum_{i=1}^{n} |a_{i,j}|$$
(maximum column sum).
- **Spectral Norm (or 2-norm):**
$$\|A\|_2 = \sqrt{\rho({}^tAA)},$$
where $\rho(B)$ is the spectral radius of B (maximum modulus of eigenvalues).
- **Frobenius Norm:**
$$\|A\|_F = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{n} |a_{i,j}|^2}.$$

Esempio 1.2. Let $A = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 4 \\ 0 & 4 & 3 \end{pmatrix}$.

- $\|A\|_\infty = \max(2, 3+4, 4+3) = 7$ (row sum);
- $\|A\|_1 = \max(2, 3+4, 4+3) = 7$ (column sum);
- $\|A\|_2$: Since A is symmetric, $\|A\|_2 = \max|\lambda(A)|$. The eigenvalues are $\lambda_1 = 2$ and, from the lower block, $\lambda_{2,3} = 3 \pm 4 \implies \{7, -1\}$. Thus $\|A\|_2 = \max\{2, 7, |-1|\} = 7$.
- $\|A\|_F = \sqrt{2^2+3^2+4^2+0+4^2+3^2} = \sqrt{54} \approx 7.35$;

Compatible and Induced Norms

Definizione 1.9. A vector norm and a matrix norm are **compatible** if $\|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\|$. A matrix norm is called **natural** or **induced** by the vector norm if:
$$\|A\| = \sup_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} = \sup_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|.$$
For natural norms, $\|I\| = 1$ holds. The matrix norms $\|\cdot\|_\infty, \|\cdot\|_1$ and $\|\cdot\|_2$ are natural (induced by the respective vector norms), while the Frobenius norm is not (since $\|I\|_F = \sqrt{n}$).

Conditioning of a Linear System We now want to study the conditioning of the linear system $A\mathbf{x} = \mathbf{b}$. We ask how the problem "responds" to possible perturbations in the initial data (matrix A and vector $\mathbf{b}$). Suppose we introduce a perturbation $\Delta A \in \mathbb{R}^{n\times n}$ on the matrix A and a perturbation $\Delta\mathbf{b} \in \mathbb{R}^n$ on the vector of known

terms $\mathbf{b}$. The solution of the perturbed problem will not be $\mathbf{x}$ but $\mathbf{y} = \mathbf{x} + \Delta\mathbf{x}$, with $\Delta\mathbf{x} \in \mathbb{R}^n$. The perturbed system is:

$$(A + \Delta A)(\mathbf{x} + \Delta\mathbf{x}) = (\mathbf{b} + \Delta\mathbf{b}).$$

Theorem (Main)

Teorema 1.2. *Let $A \in \mathbb{R}^{n\times n}$ be a non-singular matrix and $\Delta A \in \mathbb{R}^{n\times n}$ such that the condition*

$$\|A^{-1}\|\|\Delta A\| \leq \frac{1}{2}$$

is satisfied for a generic induced matrix norm $\|\cdot\|$. Then the matrix $A + \Delta A$ is also non-singular and if $\mathbf{x} \in \mathbb{R}^n$ is the solution of the system $A\mathbf{x} = \mathbf{b}$ with $\mathbf{b} \in \mathbb{R}^n$ ($\mathbf{b} \neq \mathbf{0}$) and $\Delta\mathbf{x} \in \mathbb{R}^n$ verifies the perturbed system $(A + \Delta A)(\mathbf{x} + \Delta\mathbf{x}) = (\mathbf{b} + \Delta\mathbf{b})$ for $\Delta\mathbf{b} \in \mathbb{R}^n$, one has that

$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq 2K(A)\left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|}\right)$$

where $K(A) = \|A\|\|A^{-1}\|$ is the condition number of the matrix A.

To prove the previous theorem, we need the following result:

Theorem (Auxiliary)

Teorema 1.3. *Let A be a square matrix, if $\|A\| < 1$ then the matrix $I + A$ is invertible and the following inequality holds*

$$\|(I + A)^{-1}\| \leq \frac{1}{1 - \|A\|}$$

being $\|\cdot\|$ an induced matrix norm such that $\|A\| < 1$.

Proof. Since $A + \Delta A = A(I + A^{-1}\Delta A)$ and, by hypothesis,

$$\|A^{-1}\Delta A\| \leq \|A^{-1}\|\|\Delta A\| \leq \frac{1}{2} < 1$$

applying the auxiliary theorem, the matrix $I + A^{-1}\Delta A$ and therefore the matrix $A + \Delta A$, is non-singular and it results

$$\|(I + A^{-1}\Delta A)^{-1}\| \leq \frac{1}{1 - \|A^{-1}\|\|\Delta A\|} \leq 2$$

Subtracting member by member the equations $A\mathbf{x} = b$ and $(A + \Delta A)(\mathbf{x} + \Delta\mathbf{x}) = (b + \Delta\mathbf{b})$ we obtain

$$(A + \Delta A)\Delta\mathbf{x} = -\Delta A\mathbf{x} + \Delta\mathbf{b}$$

multiplying both members by A^{-1} we have

$$(I + A^{-1}\Delta A)\Delta\mathbf{x} = A^{-1}(-\Delta A\mathbf{x} + \Delta\mathbf{b})$$

from which

$$\Delta\mathbf{x} = (I + A^{-1}\Delta A)^{-1}A^{-1}(-\Delta A\mathbf{x} + \Delta\mathbf{b})$$

and passing to norms

$$\|\Delta\mathbf{x}\| \leq 2\|A^{-1}\|(\|\Delta A\|\|\mathbf{x}\| + \|\Delta\mathbf{b}\|)$$

Dividing and multiplying by $\|A\|$ on the second member, we have

$$\|\Delta\mathbf{x}\| \leq 2K(A)\left(\frac{\|\Delta A\|}{\|A\|}\|\mathbf{x}\| + \frac{\|\Delta\mathbf{b}\|}{\|A\|}\right)$$

Since by hypothesis $\mathbf{b} \neq \mathbf{0}$ and A is non-singular, it results $\|\mathbf{x}\| > 0$ so dividing both members by $\|\mathbf{x}\|$ and taking into account that $\|\mathbf{x}\| = \|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\|$, we have

$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq 2K(A)\left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta\mathbf{b}\|}{\|A\|\|\mathbf{x}\|}\right) \leq 2K(A)\left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{x}\|}\right)$$

Thus, we obtain

$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq 2K(A)\left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{x}\|}\right)$$

From the estimate, it is evident that the potential amplification of perturbations introduced on the coefficient matrix and the vector of known terms is due to the quantity $K(A) = \|A\|\|A^{-1}\|$. This quantity is called the condition number or condition index of matrix A. □

Observations on Error Estimation From the obtained estimate, some fundamental properties can be deduced:

- For any natural matrix norm $\|\cdot\|$ (satisfying the sub-multiplicativity property), we have:

$$K(A) = \|A\|\|A^{-1}\| \geq \|AA^{-1}\| = \|I\| = 1$$

- Conditioning is an inherent characteristic of the linear system and is not linked to the particular numerical method used to determine the solution.
- Setting $K_\infty(A) = \|A\|_\infty\|A^{-1}\|_\infty$, from the relative error estimate, if:

$$\frac{\|\Delta\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} \leq 2K_\infty(A)\varepsilon \leq \frac{1}{2}10^{1-p}$$

  then p significant digits of the solution calculated using a stable algorithm are to be considered correct.

Calculation of Conditioning in MATLAB In MATLAB there is a predefined function to calculate $K(A)$:

- `cond(A, inf)`: calculates the conditioning of A in infinity norm ($K_\infty(A)$).
- `cond(A, 1)`: calculates the conditioning of A in 1-norm ($K_1(A)$).
- `cond(A)`: calculates the conditioning of A in 2-norm ($K_2(A)$).

Examples of Ill-Conditioned Matrices

Hilbert Matrices

Esempio 1.3. These are matrices defined by elements $h_{i,j} = \frac{1}{i+j-1}$. The structure of matrix H_n is as follows:

$$H_n = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \cdots & \frac{1}{n+1} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \cdots & \frac{1}{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \frac{1}{n+2} & \cdots & \frac{1}{2n-1} \end{pmatrix}$$

This matrix turns out to be ill-conditioned even for small values of n. For example, for $n = 5$ we have:

$$\text{cond}(H_5) \simeq 10^5$$

Vandermonde Matrices

Esempio 1.4. Given a vector $(y_1, \dots, y_n)$, with $y_i \neq y_j$ for $i \neq j$, these are non-singular matrices defined by:

$$V_n = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ y_1 & y_2 & y_3 & \dots & y_n \\ y_1^2 & y_2^2 & y_3^2 & \dots & y_n^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_1^{n-1} & y_2^{n-1} & y_3^{n-1} & \dots & y_n^{n-1} \end{pmatrix}$$

Also in this case, the condition number grows exponentially with respect to the dimension of the matrix.

Classification of Numerical Methods Numerical methods are subdivided into:

- **Direct methods:** In the absence of round-off errors, they calculate the exact solution in a finite number of steps.
- **Iterative methods:** They generate an infinite sequence of vectors that converges to the solution.

The choice depends on stability, memory usage, and computational cost.

Direct Methods and Sparsity Direct methods transform the problem into equivalent problems. They are efficient for dense matrices. However, for sparse matrices, direct methods cause the *fill-in* phenomenon: the number of non-zero elements grows during the procedure, potentially saturating memory. In these cases, iterative methods are preferable, as they leave the matrix A unaltered.

1.2 Diagonal and Triangular Systems

Direct Methods Direct methods are numerical methods that solve linear systems in a finite number of steps. In other words, assuming calculations are performed with infinite precision, such methods provide the exact solution of the system through a finite number of operations, known *a priori*. In finite precision arithmetic, however, if the proposed method is stable, it provides an approximate solution with the maximum achievable precision, i.e., in accordance with the conditioning of the coefficient matrix.

Diagonal Systems The most trivial of direct methods is obtained in the case of diagonal matrices. If we denote by $\mathbf{x} = {}^t(x_1, x_2, \dots, x_n)$ and $\mathbf{b} = {}^t(b_1, b_2, \dots, b_n)$ respectively the vector of unknowns and that of known terms, and if the coefficient matrix is of the type:

$$D = \begin{pmatrix} a_{1,1} & 0 & 0 & \dots & 0 \\ 0 & a_{2,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \dots & \vdots \\ 0 & 0 & 0 & \dots & a_{n,n} \end{pmatrix}$$

the system $D\mathbf{x} = \mathbf{b}$, under the hypothesis that D is non-singular (i.e., $a_{i,i} \neq 0, \forall i \in \{1, \dots, n\}$), is trivially solved via n divisions:

$$x_i = \frac{b_i}{a_{i,i}}, \quad i \in \{1, \dots, n\}$$

with an overall computational cost of n division operations. The algorithm is well-posed if the matrix is non-singular. Furthermore, since the algorithm is composed of n mutually independent divisions, it is trivially stable.

```
function x = diagSub(D, b, tol)
% DIAGSUB Solves the diagonal linear system Dx = b.
%  X = DIAGSUB(D, B, TOL) returns the solution vector X.
%
%  Input:
%    D - Diagonal square matrix (n x n)
```

```
%     b - Known term vector (n x 1)
%     tol (optional) - Tolerance for singularity check (1 x 1)
%
%   Output:
%     x - Solution vector (n x 1)

    %% 1. Input Validation
    arguments
        D (:,:) double {mustBeNumeric}
        b (:,1) double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Check Dimensions
    % Get dimensions
    [n,m] = size(D);

    % Verify D is square
    if n ~= m
        error('diagSub:NonSquareMatrix', 'Matrix D must be square (Input %dx%d).', n, m);
    end

    % Verify b is compatible with D
    if m ~= length(b)
        error('diagSub:DimensionMismatch', 'Dimensions of D and b do not match (Input (%dx%d)x(%dx1)).',n,m,length(b));
    end

    %% 3. Singularity Check
    if any(abs(diag(D)) < tol)
        error('diagSub:SingularMatrix', 'The matrix is singular.');
    end

    %% 4. Forward Substitution (Diagonal case is trivial forward sub)
    % Pre-allocation
    x = zeros(n, 1);

    % Main loop
    for i = 1 : n
        x(i) = b(i) / D(i,i);
    end

end
```

Function diagSub

Triangular Systems Let us now consider the case of systems with a triangular coefficient matrix. We denote by L a generic lower triangular matrix:

$$L = \begin{pmatrix} l_{1,1} & 0 & 0 & \dots & 0 \\ l_{2,1} & l_{2,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \dots & \vdots \\ l_{n-1,1} & l_{n-1,2} & \dots & l_{n-1,n-1} & 0 \\ l_{n,1} & l_{n,2} & \dots & l_{n,n-1} & l_{n,n} \end{pmatrix}$$

and by U a generic upper triangular matrix:

$$U = \begin{pmatrix} u_{1,1} & u_{1,2} & \dots & u_{1,n-1} & u_{1,n} \\ 0 & u_{2,2} & \dots & u_{2,n-1} & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & u_{n-1,n-1} & u_{n-1,n} \\ 0 & 0 & \dots & 0 & u_{n,n} \end{pmatrix}$$

We want to provide algorithms for solving the systems $L\mathbf{x} = \mathbf{b}$ and $U\mathbf{x} = \mathbf{b}$. A triangular matrix is non-singular if and only if its diagonal elements are all different from zero. Indeed:

$$\det(L) = \prod_{i=1}^{n} l_{ii} \neq 0 \iff l_{i,i} \neq 0, \quad \forall i \in \{1, \dots, n\};$$

$$\det(U) = \prod_{i=1}^{n} u_{ii} \neq 0 \iff u_{i,i} \neq 0, \quad \forall i \in \{1, \dots, n\}.$$

Algorithms for solving systems with a triangular coefficient matrix are essentially based on the substitution technique.

Forward Substitution Algorithm Let us examine the case of the lower triangular matrix $L\mathbf{x} = \mathbf{b}$. Explicitly, the system can be written as:

$$\begin{cases} l_{1,1}x_1 = b_1 \\ l_{2,1}x_1 + l_{2,2}x_2 = b_2 \\ \vdots \\ l_{n,1}x_1 + l_{n,2}x_2 + l_{n,3}x_3 + \cdots + l_{n,n}x_n = b_n \end{cases}$$

It is observed that x_1 is immediately obtained from the first equation. This value can be substituted into the second equation to obtain x_2, and so on.

$$x_1 = \frac{b_1}{l_{1,1}}, \quad x_2 = \frac{b_2 - l_{2,1}x_1}{l_{2,2}}, \quad \dots, \quad x_i = \frac{b_i - \sum_{k=1}^{i-1} l_{i,k}x_k}{l_{i,i}} \quad i \in \{2, \dots, n\}.$$

The algorithm is well-posed if the matrix L is non-singular.

Algorithm Scheme

```
x(1) = b(1) / L(1,1)                    % 1 flop (division)
for i = 2 : n
    x(i) = b(i)
    for k = 1 : i-1
        x(i) = x(i) - L(i,k) * x(k)     % 2 flops (1 mult, 1 sub)
    end
    x(i) = x(i) / L(i,i)                % 1 flop (division)
end
```

Computational Cost

$$\sum_{i=1}^{n} i \simeq \frac{n(n+1)}{2} \simeq \frac{n^2}{2} \text{ operations (flops)}$$

It is possible to prove that the algorithm is stable, i.e., it does not produce amplification of the round-off error.

```
function x = forwardSub(L, b, tol)
% FORWARDSUB Solves the lower triangular linear system Lx = b using forward substitution.
%  x = FORWARDSUB(L, b) returns the solution vector x.
%
%  Input:
%    L - Lower triangular square matrix (n x n)
%    b - Known term vector (n x 1)
%    tol (optional) - Tolerance for singularity check (1 x 1)
%
%  Output:
%    x - Solution vector (n x 1)

    %% 1. Input Validation
    arguments
        L (:,:) double {mustBeNumeric}
        b (:,1) double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Check Dimensions
    % Get dimensions
    [n,m] = size(L);

```

```
    % Verify L is square
    if n ~= m
        error('forwardSub:NonSquareMatrix', 'Matrix L must be square (Input %dx%d).', n, m);
    end

    % Verify b is compatible with L
    if m ~= length(b)
        error('forwardSub:DimensionMismatch', 'Dimensions of L and b do not match (Input (%dx%d)x(%dx1)).',n,m,length(b));
    end

    %% 3. Singularity Check
    if any(abs(diag(L)) < tol)
        error('forwardSub:SingularMatrix', 'The matrix is singular.');
    end

    %% 4. Forward Substitution
    % Pre-allocation
    x = zeros(n, 1);

    % First element
    x(1) = b(1) / L(1,1);

    % Main loop
    for i = 2 : n
        x(i) = b(i);
        % Inner loop: accumulate partial sum
        for k = 1:i-1
            x(i) = x(i) - L(i,k) * x(k);
        end
        % Final division by the diagonal element
        x(i) = x(i) / L(i,i);
    end

end
```

Function forwardSub

Calculation of the Inverse of a Lower Triangular Matrix The forward substitution method can be adapted for the explicit calculation of the inverse of a lower triangular matrix L. The column vectors $\mathbf{v}_i$ of the inverse $V = (\mathbf{v}_1, \ldots, \mathbf{v}_n)$ of L satisfy the linear systems:

$$L\mathbf{v}_i = \mathbf{e}_i, \quad i \in \{1, \ldots, n\}$$

where $\mathbf{e}_i$ is the i-th column of the identity matrix. Since the inverse of a lower triangular matrix is also lower triangular, the columns of V have all superdiagonal elements equal to zero. Exploiting this property, for each column j, the only unknown components are those from j to n. The calculation of the non-zero components $v_{i,j}$ (with $i \geq j$) takes place by solving the systems with forward substitution.

Inversion Algorithm

$$v_{j,j} = \frac{1}{l_{j,j}}, \quad \text{for } j \in \{1, \ldots, n\},$$

$$v_{i,j} = -\frac{1}{l_{i,i}} \sum_{k=j}^{i-1} l_{i,k} v_{k,j}, \quad \text{for } i \in \{j+1, \ldots, n\}.$$

Algorithm Scheme

```
for j = 1 : n
    v(j,j) = 1 / L(j,j)
    for i = j+1 : n
        v(i,j) = 0
        for k = j : i-1
            v(i,j) = v(i,j) - L(i,k) * v(k,j)
        end
```

```
        v(i,j) = v(i,j) / L(i,i)
    end
end
```

Computational Cost

$$\sum_{j=1}^{n}\sum_{i=j}^{n}(i-j+1) \simeq \frac{n^3}{6} \text{ operations.}$$

```
function V = invL(L, tol)
% INVL Calculates the inverse of a lower triangular matrix by optimizing
% the forward substitution with a given tolerance.
%  V = INVL(L, tol) returns the inverse of the matrix L.
%
%  Input:
%    L - Numerical lower triangular matrix (n x n)
%    tol (optional) - Tolerance for singularity check (1 x 1)
%
%  Output:
%    V - Numerical lower triangular inverse matrix of L (n x n)

    %% 1. Input Validation
    arguments
        L (:,:) double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n,m] = size(L);

    % Verify L is square
    if n ~= m
        error('invL:NonSquareMatrix', 'Matrix L must be square (Input %dx%d).', n, m);
    end

    % Verify L is non-singular
    if any(abs(diag(L)) < tol)
        error('invL:SingularMatrix', 'Matrix L is singular.');
    end

        %% 3. Inverse Calculation
    % Pre-allocation (V will be lower triangular, the rest remains 0)
    V = zeros(n);

    for j = 1:n
        % 1. Diagonal: inverse of the pivot
        V(j,j) = 1 / L(j,j);

        % 2. Forward substitution for column j (rows i > j)
        for i = j+1:n
            V(i,j) = 0;
            % Initialize accumulator for the summation
            for k = j:i-1
                V(i,j) = V(i,j) - L(i,k) * V(k,j);
            end

            % Final division by the diagonal coefficient
            V(i,j) = V(i,j) / L(i,i);
        end
    end

end
```

Function invL

Backward Substitution Algorithm We want to solve the system $U\mathbf{x} = \mathbf{b}$ where U is upper triangular:

$$\begin{cases} u_{1,1}x_1 + u_{1,2}x_2 + \cdots + u_{1,n}x_n = b_1 \\ \vdots \\ u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n = b_{n-1} \\ u_{n,n}x_n = b_n \end{cases}$$

Proceeding from the last equation towards the first:

$$x_n = \frac{b_n}{u_{n,n}}, \quad x_{n-1} = \frac{b_{n-1} - u_{n-1,n}x_n}{u_{n-1,n-1}}, \quad \ldots, \quad x_i = \frac{b_i - \sum_{k=i+1}^{n} u_{i,k}x_k}{u_{i,i}} \quad i \in \{n-1, \ldots, 1\}.$$

Also in this case, the algorithm is well-posed if U is non-singular.

Algorithm Scheme

```
x(n) = b(n) / U(n,n)                    % 1 flop (division)
for i = n-1 : -1 : 1
    x(i) = b(i)
    for k = i+1 : n
        x(i) = x(i) - U(i,k) * x(k)     % 2 flops (1 mult, 1 sub)
    end
    x(i) = x(i) / U(i,i)                % 1 flop (division)
end
```

Computational Cost

$$\sum_{i=1}^{n}(n-i+1) = \frac{n(n+1)}{2} \simeq \frac{n^2}{2} \text{ operations.}$$

The backward substitution algorithm is also stable.

```
function x = backwardSub(U, b, tol)
% BACKWARDSUB Solves the upper triangular linear system Ux = b using backward substitution.
%  x = BACKWARDSUB(U, b, tol) returns the solution vector x.
%
%  Input:
%    U - Upper triangular square matrix (n x n)
%    b - Known term vector (n x 1)
%    tol (optional) - Tolerance for singularity check (1 x 1)
%
%  Output:
%    x - Solution vector (n x 1)

    %% 1. Input Validation
    arguments
        U (:,:) double {mustBeNumeric}
        b (:,1) double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Check Dimensions
    % Get dimensions
    [n, m] = size(U);

    % Verify U is square
    if n ~= m
        error('backwardSub:NonSquareMatrix', 'Matrix U must be square (Input %dx%d).', n, m);
    end

    % Verify b is compatible with U
    if m ~= length(b)
        error('backwardSub:DimensionMismatch', 'Dimensions of U and b do not match (Input (%dx%d)x(%dx1)).', n, m, length(b));
    end

```

```
    %% 3. Singularity Check
    if any(abs(diag(U)) < tol)
        error('backwardSub:SingularMatrix', 'The matrix is singular.');
    end

    %% 4. Backward Substitution
    % Pre-allocation
    x = zeros(n, 1);

    % Last element
    x(n) = b(n) / U(n,n);

    % Main loop
    for i = n-1 : -1 : 1
        x(i) = b(i);
        % Inner loop: accumulate partial sum
        for k = i+1:n
            x(i) = x(i) - U(i,k) * x(k);
        end
        % Final division by the diagonal element
        x(i) = x(i) / U(i,i);
    end

end
```

Function backwardSub

Calculation of the Inverse of an Upper Triangular Matrix The backward substitution method can be adapted for the explicit calculation of the inverse of an upper triangular matrix U. The column vectors $\mathbf{v}_i$ of the inverse $V = (\mathbf{v}_1, \ldots, \mathbf{v}_n)$ of U satisfy the linear systems:

$$U\mathbf{v}_i = \mathbf{e}_i, \quad i \in \{1, \ldots, n\}$$

where $\mathbf{e}_i$ is the i-th column of the identity matrix. Since the inverse of an upper triangular matrix is also upper triangular, the columns of V have all subdiagonal elements equal to zero. Exploiting this property, let us denote by $\mathbf{v}'_j = {}^t(v_{1,j}, \ldots, v_{j,j})$ the vector of dimension j such that:

$$U_j\mathbf{v}'_j = \mathbf{e}'_j, \quad j \in \{1, \ldots, n\}$$

where U_j is the leading principal submatrix of U of order j and $\mathbf{e}'_j$ is the vector in $\mathbb{R}^j$ with all components zero except the last one which is 1. The systems are upper triangular of order j and are solved with backward substitution.

Inversion Algorithm

$$v_{j,j} = \frac{1}{u_{j,j}}, \quad \text{for } j \in \{1, \ldots, n\},$$

$$v_{i,j} = -\frac{1}{u_{i,i}} \sum_{k=i+1}^{j} u_{i,k} v_{k,j}, \quad \text{for } i \in \{j-1, \ldots, 1\}.$$

Algorithm Scheme

```
for j = 1 : n
    v(j,j) = 1 / U(j,j)
    for i = j-1 : -1 : 1
        v(i,j) = 0
        for k = i+1 : j
            v(i,j) = v(i,j) - U(i,k) * v(k,j)
        end
        v(i,j) = v(i,j) / U(i,i)
    end
end
```

Computational Cost

$$\sum_{j=1}^{n}\sum_{i=1}^{j}(j-i+1) \simeq \frac{n^3}{6} \text{ operations.}$$

```
function V = invU(U, tol)
% INVU Calculates the inverse of an upper triangular matrix by optimizing
% the backward substitution with a given tolerance.
%  V = INVU(U, tol) returns the inverse of the matrix U.
%
%  Input:
%    U - Numerical upper triangular matrix (n x n)
%    tol (optional) - Tolerance for singularity check (1 x 1)
%
%  Output:
%    V - Numerical upper triangular inverse matrix of U (n x n)

    %% 1. Input Validation
    arguments
        U (:,:) double {mustBeNumeric}
        tol (1,1) double {mustBeNumeric} = 1e-10
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n,m] = size(U);

    % Verify U is square
    if n ~= m
        error('invU:NonSquareMatrix', 'Matrix U must be square (Input %dx%d).', n, m);
    end

    % Verify U is non-singular
    if any(abs(diag(U)) < tol)
        error('invU:SingularMatrix', 'Matrix U is singular.');
    end

    %% 3. Inverse Calculation
    % Pre-allocation
    V = zeros(n);

    % Construction of inverse V
    for j = 1:n
        % Calculation of diagonal element (Pivot)
        V(j,j) = 1 / U(j,j);
        for i = j-1:-1:1
            % Initialize accumulator for summation
            V(i,j) = 0;
            for k = i+1:j
                V(i,j) = V(i,j) - U(i,k) * V(k,j);
            end
            % Final division by the diagonal coefficient
            V(i,j) = V(i,j) / U(i,i);
        end
    end

end
```

Function invU

1.3 Gaussian Elimination Method

Gaussian Elimination Method Let us consider the case of a generic matrix of order n. Let $A \in \mathbb{R}^{n\times n}$ and as usual $x, \mathbf{b} \in \mathbb{R}^n$. We want to solve the generic system $Ax = b$. Gaussian method is based on the technique of eliminating unknowns from fixed equations. The goal of the method is to reduce, through a finite number of steps, the initial system to an equivalent one that has a triangular coefficient matrix.

Description of the method (case $n = 4$) For simplicity, let us start by describing the method in the case of a system of dimension $n = 4$.

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + a_{2,3}x_3 + a_{2,4}x_4 = b_2 \\ a_{3,1}x_1 + a_{3,2}x_2 + a_{3,3}x_3 + a_{3,4}x_4 = b_3 \\ a_{4,1}x_1 + a_{4,2}x_2 + a_{4,3}x_3 + a_{4,4}x_4 = b_4 \end{cases}$$

Assume that $a_{1,1} \neq 0$ and consider the quantity $m_{2,1} = -\frac{a_{2,1}}{a_{1,1}}$. Multiplying both sides of the first equation by $m_{2,1}$ and adding it to the second equation, we obtain:

$$(-a_{2,1} + a_{2,1})x_1 + (a_{2,2} - \frac{a_{2,1}}{a_{1,1}}a_{1,2})x_2 + \cdots + (a_{2,4} - \frac{a_{2,1}}{a_{1,1}}a_{1,4})x_4 = (b_2 - \frac{a_{2,1}}{a_{1,1}}b_1)$$

that is

$$a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)}$$

thus

$$a_{2,j}^{(2)} = a_{2,j} + m_{2,1}a_{1,j}, \quad j = 2, 3, 4$$

Repeating the procedure for the third and fourth equations with the multipliers $m_{3,1} = -\frac{a_{3,1}}{a_{1,1}}$ and $m_{4,1} = -\frac{a_{4,1}}{a_{1,1}}$, we obtain the equivalent system at step 2:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,2}^{(2)}x_2 + a_{3,3}^{(2)}x_3 + a_{3,4}^{(2)}x_4 = b_3^{(2)} \\ a_{4,2}^{(2)}x_2 + a_{4,3}^{(2)}x_3 + a_{4,4}^{(2)}x_4 = b_4^{(2)} \end{cases}$$

where in general $a_{i,j}^{(2)} = a_{i,j} + m_{i,1}a_{1,j}$ and $\mathbf{b}_i^{(2)} = b_i + m_{i,1}b_1$.

Assuming now that $a_{2,2}^{(2)} \neq 0$, we eliminate the unknown x_2 from the third and fourth equations using the multipliers $m_{3,2} = -\frac{a_{3,2}^{(2)}}{a_{2,2}^{(2)}}$ and $m_{4,2} = -\frac{a_{4,2}^{(2)}}{a_{2,2}^{(2)}}$. We obtain the system at step 3:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,3}^{(3)}x_3 + a_{3,4}^{(3)}x_4 = b_3^{(3)} \\ a_{4,3}^{(3)}x_3 + a_{4,4}^{(3)}x_4 = b_4^{(3)} \end{cases}$$

Finally, assuming $a_{3,3}^{(3)} \neq 0$, we eliminate x_3 from the last equation with $m_{4,3} = -\frac{a_{4,3}^{(3)}}{a_{3,3}^{(3)}}$, obtaining the final triangular system:

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 = b_1 \\ a_{2,2}^{(2)}x_2 + a_{2,3}^{(2)}x_3 + a_{2,4}^{(2)}x_4 = b_2^{(2)} \\ a_{3,3}^{(3)}x_3 + a_{3,4}^{(3)}x_4 = b_3^{(3)} \\ a_{4,4}^{(4)}x_4 = b_4^{(4)} \end{cases}$$

Generalization to the case of order n Assume that $a_{1,1} \neq 0$. We can eliminate the unknown x_1 from the $2^{nd}, \ldots, n$-th equation by adding to the i-th equation ($i = 2, \ldots, n$) the first equation multiplied by $m_{i,1} = -\frac{a_{i,1}}{a_{1,1}}$. The general update formulas at step $k \in 1, \ldots, n-1$ (to eliminate the unknown x_k from subsequent equations) are for $i, j \in \{k+1, \ldots, n\}$:

$$m_{i,k} = -\frac{a_{i,k}^{(k)}}{a_{k,k}^{(k)}}, \qquad a_{i,j}^{(k+1)} = a_{i,j}^{(k)} + m_{i,k}a_{k,j}^{(k)}, \qquad b_i^{(k+1)} = b_i^{(k)} + m_{i,k}b_k^{(k)}.$$

The elements $a_{1,1}, a_{2,2}^{(2)}, a_{3,3}^{(3)}, \ldots$ that appear in the denominator are called **pivot elements**, while the quantities $m_{i,k}$ are called **multipliers**. After $n-1$ steps, if all pivots are non-zero, an upper triangular system is obtained which can be solved by backward substitution.

Esempio 1.5. Consider the system:

$$\begin{cases} 2x_1 - x_2 + x_3 - 2x_4 = 0 \\ 2x_2 - x_4 = 1 \\ x_1 - 2x_3 + x_4 = 0 \\ 2x_2 + x_3 + x_4 = 4 \end{cases}.$$

1. **Step 1:** $m_{2,1} = 0, m_{3,1} = -1/2, m_{4,1} = 0$. The system becomes:

$$\begin{cases} 2x_1 - x_2 + x_3 - 2x_4 = 0 \\ 2x_2 - x_4 = 1 \\ \frac{1}{2}x_2 - \frac{5}{2}x_3 + 2x_4 = 0 \\ 2x_2 + x_3 + x_4 = 4 \end{cases}$$

2. **Step 2:** $m_{3,2} = -1/4, m_{4,2} = -1$. The system becomes:

$$\begin{cases} 2x_1 - x_2 + x_3 - 2x_4 = 0 \\ 2x_2 - x_4 = 1 \\ -\frac{5}{4}x_3 + \frac{9}{4}x_4 = -\frac{1}{4} \\ x_3 + 2x_4 = 3 \end{cases}$$

3. **Step 3:** $m_{4,3} = \frac{4}{5}$. The final system is:

$$\begin{cases} 2x_1 - x_2 + x_3 - 2x_4 = 0 \\ 2x_2 - x_4 = 1 \\ -\frac{5}{4}x_3 + \frac{9}{4}x_4 = -\frac{1}{4} \\ \frac{38}{10}x_4 = \frac{28}{10} \implies x_4 = 1 \ldots \end{cases}$$

Substituting backwards gives the solution $\mathbf{x} = {}^t(1, 1, 1, 1)$.

Algorithm Scheme

```
for k = 1 : n-1
    for i = k+1 : n
        mol = -a(i,k) / a(k,k)      % 1 operation
        for j = k+1 : n
            a(i,j) = a(i,j) + mol * a(k,j)  % n-k operations
        end
        b(i) = b(i) + mol * b(k)    % 2 operations
    end
end
```

Computational Cost

$$\sum_{k=1}^{n-1} [2(n-k) + (n-k)^2] \simeq \frac{n^3}{3} \text{ operations}$$

```
function [Ag,bg] = gaussElim(A,b)
% GAUSSELIM Applies the Gaussian elimination method to the coefficient matrix and the vector
    of known terms.
%   [Ag,bg] = gaussElim(A,b) returns A and b resulting from the elimination.
%
%   Input:
%     A - Coefficient matrix (n x n)
%     b - Vector of known terms (n x 1)
%
%   Output:
```

```
%     Ag - Coefficient matrix post-GE (n x n)
%     bg - Vector of known terms post-GE (n x 1)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
        b (:,1) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    [n, m] = size(A);

    % Check if matrix is square
    if n ~= m
        error('gaussElim:NonSquareMatrix', 'Matrix is not square (Size %dx%d).', n, m);
    end

    % Check if product Ab is compatible
    if n ~= length(b)
        error('gaussElim:NonCompatible', 'Product is not compatible (Size (%dx%d)*(%dx1)).', n
    , m, length(b));
    end


    %% 3. Gaussian Elimination
    for k = 1 : n-1
        % Check for zero pivot (safety)
        if A(k,k) == 0
             error('gaussElim:ZeroPivot', 'Zero pivot at row %d.', k);
        end

        % Elimination
        for i = k+1 : n
            mol = -A(i,k) / A(k,k);
            for j = k+1:n
                A(i,j) = A(i,j) + mol * A(k,j);
            end
            b(i) = b(i) + mol * b(k);
        end
    end

    % Note: Elements A(i,k) below the diagonal are not physically zeroed out
    % to save time, but are logically considered zero.

    Ag = A;
    bg = b;
end
```

Function gaussElim

Existence of Pivots

Teorema 1.4. *Let $A \in \mathbb{R}^{n\times n}$. The pivot elements $a_{k,k}^{(k)}$ are all non-zero if and only if all leading principal submatrices of A, denoted by $A_k = (a_{i,j})_{i,j\in\{1,\dots,k\}}$, are non-singular, that is:*

$$\det(A_k) \neq 0 \quad \forall k \in \{1,\dots,n\}.$$

Nota 5. Since this condition is difficult to verify, there exist families of matrices for which it is guaranteed *a priori*:

- **Row or column diagonally dominant** matrices;
- **Symmetric positive definite** matrices (${}^tA = A$ and ${}^t\mathbf{y}A\mathbf{y} > 0, \forall \mathbf{y} \neq \mathbf{0}$).

1.4 Pivoting Strategy and LU Factorization

Need for Pivoting The Gaussian method described above stops if, at a certain step k, the pivot element $a_{k,k}^{(k)}$ turns out to be zero. However, if the matrix is non-singular, there must exist at least one element $a_{i,k}^{(k)} \neq 0$ with $i > k$ in the same column. To proceed, it is sufficient to swap the k-th row with the i-th row. Even when the pivot is not zero but is very small in absolute value compared to other elements, disastrous round-off errors can occur (error amplification or numerical cancellation). To ensure numerical stability, it is necessary to permute rows to choose an "optimal" pivot. This strategy is called *pivoting*.

Pivoting Strategies

- **Partial Pivoting:** At step k, the element with the maximum modulus in the k-th column, below or on the main diagonal, is chosen as the pivot. One seeks the index $r \geq k$ such that:

$$|a_{r,k}^{(k)}| = \max_{k \leq i \leq n} |a_{i,k}^{(k)}|$$

  If $r \neq k$, swap row k with row r.

- **Total Pivoting:** The maximum element is sought over the entire active submatrix. Indices $r, s \geq k$ are chosen such that:

$$|a_{r,s}^{(k)}| = \max_{k \leq i,j \leq n} |a_{i,j}^{(k)}|$$

  Rows k and r and columns k and s are swapped (the latter swap involves reordering the unknowns).

Partial pivoting is the most used strategy because it is computationally less expensive ($O(n^2)$ comparisons) and generally sufficient to ensure stability.

Stability without Pivoting Gaussian elimination without pivoting is stable *a priori* only for certain classes of matrices:

- **Row or column diagonally dominant** matrices;
- **Symmetric** and **positive definite** matrices.

Example of Instability

Esempio 1.6. Consider a system of order $n = 18$ with matrix defined by $a_{i,j} = \cos((j-1)\frac{2i-1}{2n}\pi)$ and known term equal to the sum of the row elements (exact solution $x_i = 1$). The matrix is well-conditioned ($K \approx 16.9$). Solving with Gauss without pivoting, significant errors are obtained (e.g., $x_{15} \approx 1.0000000002...$). Solving with Gauss and partial pivoting, the correct solution is obtained with maximum precision (e.g., $x_{15} \approx 1.000000000000000$).

Gauss (without pivoting)	Gauss + Partial Pivoting
9.999999968425205e-001	9.999999999999993e-001
1.000000005848771e+000	1.000000000000000e+000
9.999999953310822e-001	1.000000000000001e+000
1.000000003291504e+000	1.000000000000001e+000
9.999999977773912e-001	9.999999999999997e-001
1.000000001703849e+000	1.000000000000000e+000
9.999999983501708e-001	1.000000000000000e+000
1.000000001767299e+000	1.000000000000001e+000
9.999999982508074e-001	9.999999999999998e-001
1.000000001433063e+000	9.999999999999999e-001
9.999999991425489e-001	9.999999999999993e-001
1.000000000200525e+000	1.000000000000000e+000
1.000000000347843e+000	9.999999999999996e-001
9.999999993163441e-001	9.999999999999990e-001
1.000000000800448e+000	9.999999999999996e-001
9.999999992564082e-001	1.000000000000000e+000
1.000000000564552e+000	1.000000000000000e+000
9.999999996959291e-001	9.999999999999999e-001

Algorithm Scheme

```
for k = 1 : n-1
    % Search for maximum pivot in the k-th column
    find p such that: |a(p,k)| = max(|a(i,k)|) for i >= k

    if k ~= p
        swap rows k and p of A
        swap rows k and p of b
    end

    % Standard Gaussian Elimination
    for i = k+1 : n
        mol = -a(i,k) / a(k,k)
        for j = k+1 : n
            a(i,j) = a(i,j) + mol * a(k,j)
        end
        b(i) = b(i) + mol * b(k)
    end
end
```

Computational Cost The total computational cost of the Gaussian algorithm with pivoting is given by the sum of the arithmetic operations (flops) required for elimination and the comparisons required for finding the maximum pivot.

- **Arithmetic Operations:** $\approx \frac{n^3}{3}$ (unchanged with respect to Gauss without pivoting).
- **Comparisons:** At each step k, $n-k$ comparisons are performed. The total is:

$$\sum_{k=1}^{n-1}(n-k) = \frac{n(n-1)}{2} \approx \frac{n^2}{2} \text{ comparisons}$$

Since n^2 is negligible compared to n^3 for large n, the asymptotic cost remains dominated by $n^3/3$.

```
function [Ag,bg] = gaussElimPP(A,b)
% GAUSSELIMPP Applies Gaussian elimination to the coefficient matrix and
%  the known term vector implementing partial pivoting.
%  [Ag,bg] = gaussElimPP(A,b) returns A and b resulting from elimination with partial pivoting
    .
%
%  Input:
%    A - Coefficient matrix (n x n)
%    b - Known term vector (n x 1)
%
%  Output:
%    Ag - Coefficient matrix post-GE (n x n)
%    bg - Known term vector post-GE (n x 1)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
        b (:,1) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    [n, m] = size(A);

    % Verify if the matrix is square
    if n ~= m
        error('gaussElimPP:NonSquareMatrix', 'The matrix is not square (Size %dx%d).', n, m);
    end

    % Verify if the product Ab is compatible
    if n ~= length(b)
        error('gaussElimPP:NonCompatible', 'The product is not compatible (Size (%dx%d)*(%dx1)
    ).', n, m, length(b));
```

```
    end


    %% 3. Gaussian Elimination
    for k = 1 : n-1
        % Partial Pivoting
        % Find the maximum in column k (from the pivot downwards)
        [~, r_rel] = max(abs(A(k:n, k)));

        % Convert to global index
        r_glob = r_rel + k - 1;

        % If the best pivot is not the current one, swap
        if r_glob ~= k
            A([k, r_glob], :) = A([r_glob, k], :);
            b([k, r_glob])    = b([r_glob, k]);
        end

        % Check for zero pivot (safety)
        if A(k,k) == 0
             error('gaussElimPP:ZeroPivot', 'Zero pivot at row %d.', k);
        end

        % Gaussian Elimination
        for i = k+1 : n
            mol = -A(i,k) / A(k,k);

            for j = k+1:n
                A(i,j) = A(i,j) + mol * A(k,j);
            end

            % Update the known term
            b(i) = b(i) + mol * b(k);
        end
    end

    % Note: Elements A(i,k) below the diagonal are not physically zeroed out
    % to save time, but are logically considered zero.

    Ag = A;
    bg = b;
end
```

Function gaussElimPP

LU Factorization The Gaussian elimination method, from a matrix point of view, can be reinterpreted as the construction of a sequence of matrices:

$$[A|\mathbf{b}] = [A^{(1)}|\mathbf{b}^{(1)}], \dots, [A^{(k)}|\mathbf{b}^{(k)}], \dots, [A^{(n)}|\mathbf{b}^{(n)}]$$

such that $A^{(n)}$ is upper triangular and $\mathbf{b}^{(n)}$ is the new known term. The matrices in the sequence are related by a transformation of the type:

$$[A^{(k+1)}|\mathbf{b}^{(k+1)}] = M^{(k)}[A^{(k)}|\mathbf{b}^{(k)}], \quad k \in \{1, \dots, n-1\}$$

where $M^{(k)}$ is called the **Gaussian elementary matrix** and is defined as:

$$M^{(k)} = \begin{pmatrix} 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & \dots & m_{k+1,k} & 1 & \dots & 0 \\ \vdots & & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & m_{n,k} & 0 & \dots & 1 \end{pmatrix}$$

It follows that:

$$A = [M^{(1)}]^{-1}A^{(2)} = [M^{(1)}]^{-1}[M^{(2)}]^{-1}A^{(3)} = \dots = [M^{(1)}]^{-1} \dots [M^{(n-1)}]^{-1}A^{(n)}$$

and analogously for the known term:

$$\mathbf{b} = [M^{(1)}]^{-1} \dots [M^{(n-1)}]^{-1}\mathbf{b}^{(n)}$$

Setting:

$$L = \left(\prod_{k=1}^{n-1} [M^{(k)}]^{-1} \right), \quad U = A^{(n)}, \quad \mathbf{y} = \mathbf{b}^{(n)},$$

we obtain the factorization:

$$A = LU \quad \text{and} \quad \mathbf{b} = L\mathbf{y}$$

where U is the upper triangular matrix obtained at the end of the Gaussian elimination method and L is a lower triangular matrix. The structure of L is as follows:

$$L = \begin{pmatrix} 1 & 0 & \dots & & \dots & 0 & \\ -m_{2,1} & 1 & 0 & & & \vdots & \\ \vdots & \ddots & \ddots & \ddots & & & \\ & \ddots & 1 & & & \vdots & \\ \vdots & & -m_{k+1,k} & 1 & \ddots & & \\ & & -m_{k+2,k} & \ddots & \ddots & & \\ \vdots & & \vdots & & \ddots & 1 & 0 \\ -m_{n,1} & \dots & -m_{n,k} & \dots & \dots & -m_{n,n-1} & 1 \end{pmatrix}$$

To construct it, it suffices to store the sign-changed multipliers at each step of the Gauss method.

Algorithm Scheme

```
for k = 1 : n-1
    for i = k+1 : n
        A(i,k) = -A(i,k) / A(k,k)      % Calculate multipliers
        for j = k+1 : n
            A(i,j) = A(i,j) + A(i,k) * A(k,j)
        end
    end
end
L = eye(n) - tril(A, -1)   % Lower triangular part with 1 on the diagonal
U = triu(A)                % Upper triangular part
```

```
function [L, U] = factorLU(A)
% FACTORLU Performs LU factorization on a matrix A.
%  [L, U] = factorLU(A) factorizes matrix A into L * U using
%  Gaussian elimination without pivoting.
%
%  Input:
%    A - Coefficient matrix (n x n)
%
%  Output:
%    L - Unit lower triangular matrix (n x n)
%    U - Upper triangular matrix (n x n)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n, m] = size(A);

    % Verify if the matrix is square
    if n ~= m
        error('factorLU:NonSquareMatrix', 'The coefficient matrix must be square (Size: %dx%d).', n, m);
    end

    %% 3. Gaussian Elimination
    for k = 1 : n-1
```

```
        % Check for zero pivot
        if A(k,k) == 0
             error('factorLU:ZeroPivot', 'Zero pivot encountered at row %d.', k);
        end

        % Loop over rows below
        for i = k+1 : n
            % Calculate and assign the multiplier
            A(i,k) = -A(i,k) / A(k,k);

            % Update row i (Elimination)
            for j = k+1:n
                A(i,j) = A(i,j) + A(i,k) * A(k,j);
            end
        end
    end

    % Construction of L and U
    L = eye(n) - tril(A, -1);
    U = triu(A);
end
```

Function factorLU

Solving Systems with LU Once factorized $A = LU$, the system $A\mathbf{x} = \mathbf{b}$ becomes $LU\mathbf{x} = \mathbf{b}$. It is solved in two steps:

1. Resolution of the lower triangular system $L\mathbf{y} = \mathbf{b}$ (forward substitution).
2. Resolution of the upper triangular system $U\mathbf{x} = \mathbf{y}$ (backward substitution).

Computational Cost Total cost: $\frac{n^3}{3}$ (factorization) $+ n^2$ (system solution).

Applications of LU Factorization

1. **Calculation of the Determinant of A**
   The determinant of a matrix A can be calculated using Binet's formula. Having the factorization $A = LU$, we have:
   $$\det(A) = \det(LU) = \det(L)\det(U)$$
   Since L is lower triangular with all 1s on the diagonal, $\det(L) = 1$. Since U is upper triangular, its determinant is the product of the diagonal elements. Thus:
   $$\det(A) = \det(U) = \prod_{i=1}^{n} u_{i,i}$$
   The overall computational cost is $\frac{n^3}{3} + n$ (where $\frac{n^3}{3}$ is the factorization cost).
2. **Solving p systems with the same coefficient matrix**
   Suppose we need to solve the systems:
   $$A\mathbf{x}_1 = \mathbf{b}_1, \quad A\mathbf{x}_2 = \mathbf{b}_2, \quad \ldots, \quad A\mathbf{x}_p = \mathbf{b}_p$$
   or in other terms the matrix system:
   $$AX = B$$
   with $A \in \mathbb{R}^{n\times n}$ and $X, B \in \mathbb{R}^{n\times p}$. Having calculated the matrices L and U only once, each system $A\mathbf{x}_i = \mathbf{b}_i$ is solved by solving the two triangular systems:
   $$\begin{cases} L\mathbf{y} = \mathbf{b}_i \\ U\mathbf{x}_i = \mathbf{y} \end{cases} \qquad i \in \{1, \ldots, p\}$$
   using the substitution algorithms. The overall computational cost is:
   $$\frac{n^3}{3} + pn^2$$

If instead each system were solved independently from the others with Gauss's method, the cost would be much higher: $p(\frac{n^3}{3} + \frac{n^2}{2})$.

3. **Calculation of the Inverse of A**
   If $p = n$ and $B = I$ (identity matrix), solving the system $AX = B$ is equivalent to calculating the inverse A^{-1}. Therefore, we solve the systems:

$$\begin{cases} L\mathbf{y} = \mathbf{e}_i \\ U\mathbf{x}_i = \mathbf{y} \end{cases} \quad , \quad i \in \{1, \dots, n\}$$

   where the vectors $\mathbf{e}_i$ represent the columns of the matrix I (vectors of the canonical basis). The "raw" overall computational cost would be $\frac{n^3}{3} + n^3$. However, by appropriately exploiting the nature of the vectors $\mathbf{e}_i$ (which contain many zeros), the cost reduces to approximately n^3. In detail:

   - Solving the matrix system $LY = I$ has a cost of $\frac{n^3}{6}$.
   - Solving the matrix system $UX = Y$ has a cost of $\frac{n^3}{2}$.

   Summing the factorization cost ($\frac{n^3}{3}$), the asymptotic total cost is n^3.

Gauss Method with Pivoting Variant The Gauss method with the pivoting variant performs another matrix factorization in the following terms:

$$PA = \hat{L}U \quad \text{and} \quad P\mathbf{b} = \hat{L}\mathbf{y}$$

where $P \in \mathbb{R}^{n \times n}$, called a permutation matrix, contains information regarding row swaps. The following holds:

Teorema 1.5. *For every matrix $A \in \mathbb{R}^{n \times n}$ there exists a permutation matrix $P \in \mathbb{R}^{n \times n}$ such that*

$$PA = \hat{L}U.$$

Permutation Matrices Let $1 \leq r, s \leq n$ and let $P_{r,s}$ be the matrix obtained from I by swapping the r-th and s-th rows.

$$P_{r,s} = \begin{array}{cc} \begin{pmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & 0 & \cdots & 1 & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix} & \begin{array}{l} \\ \\ \leftarrow r \\ \\ \leftarrow s \\ \\ \\ \end{array} \\ \begin{array}{ccccccc} & & \uparrow & & \uparrow & & \\ & & r & & s & & \end{array} & \end{array}$$

Proposizione 1.6. *The matrix $P_{r,s}$ is called a* ***permutation matrix*** *and enjoys the following properties:*

1. $det(P_{r,s}) = -1$;
2. $P_{r,s} = {}^tP_{r,s}$;
3. $P_{r,s}^2 = I$;
4. $P_{r,s}^{-1} = P_{r,s}$.

Proof. 1. Trivial;

2. Follows from the fact that the element at position (r, s) coincides with that at position (s, r), while all other non-zero elements lie on the diagonal;

3. $$P_{r,s}P_{r,s} = {}^t(\mathbf{e}_1, \dots, \mathbf{e}_{r-1}, \mathbf{e}_s, \mathbf{e}_{r+1}, \dots, \mathbf{e}_{s-1}, \mathbf{e}_r, \mathbf{e}_{s+1}, \dots, \mathbf{e}_n)(\mathbf{e}_1, \dots, \mathbf{e}_s, \dots, \mathbf{e}_r, \dots, \mathbf{e}_n)$$
$$= \mathbf{e}_1{}^t\mathbf{e}_1 + \dots + \mathbf{e}_{r-1}{}^t\mathbf{e}_{r-1} + \mathbf{e}_s{}^t\mathbf{e}_s + \mathbf{e}_{r+1}{}^t\mathbf{e}_{r+1} + \dots + \mathbf{e}_{s-1}{}^t\mathbf{e}_{s-1} + \mathbf{e}_r{}^t\mathbf{e}_r + \mathbf{e}_{s+1}{}^t\mathbf{e}_{s+1} + \dots + \mathbf{e}_n{}^t\mathbf{e}_n = I$$
where $\mathbf{e}_i$ is the i-th column vector of the canonical basis;

4. Follows from 3.

□

Osservazione 1. It is also seen that:

- the product $P_{r,s}A$ produces the swap of the r-th and s-th rows in A.
- the product $AP_{r,s}$ produces the swap of the r-th and s-th columns in A.

LU Factorization with Pivoting The steps of LU factorization without pivoting for matrix A can be summarized as follows:

- If $a_{1,1} \neq 0$: elimination of elements in the first column starting from the second row of the initial matrix, obtained by the product $M^{(1)}A$.
- If $a^{(2)}_{2,2} \neq 0$: elimination of elements in the second column starting from the third row of the active matrix, obtained by the product $M^{(2)}M^{(1)}A$.
- ...
- If $a^{(n-1)}_{n-1,n-1} \neq 0$: elimination of elements in the $(n-1)$-th column starting from the n-th row of the active matrix, obtained by the product $M^{(n-1)} \dots M^{(2)}M^{(1)}A$.

At the end, we have $M^{(n-1)} \dots M^{(2)}M^{(1)}A = U$, which is upper triangular. We can describe the pivoting technique using permutation matrices in the following way:

- Let $|a_{r_1,1}| = \max_{i\geq 1} |a_{i,1}|$. If $r_1 \neq 1$, set $P_1 = P_{1,r_1}$ and perform the swap of the first row with the r_1-th row via the product P_1A. If $r_1 = 1$, set $P_1 = I$ and no swaps are performed. The elimination of elements in the first column starting from the second row of the matrix P_1A is performed via the product $M^{(1)}P_1A$.
- Let $|a^{(2)}_{r_2,2}| = \max_{i\geq 2} |a^{(2)}_{i,2}|$. If $r_2 \neq 2$, set $P_2 = P_{2,r_2}$ and perform the swap of the second row with the r_2-th row of the matrix $M^{(1)}P_1A$ via the product $P_2M^{(1)}P_1A$. If $r_2 = 2$, set $P_2 = I$. The elimination of elements in the second column starting from the third row of the matrix $P_2M^{(1)}P_1A$ is performed via the product $M^{(2)}P_2M^{(1)}P_1A$.
- ...

At the end of the procedure, we obtain:

$$M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1A = U$$

or

$$GA = U, \quad G = M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1.$$

We now want to show how from $GA = U$ we arrive at the following factorization of matrix A:

$$PA = \hat{L}U$$

where P is an appropriate permutation matrix, which accounts for all row swaps, $\hat{L}$ is a lower triangular matrix with diagonal elements equal to 1, and U is the upper triangular matrix from the Gaussian elimination method. Obviously, in the absence of swaps, $P = I$.

Indeed, the matrices can be reordered to obtain:

$$M^{(n-1)}P_{n-1} \dots M^{(2)}P_2M^{(1)}P_1A = (\overline{L}_{n-1} \dots \overline{L}_2\overline{L}_1)^{-1}(P_{n-1} \dots P_2P_1)A$$

and, letting $\hat{L}^{-1} = \overline{L}_{n-1} \dots \overline{L}_2 \overline{L}_1$ and $P = P_{n-1} \dots P_2 P_1$, we can write:

$$\hat{L}^{-1} P A = U$$

from which

$$PA = \hat{L} U.$$

Esempio 1.7. Let us examine the particular case $n = 4$. We have:

$$M^{(3)} P_3 M^{(2)} P_2 M^{(1)} P_1 A = M^{(3)} P_3 M^{(2)} P_3 P_3 P_2 M^{(1)} P_2 P_2 P_1 A$$

Setting $L_1^{(1)} = P_2 M^{(1)} P_2$:

$$= M^{(3)} P_3 M^{(2)} P_3 P_3 L_1^{(1)} P_2 P_1 A$$

Setting $L_2^{(1)} = P_3 M^{(2)} P_3$:

$$= M^{(3)} L_2^{(1)} P_3 L_1^{(1)} P_3 P_3 P_2 P_1 A$$

Setting $L_1^{(2)} = P_3 L_1^{(1)} P_3$:

$$= M^{(3)} L_2^{(1)} L_1^{(2)} P_3 P_2 P_1 A$$

In general (this can be proven by induction):

$$M^{(n-1)} P_{n-1} M^{(n-2)} P_{n-2} \dots M^{(2)} P_2 M^{(1)} P_1 A = \overline{L}_{n-1} \dots \overline{L}_2 \overline{L}_1 P_{n-1} \dots P_2 P_1 A$$

with

$$\overline{L}_j = \begin{cases} L_j^{(n-1-j)} = (P_{n-1} P_{n-2} \dots P_{j+1}) M^{(j)} (P_{j+1} \dots P_{n-2} P_{n-1}), & j < n-1 \\ M^{(j)}, & j = n-1 \end{cases}$$

Osservazione 2. The matrices $M^{(i)}$ and $\overline{L}_i$ (and thus also L and $\hat{L}$) differ only by a different ordering (within the same column and below the diagonal element) of the multipliers $m_{i,j}$, as a consequence of the row swaps performed previously. At the end of the elimination process, in place of the initial matrix A, we will have:

$$\begin{pmatrix} u_{11} & u_{12} & u_{13} & \dots & u_{1n} \\ m_{21} & u_{22} & u_{23} & \dots & u_{2n} \\ m_{31} & m_{32} & u_{33} & \dots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & m_{n3} & \dots & u_{nn} \end{pmatrix}$$

Osservazione 3. The construction of the matrix P is done by swapping the corresponding rows of the matrix I every time a row swap is performed on matrix A.

Algorithm Scheme

```
for k = 1:n-1
    find p : |a(p,k)| = max(|a(i,k)|) for i >= k
    if k ~= p
        swap rows k and p of A
        swap rows k and p of I
    end
    for i = k+1:n
        a(i,k) = a(i,k) / a(k,k)
        for j = k+1:n
            a(i,j) = a(i,j) - a(i,k) * a(k,j)
        end
    end
end
L = eye(n) + tril(A, -1)
U = triu(A)
```

Computational Cost

$$\frac{n^3}{3} \text{ operations} + \sum_{k=1}^{n-1}(n-k) \text{ comparisons} \simeq \frac{n^3}{3} \text{ operations} + \frac{n^2}{2} \text{ comparisons}$$

```
function [L, U, P, s] = factorLUPP(A)
% FACTORLUPP Performs LU factorization with partial pivoting.
%  [L, U, P, s] = factorLUPP(A) calculates the factorization PA = LU and
%  returns the number of swaps performed.
%
%    Input:
%      A - Coefficient matrix (n x n)
%
%    Output:
%      L - Unit lower triangular matrix (n x n)
%      U - Upper triangular matrix (n x n)
%      P - Permutation matrix (n x n)
%      s - Number of swaps performed (1 x 1)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n, m] = size(A);

    % Verify if the matrix is square
    if n ~= m
        error('factorLUPP:NonSquareMatrix', 'The matrix must be square (Size: %dx%d).', n, m);
    end

    %% 3. Gaussian Elimination with Partial Pivoting
    % Initialize P as identity matrix
    P = eye(n);
    % Initialize s
    s = 0;

    for k = 1 : n-1
        % Partial Pivoting
        % Find the maximum in column k (from the pivot downwards)
        [~, r_rel] = max(abs(A(k:n, k)));

        % Convert to global index
        r_glob = r_rel + k - 1;

        % If the best pivot is not the current one, swap
        if r_glob ~= k
            % Swap rows in A
            A([k, r_glob], :) = A([r_glob, k], :);

            % Swap rows in the permutation matrix P
            P([k, r_glob], :) = P([r_glob, k], :);

            % Update variable s
            s = s + 1;
        end

        % Check for zero pivot (safety)
        if A(k,k) == 0
             error('factorLUPP:ZeroPivot', 'Singular matrix (zero pivot at row %d).', k);
        end

        % Loop over rows below
        for i = k+1 : n
            % Calculate and assign the multiplier
            A(i,k) = -A(i,k) / A(k,k);

            % Update row i
            for j = k+1:n
                A(i,j) = A(i,j) + A(i,k) * A(k,j);
            end
        end
```

```
    end

    %% 4. Output Construction
    L = eye(n) - tril(A, -1);
    U = triu(A);
    % P was already constructed during the loops
end
```

Function factorLUPP

Solving linear systems using LU with pivoting Given the factorization of the type $PA = \hat{L}U$, to determine the solution of the linear system $A\mathbf{x} = \mathbf{b}$, two triangular systems are solved:

$$\begin{cases} \hat{L}\mathbf{y} = P\mathbf{b} \\ U\mathbf{x} = \mathbf{y} \end{cases}$$

which we deduce from the relation $PA\mathbf{x} = P\mathbf{b}$, or $\hat{L}U\mathbf{x} = P\mathbf{b}$.

Computational Cost The entire procedure (factorization $PA = \hat{L}U$ and solution of the two triangular systems) consists of $\frac{n^3}{3} + n^2$ arithmetic operations. There are also $(n-1)+(n-2)+\cdots+1 = \frac{n(n-1)}{2}$ comparisons to determine the maximum pivot at each step.

Applications of LU Factorization

1. **Determinant Calculation**
   From $PA = \hat{L}U$, since $P^{-1} = P^T$,
   $$\det(A) = \det(P^T)\det(\hat{L})\det(U) = \det(P)\det(U)$$
   Since
   $$\det(P) = \det(P_{n-1})\dots\det(P_2)\det(P_1),$$
   and
   $$\det(P_j) = \begin{cases} 1 & \text{if } P_j = I \\ -1 & \text{otherwise} \end{cases}$$
   we have
   $$\det(A) = (-1)^s \prod_{i=1}^{n} a_{i,i}^{(i)}$$
   where s denotes the total number of swaps performed.
2. **Calculation of the Inverse of A**
   Denoting by $\mathbf{x}_1, \dots, \mathbf{x}_n$ the columns of the inverse matrix of A, the latter can be calculated by solving the $2n$ systems:
   $$\begin{cases} \hat{L}\mathbf{y} = P\mathbf{e}_i \\ U\mathbf{x}_i = \mathbf{y} \end{cases} \qquad i \in \{1, \dots, n\}$$
   the first being lower triangular and the second upper triangular. The overall computational cost is $\frac{n^3}{3} + n^3$ but can be reduced to n^3 if the nature of the vectors $P\mathbf{e}_i$ is taken into account.

1.5 Cholesky Method

LDU Factorization Let $A \in \mathbb{R}^{n\times n}$ be a matrix for which the LU factorization of A without pivoting exists. This decomposition can always be rewritten as:

$$A = LU = LDU_1$$

where L, U_1 are both unit triangular matrices and D is a diagonal matrix such that $(D)_{i,i} = (U)_{i,i}$. If A is symmetric, then $U_1 = {}^tL$, and we have:

$$A = LD^tL$$

L^tL Factorization If $A \in \mathbb{R}^{n\times n}$ is symmetric positive definite, since the elements of D are all positive, we have:

$$D = D^{\frac{1}{2}} D^{\frac{1}{2}}$$

where the elements of $(D^{\frac{1}{2}})_{i,i} = \sqrt{D_{i,i}}$. From which follows the theorem:

Teorema 1.7. *If $A \in \mathbb{R}^{n\times n}$ is a symmetric positive definite matrix, there exists a unique factorization:*

$$A = L_1{}^t L_1 \quad \textit{with} \quad L_1 = LD^{\frac{1}{2}}$$

where L_1 is a lower triangular matrix with non-zero diagonal elements.

In the following, we refer to the factorization mentioned in the previous theorem as the L^tL type factorization. The L^tL factorization can be obtained through the LU factorization procedure without pivoting with a cost equal to $n^3/6$, that is, about half of the LU factorization. In fact, it is possible to organize the algorithm to exploit the symmetry of all active matrices in the $n-1$ steps. We will now see how it is possible to obtain the same factorization through a "compact" technique that leads to the Cholesky method and costs about $n^3/6$ operations.

Construction of the Cholesky Method Letting $A = (a_{i,j})_{i,j\in\{1,\dots,n\}}$ and $L = (l_{i,j})_{i,j\in\{1,\dots,n\}}$, we have:

$$a_{i,j} = \sum_{k=1}^{n} l_{i,k}{}^t l_{k,j} = \sum_{k=1}^{n} l_{i,k} l_{j,k}$$

Since the matrix is symmetric, we can consider only the elements of A with $j \le i$. We obtain for $i \in \{1,\dots,n\}$, $j \in \{1,\dots,i-1\}$:

$$a_{i,j} = \sum_{k=1}^{j-1} l_{i,k} l_{j,k} + l_{i,j} l_{j,j}, \quad a_{i,i} = \sum_{k=1}^{i-1} l_{i,k}^2 + l_{i,i}^2.$$

From which the formulas for the elements of L for $i \in \{2,\dots,n\}, j \in \{1,\dots,i-1\}$ are derived:

$$l_{1,1} = \sqrt{a_{1,1}}, \qquad l_{i,j} = \frac{1}{l_{j,j}}\left(a_{i,j} - \sum_{k=1}^{j-1} l_{i,k} l_{j,k}\right), \qquad l_{i,i} = \sqrt{a_{i,i} - \sum_{k=1}^{i-1} l_{i,k}^2}.$$

It is permissible to assume that the square root operation is equivalent (in terms of time) to multiplication and division operations.

Algorithm Scheme

```
l(1,1) = sqrt(a(1,1))                    % 1 operation
for i = 2:n
    for j = 1:i-1
        l(i,j) = 0;
        for k = 1:j-1
            l(i,j) = l(i,j) + l(i,k) * l(j,k)
        end
        l(i,j) = (a(i,j) - l(i,j)) / l(j,j)    % j-1 operations + 1 op.
    end
    l(i,i) = 0;
    for k = 1:i-1
        l(i,i) = l(i,i) + l(i,k)^2
    end
    l(i,i) = sqrt(a(i,i) - l(i,i))             % i-1 operations + 1 op.
end
```

Computational Cost

$$1+\sum_{i=2}^{n}\left[i+\sum_{j=1}^{i-1}j\right]=\sum_{i=1}^{n}i+\sum_{i=1}^{n}\frac{i(i-1)}{2}$$

$$=\sum_{i=1}^{n}i+\frac{1}{2}\sum_{i=1}^{n}i^2-\frac{1}{2}\sum_{i=1}^{n}i=\frac{1}{2}\sum_{i=1}^{n}i+\frac{1}{2}\sum_{i=1}^{n}i^2$$

$$=\frac{n(n+1)}{4}+\frac{n(n+1)(2n+1)}{12}\simeq\frac{n^3}{6}$$

```matlab
function L = cholDec(A)
% CHOLDEC Performs Cholesky factorization.
%  L = cholDec(A) performs Cholesky factorization (A=L*L')
%  The matrix A must be Symmetric Positive Definite.
%
%  Input:
%    A - Coefficient matrix (n x n)
%
%  Output:
%    L - Lower triangular matrix (n x n)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n, m] = size(A);

    % Verify if the matrix is square
    if n ~= m
        error('cholDec:NonSquareMatrix', 'The coefficient matrix must be square (Size: %dx%d).', n, m);
    end

    %% 3. Cholesky Method
    L = zeros(n);

    % First element
    L(1,1) = sqrt(A(1,1));

    for i = 2:n
        % Calculate off-diagonal elements
        for j = 1:i-1
            L(i,j) = 0;     % Initialize accumulator

            for k = 1:j-1
                L(i,j) = L(i,j) + L(i,k) * L(j,k);
            end

            L(i,j) = (A(i,j) - L(i,j)) / L(j,j);
        end

        % Calculate diagonal element
        L(i,i) = 0;         % Initialize accumulator
        for k = 1:i-1
            L(i,i) = L(i,i) + L(i,k)^2;
        end

        % Final square root for the diagonal
        argRad = A(i,i) - L(i,i);

        % Safety check
        if argRad <= 0
            error('cholDec:nonDefPosMatrix','The matrix is not positive definite.')
        end

        L(i,i) = sqrt(argRad);
    end
end
```

Function cholDec

Stability and Resolution It is possible to prove that the Cholesky algorithm is stable. Recall that, moreover, the Gauss algorithm without pivoting is also stable for symmetric positive definite matrices. Many automatic routines calculate $R = {}^tL$ instead of L, but it is evident that the algorithm is the same (with careful use of indices), given the symmetry of the starting matrix A. In such cases, we will write:

$$A = {}^tRR$$

Finally, let us observe that if one wishes to solve the system $A\mathbf{x} = \mathbf{b}$ using Cholesky factorization, it will suffice, once the factorization is computed and thus L (or respectively R) is calculated, to solve the following two systems:

$$L\mathbf{y} = \mathbf{b}, \quad {}^tL\mathbf{x} = \mathbf{y}$$

(or ${}^tR\mathbf{y} = \mathbf{b}, \quad R\mathbf{x} = \mathbf{y}$ respectively).

MATLAB Functions We conclude this overview of the main direct methods for solving a linear system by describing the main MATLAB functions that implement such methods. Let A be a non-singular matrix of order n and $\mathbf{b}$ be a column vector of order n.

- **Solving diagonal systems:** If the coefficient matrix A is diagonal, `A\b` solves the system using n division operations.
- **Solving triangular systems:** If the coefficient matrix A is triangular (upper or lower), `A\b` solves the system using the substitution algorithm (backward or forward depending on the matrix structure).
- **Solving systems with arbitrary matrix:** `A\b` solves the system using the Gaussian elimination method (actually with LU factorization) with pivoting and substitution algorithms.
- **Solving symmetric positive definite systems:** `A\b` solves the system using the Cholesky method and substitution algorithms.
- **LU Factorization:** The command `[L,U,P] = lu(A)` calculates the factors L, U and the permutation matrix P such that $PA = LU$.
- **Cholesky Factorization:** If the matrix A is symmetric and positive definite, the command `R = chol(A)` calculates the upper triangular factor R such that $A = {}^tRR$.
- **Determinant Calculation:** `det(A)` calculates the determinant of the non-singular matrix A, using LU factorization.
- **Conditioning:** The command `cond(A, p)`, with `p = 1, 2, inf, 'fro'`, calculates the condition number in 1-norm, 2-norm, infinity norm, and Frobenius norm, respectively.
- **Inverse of a matrix:** `inv(A)` calculates the inverse of the matrix using LU factorization.

1.6 Stability of Gauss Methods

Backward Error Analysis With **Backward Error Analysis** (BEA), one measures the perturbation in the input data that leads to the final result, assuming that operations are performed with infinite precision. Essentially, it evaluates how much the input data is perturbed considering the output result as "exact". It was introduced by Wilkinson in 1963 with the aim of studying errors in Linear Algebra algorithms and is particularly suitable when the number of operations is high. The algorithm is said to be **stable** if the perturbation in the input data is of the order of the precision in which one is working.

Backward Error Analysis in LU Factorization

Teorema 1.8. *Let A be a matrix of order n whose elements are machine numbers. Let L and U be the factorization matrices obtained without the pivoting technique. Then there exists a matrix E such that:*

$$(A + E) = LU$$

with E such that:

$$\|E\| \leq nu\|L\|\|U\|$$

where $u = \frac{eps}{2}$.

Since $U = A^{(n-1)}$, the stability of the Gaussian elimination method can be better understood by measuring the growth of the elements of the matrices $A^{(k)}$ and, therefore, by measuring the growth factor.

Growth Factor

Definizione 1.10. The growth factor is defined as the following quantity:

$$\rho = \frac{\max(\alpha_1, \ldots, \alpha_n)}{\alpha_1}$$

with:

$$\alpha_1 = \max_{1 \leq i,j \leq n} |a_{i,j}|$$

$$\alpha_k = \max_{1 \leq i,j \leq n} |a_{i,j}^{(k)}|, \quad k \in \{2, \ldots, n\}$$

where $A^{(k)} = (a_{i,j}^{(k)})_{i,j \in \{1,\ldots,n\}}$ is the matrix obtained at the end of the k-th step of the LU factorization.

```
function [L, U, rho] = factorLUr(A)
% FACTORLUR Performs LU factorization on a matrix A.
%  [L, U, rho] = factorLUr(A) factorizes A into L*U (Gauss no pivoting)
%  and returns the growth factor rho.
%
%  Input:
%    A - Coefficient matrix (n x n)
%
%  Output:
%    L   - Unit lower triangular matrix (n x n)
%    U   - Upper triangular matrix (n x n)
%    rho - Growth factor for stability

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n, m] = size(A);

    % Check for square matrix
    if n ~= m
        error('factorLUr:NonSquareMatrix', 'The matrix must be square (%dx%d).', n, m);
    end

    %% 3. Growth Factor Initialization
    % Pre-calculate module for rho
    absA = abs(A);

    % Vector to track local maxima at each step
    rhos = zeros(1,n);

    % Store the initial absolute maximum (denominator of rho)
    rhos(1) = max(absA(:));

    %% 4. Gaussian Elimination
    for k = 1 : n-1
        % Check for zero pivot
        if A(k,k) == 0
            error('factorLUr:ZeroPivot', 'Zero pivot at row %d.', k);
        end

        % Loop over rows below (i)
```

```
        for i = k+1 : n
            % Calculate multiplier (saved with negative sign)
            A(i,k) = -A(i,k) / A(k,k);

            % Update row i (Elimination)
            for j = k+1:n
                A(i,j) = A(i,j) + A(i,k) * A(k,j);
            end
        end

        % --- Calculate Local Maximum for Rho ---
        % Extract current active submatrix (dimensions n-k+1)
        tempSub = abs(A(k:n, k:n));

        % Zero out multipliers: they are in local column 1,
        % from row 2 onwards (local indices). They must be excluded from max on U.
        tempSub(2:end, 1) = 0;

        % Store the maximum value found in the active part/U
        rhos(k+1) = max(tempSub(:));
    end

    %% 5. Output Construction
    L = eye(n) - tril(A, -1);
    U = triu(A);

    % Final rho calculation: max growth / initial max
    rho = max(rhos) / rhos(1);
end
```

Function LUr

The definition holds both with pivoting and without pivoting. Observe that, although pivoting ensures that the multipliers are all smaller than 1, the elements of the matrices $A^{(k)}$ can still grow arbitrarily.

Backward Error Analysis in LUPP Factorization

Teorema 1.9. *Let L, U be the factorization matrices with partial pivoting. We have:*

$$P(A+E) = LU$$

with E such that:

$$\|E\| \leq n^3 u \rho \|A\|_\infty$$

where $u = \frac{eps}{2}$.

We ask ourselves: how large can ρ be?

Esempio 1.8. Consider the linear system:

$$\begin{cases} \varepsilon x_1 + x_2 = 1 + \varepsilon \\ x_1 = 1 \end{cases}$$

If we choose $\varepsilon = 0.5 \cdot 10^{-5}$, we have:

$$L = \begin{pmatrix} 1 & 0 \\ 2 \cdot 10^5 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 0.5 \cdot 10^{-5} & 1 \\ 0 & -2 \cdot 10^5 \end{pmatrix}$$

Since $\|L\|_\infty = (1 + 2 \cdot 10^5)$ and $\|U\|_\infty = 2 \cdot 10^5$, by Theorem 1 we have:

$$\|E\|_\infty \leq nu\|L\|_\infty \|U\|_\infty = \text{eps}(1 + 2 \cdot 10^5) 2 \cdot 10^5 \sim 0.9 \cdot 10^{-5} < 0.5 \cdot 10^{-4}$$

Furthermore, the growth factor is:

$$\rho = \frac{\max(1, 2 \cdot 10^5)}{1} = 2 \cdot 10^5$$

Esempio 1.9. Applying pivoting, we have:

$$L = \begin{pmatrix} 1 & 0 \\ 5.00 \cdot 10^{-6} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Since the growth factor is $\rho = 1$ and $\|A\|_\infty = 1 + 0.5 \cdot 10^{-5}$, by Theorem 2 we have:

$$\|E\|_\infty \leq 8 \cdot u \cdot 1 \cdot (1 + 0.5 \cdot 10^{-5}) \sim 0.9 \cdot 10^{-15} < 0.5 \cdot 10^{-14}$$

Growth of the factor ρ in pivoting

Teorema 1.10. *For LU factorization with partial pivoting (and for Gaussian elimination with pivoting), for every matrix A of order n, we have:*

$$\rho \leq 2^{n-1}$$

```
function [L, U, P, s, rho] = factorLUPPr(A)
% FACTORLUPPR LU Factorization with Partial Pivoting (PA = LU).
%  [L, U, P, s, rho] = factorLUPPr(A)
%
%  Input:
%    A - Coefficient matrix (n x n)
%
%  Output:
%    L   - Unit lower triangular matrix
%    U   - Upper triangular matrix
%    P   - Permutation matrix
%    s   - Number of swaps performed (for the determinant)
%    rho - Growth factor (numerical stability)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    % Extract dimensions
    [n, m] = size(A);

    % Verify if the matrix is square
    if n ~= m
        error('factorLUPP:NonSquareMatrix', 'The matrix must be square (Size: %dx%d).', n, m);
    end

    %% 3. Initialization
    absA = abs(A);
    P = eye(n);       % Initial permutation matrix
    s = 0;            % Swap counter

    % Initialize vector for rho
    rhos = zeros(1,n);
    if n > 0
        rhos(1) = max(absA(:)); % Initial maximum
    else
        rhos(1) = 0;
    end

    %% 4. Gaussian Elimination with Partial Pivoting
    for k = 1 : n-1
        % Partial Pivoting
        % Find the maximum in column k (from the pivot downwards)
        [~, r_rel] = max(abs(A(k:n, k)));

        % Convert to global index
        r_glob = r_rel + k - 1;

        % If the best pivot is not the current one, swap
        if r_glob ~= k
            % Swap rows in A (including previous multipliers)
            A([k, r_glob], :) = A([r_glob, k], :);
```

```
            % Swap rows in P
            P([k, r_glob], :) = P([r_glob, k], :);

            % Update variable s
            s = s + 1;
        end

        % Check for zero pivot (safety)
        if A(k,k) == 0
             error('factorLUPPr:SingularMatrix', 'Singular matrix: zero pivot at row %d.', k);
        end

        % Elimination
        for i = k+1 : n
            % Calculate and assign the multiplier
            A(i,k) = -A(i,k) / A(k,k);

            % Update row i
            for j = k+1:n
                A(i,j) = A(i,j) + A(i,k) * A(k,j);
            end
        end

        % Calculate rho (Local Maximum)
        % Extract active submatrix
        tempSub = abs(A(k:n, k:n));

        % Zero out the multipliers just calculated
        % to measure only the growth of elements in U.
        tempSub(2:end, 1) = 0;

        rhos(k+1) = max(tempSub(:));
    end

    %% 5. Output Construction
    % L: Identity - lower triangular part (sign compensated)
    L = eye(n) - tril(A, -1);

    % U: Upper triangular part
    U = triu(A);

    % Calculate final Rho
    rho = max(rhos) / rhos(1);
end
```

Function LUPPr

Unfortunately, there exist matrices that have a growth factor equal to 2^{n-1}. One of these is the Wilkinson matrix:

$$a_{ij} = \begin{cases} 1, & i = j \\ -1, & i > j \\ 1, & j = n \\ 0 & \text{elsewhere} \end{cases}$$

Total Pivoting One chooses a pair (r, s), with $r, s \geq k$ such that:

$$|a_{rs}^{(k)}| = \max_{k \leq i,j \leq n} |a_{ij}^{(k)}|$$

and swaps the k-th equation with the r-th equation and the k-th unknown (with its coefficient) with the s-th unknown. In LU factorization with total pivoting, two permutation matrices P and Q are constructed such that:

$$PAQ = LU$$

The linear system $A\mathbf{x} = \mathbf{b}$ becomes equivalent to ($Q^{-1} = {}^tQ$):

$$PAQ^tQ\mathbf{x} = P\mathbf{b}$$

Setting ${}^tQ\mathbf{x} = \mathbf{y}$, we have $PAQ\mathbf{y} = P\mathbf{b}$, and using the LU factorization, we have $LU\mathbf{y} = P\mathbf{b}$. Setting $U\mathbf{y} = \mathbf{z}$, one calculates the solution vector of $L\mathbf{z} = P\mathbf{b}$ and then the solution vector of $U\mathbf{y} = \mathbf{z}$. Finally, the solution $\mathbf{x}$

of the initial system is calculated as $\mathbf{x} = Q\mathbf{y}$. In LU factorization with total pivoting, the growth factor ρ grows very slowly; indeed, it is proven that:

$$\rho \leq \sqrt{n \cdot 2^1 \cdot 3^{\frac{1}{2}} \cdot 4^{\frac{1}{3}} \cdots n^{\frac{1}{n-1}}}.$$

```
function [L, U, P, Q] = factorLUTP(A)
% FACTORLUTP Performs LU factorization with total pivoting.
%   [L, U, P, Q] = factorLUTP(A) calculates the factorization PAQ = LU
%   where P and Q are permutation matrices.
%
%   Input:
%     A - Square coefficient matrix (n x n)
%
%   Output:
%     L - Unit lower triangular matrix (n x n)
%     U - Upper triangular matrix (n x n)
%     P - Row permutation matrix (n x n)
%     Q - Column permutation matrix (n x n)

    %% 1. Input Validation
    arguments
        A (:,:) double {mustBeNumeric}
    end

    %% 2. Check Dimensions
    [n, m] = size(A);
    if n ~= m
        error('factorLUTP:NonSquareMatrix', 'The matrix must be square (Size: %dx%d).', n, m);
    end

    %% 3. Initialization
    % Initialize permutation matrices as identity
    P = eye(n);
    Q = eye(n);

    %% 4. Gaussian Elimination with Total Pivoting
    for k = 1 : n-1
        % Pivot search
        subA = abs(A(k:n, k:n)); % Extract current submatrix

        % Find the maximum value and its linear index
        [max_val, idx_lin] = max(subA(:));

        % Convert linear index to row/col indices relative to submatrix
        [r_local, c_local] = ind2sub(size(subA), idx_lin);

        % Convert to global indices (relative to the full matrix A)
        r_glob = r_local + k - 1;
        c_glob = c_local + k - 1;

        % Check for singularity
        if max_val == 0
             error('factorLUTP:SingularMatrix', 'The matrix is singular (zero pivot).');
        end

        % Swap rows (A and P)
        if r_glob ~= k
            A([k, r_glob], :) = A([r_glob, k], :);
            P([k, r_glob], :) = P([r_glob, k], :);
        end

        % Swap columns (A and Q)
        if c_glob ~= k
            A(:, [k, c_glob]) = A(:, [c_glob, k]);
            Q(:, [k, c_glob]) = Q(:, [c_glob, k]);
        end

        % Gaussian Elimination
        A(k+1:n, k) = -A(k+1:n, k) / A(k, k);
        A(k+1:n, k+1:n) = A(k+1:n, k+1:n) + A(k+1:n, k) * A(k, k+1:n);
    end

    %% 5. Output Construction
    U = triu(A);
```

```
    L = eye(n) - tril(A, -1);

end
```

Function factorLUTP

Conclusions on Stability Gaussian elimination without pivoting has a growth factor equal to 1 for symmetric positive definite matrices and ≤ 2 for row or column diagonally dominant matrices. It follows that Gauss's method without pivoting is stable when applied to symmetric positive definite or diagonally dominant matrices.

Summary Table

- **GE** (or $A = LU$): not stable in general (stable if the matrix is symmetric positive definite or diagonally dominant).
- **GEPP** (or $PA = LU$): almost always stable.
- **GETP** (or $PAQ = LU$): stable.

Residual and its Evaluation With reference to square linear systems, let $\mathbf{x}$ be the exact solution and $\mathbf{x}^*$ be the computed solution. We demonstrate that the following estimate of the relative error holds:

Proposizione 1.11.

$$\frac{\|\mathbf{x} - \mathbf{x}^*\|}{\|\mathbf{x}\|} \leq K(A) \frac{\|A\mathbf{x}^* - \mathbf{b}\|}{\|\mathbf{b}\|}.$$

Proof. We have:

$$\|\mathbf{x} - \mathbf{x}^*\| = \|A^{-1}(A\mathbf{x} - A\mathbf{x}^*)\| \leq \|A^{-1}\|\|\mathbf{b} - A\mathbf{x}^*\| = \|A^{-1}\|\|\mathbf{b}\| \frac{\|\mathbf{b} - A\mathbf{x}^*\|}{\|\mathbf{b}\|}.$$

Furthermore, since $\|\mathbf{b}\| \leq \|A\|\|\mathbf{x}\|$, we obtain:

$$\frac{\|\mathbf{x} - \mathbf{x}^*\|}{\|\mathbf{x}\|} \leq \|A\|\|A^{-1}\| \frac{\|A\mathbf{x}^* - \mathbf{b}\|}{\|\mathbf{b}\|}.$$

□

From the previous estimate, we deduce that the norm of the residual $\|A\mathbf{x}^* - \mathbf{b}\|$, even if small (perhaps of the order of eps), does not guarantee a small relative error, particularly for matrices with a high condition number. Therefore, the quantity $\|A\mathbf{x}^* - \mathbf{b}\|$ must be used with caution as an *a posteriori* error estimator.

1.7 Exercises

1. Consider the matrix
$$A = rand(10), \quad A = A * {}^t A.$$
   - Verify if it is symmetric;
   - Verify if it is symmetric and positive definite;
2. Consider the matrix
$$A = rand(10) + 100 * diag(ones(1, 10)).$$
   Verify if it is row diagonally dominant.
3. Write a MATLAB function that checks if a matrix is column diagonally dominant.

4. Consider the matrix

$$A(i,j) = \begin{cases} -1 & i > j \\ 0 & i < j, \\ 100 & i = j \end{cases} \qquad i,j \in \{1,\dots,n\}$$

with $n = 15$.

- Verify if it is column diagonally dominant;
- Verify if it is row diagonally dominant.

5. Consider the linear system $A\mathbf{x} = \mathbf{b}$ with

$$A = tril(rand(10)), \quad \mathbf{b} = sum(A,2).$$

Solve the system using the forward substitution method.

6. Consider the linear system $A\mathbf{x} = \mathbf{b}$ with

$$A = triu(rand(10)), \quad \mathbf{b} = sum(A,2).$$

- Solve the system using the backward substitution method;
- calculate the inverse of A.

7. Solve the linear system $A\mathbf{x} = \mathbf{b}$ with

$$A = diag(diag(rand(10))), \quad \mathbf{b} = sum(A,2).$$

8. Calculate the inverse of the matrix

$$A = tril(rand(20)).$$

9. Consider the linear system $A\mathbf{x} = \mathbf{b}$ with

$$A = rand(100), \quad \mathbf{b} = sum(A,2).$$

Solve the system using Gauss's method.

10. Consider the system of linear equations $Ax = b$ of order $n = 15$, with

$$A(i,j) = \begin{cases} -1 & i > j \\ 0 & i < j, \\ 100 & i = j \end{cases} \qquad i,j \in \{1,\dots,n\}$$

and

$$\mathbf{b} = {}^t(1,1,\dots,1).$$

- Calculate the condition number and the maximum number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the solution vector using the forward substitution method and report its first two components with the significant digits that can certainly be considered correct.
- Compare the obtained solution (vector x) with the solution obtained using the default MATLAB function (vector y). By how much do they differ at most?

11. Consider the system of linear equations $Ax = b$ of order $n = 10$, with

$$A = \begin{pmatrix} \frac{1}{2} & 2 & 4 & 4 & \dots & 4 & 4 \\ 1 & \frac{1}{3} & 2 & 4 & \dots & & 4 \\ 2 & 1 & \frac{1}{4} & \ddots & \ddots & & \vdots \\ 0 & 2 & \ddots & \ddots & \ddots & 4 & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 2 & 4 \\ 0 & \dots & 0 & 2 & 1 & \frac{1}{n} & 2 \\ 0 & \dots & \dots & 0 & 2 & 1 & \frac{1}{n+1} \end{pmatrix}$$

and

$$\mathbf{b} = (b_i)_{i\in\{1,\dots,n\}}, \quad b(i) = \sum_{j=1}^{n} A_{i,j}.$$

- Calculate the condition number and the maximum number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the solution vector using the Gaussian elimination method and report its first two components with the significant digits that can certainly be considered correct.
- Compare the obtained solution (vector x) with the solution obtained using the default MATLAB function (vector y). By how much do they differ at most?
- Compare the obtained solution (vector x) with the vector $t = {}^t(1, \dots, 1)$ which represents the exact solution of the system. By how much do they differ at most?

12. Consider the system of linear equations $A\mathbf{x} = \mathbf{b}$ of order $n = 100$, with

$$A = \begin{pmatrix} 1 & 1 & 4 & 0 & \dots & 0 & 0 \\ 6 & 3 & 1 & 4 & \ddots & & 0 \\ 0 & 6 & 5 & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & \ddots & \ddots & 4 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & 1 & 4 \\ 0 & \dots & 0 & 0 & 6 & \ddots & 1 \\ 0 & \dots & \dots & \dots & 0 & 6 & 2n-1 \end{pmatrix}$$

and

$$\mathbf{b} = (b_i)_{i \in \{1,\dots,n\}}, \quad b(i) = \sum_{j=1}^{n} A_{i,j}.$$

- Calculate the condition number and the maximum number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the solution vector using the Gaussian elimination method and report the first two components of the solution vector with the significant digits that can certainly be considered correct.
- Compare the obtained solution (vector x) with the vector $t = {}^t(1, \dots, 1)$ which represents the exact solution of the system. By how much do they differ at most?

13. Consider the linear system $A\mathbf{x} = \mathbf{b}$ of order $n = 18$, where

$$A_{i,j} = \cos\left((j-1)\frac{2i-1}{2n}\pi\right), \quad i, j \in \{1, \dots, n\},$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the approximate solution of the system using Gauss's method and calculate the relative error. How many correct significant digits are there?
- Calculate the approximate solution of the system using Gauss's method with partial pivoting and calculate the relative error. How many correct significant digits are there?
- Which method is more stable?

14. Consider the linear system $A\mathbf{x} = \mathbf{b}$ of order $n = 50$, where

$$A = \begin{pmatrix} 3 & 2 & 2 & 2 & \dots & 2 & 6 \\ 2 & \frac{5}{2} & 2 & 2 & \ddots & & 2 \\ 2 & 2 & \frac{7}{3} & 2 & \ddots & \ddots & \vdots \\ 2 & 2 & 2 & \ddots & \ddots & 2 & 2 \\ \vdots & \vdots & \ddots & \ddots & \ddots & 2 & 2 \\ 2 & 2 & \dots & 2 & 2 & 2+\frac{1}{n-1} & 2 \\ 6 & 2 & 2 & \dots & 2 & 2 & 2+\frac{1}{n} \end{pmatrix}$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the approximate solution of the system using Gauss's method and calculate the relative error. How many correct significant digits are there?
- Calculate the approximate solution of the system using Gauss's method with partial pivoting and calculate the relative error. How many correct significant digits are there?
- Which method is more stable?

15. Consider the problem of $n = 80$

$$AX = B, \quad A \in \mathbb{R}^{n\times n}, X \in \mathbb{R}^{n\times 3}, B \in \mathbb{R}^{n\times 3},$$

where

$$A = \begin{pmatrix} 5 & 0 & \frac{1}{2} & 0 & \dots & 0 \\ 0 & 9 & 0 & \frac{1}{3} & \ddots & \vdots \\ \vdots & 0 & 13 & \ddots & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \frac{1}{n-1} \\ \frac{1}{3} & \ddots & 0 & \dots & 4(n-1)+1 & 0 \\ 0 & \frac{1}{3} & 0 & \dots & 0 & 4n+1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 2 \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \\ 1 & 2 & n-1 \\ 1 & 2 & n \end{pmatrix}.$$

(In this case the transcription is correct but check better with pdf exercises for confirmation on the structure.)

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the solution matrix X.
- Calculate the approximate solution Y using appropriately the LU factorization of the matrix A. Report the components of the first row of matrix Y with the digits that can be considered correct.
- Calculate the relative residual in infinity norm.

16. Write a MATLAB function that appropriately implements the Gaussian elimination method to solve a tridiagonal matrix system.

17. Write a MATLAB function that appropriately implements the Gaussian elimination method to solve an upper Hessenberg matrix system.

18. Consider the linear system $A\mathbf{x} = \mathbf{b}$ of order $n = 80$, where

$$A = \begin{pmatrix} 4 & 2 & 2 & 2 & \dots & 2 & 8 \\ 4 & \frac{7}{2} & 2 & 2 & \ddots & & 2 \\ 4 & 4 & \frac{10}{3} & 2 & \ddots & \ddots & \vdots \\ 4 & 4 & 4 & \ddots & \ddots & 2 & 2 \\ \vdots & \vdots & \ddots & \ddots & \ddots & 2 & 2 \\ 4 & 4 & \dots & 4 & 4 & 3+\frac{1}{n-1} & 2 \\ 10 & 4 & 4 & \dots & 4 & 4 & 3+\frac{1}{n} \end{pmatrix}$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.

- Calculate the approximate solution of the system using appropriately Gauss's method and calculate the relative error. How many correct significant digits are there?
- Calculate the determinant of the matrix A.
- Calculate the inverse of the matrix A.

19. Given the matrix of order $n = 40$

$$A = \begin{pmatrix} \frac{1}{10} & 0 & \frac{1}{2} & 0 & \dots & 0 \\ 0 & \frac{1}{10} & 0 & \frac{1}{2} & \ddots & \vdots \\ \vdots & 0 & \frac{1}{10} & \ddots & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \frac{1}{2} \\ \frac{1}{2} & \ddots & 0 & \dots & \frac{1}{10} & 0 \\ 0 & \frac{1}{3} & 0 & \dots & 0 & \frac{1}{10} \end{pmatrix},$$

Calculate the condition number and the number of correct significant digits that can be expected when calculating:

- its determinant;
- its inverse.

20. Given the matrix of order $n = 40$

$$A = \begin{pmatrix} 10 & 1 & \frac{1}{2} & 3 & \dots & 3 & 3 \\ 0 & 10 & 1 & \frac{1}{3} & \ddots & & 3 \\ 2 & 0 & 10 & 1 & \ddots & \ddots & \vdots \\ \vdots & 2 & 0 & \ddots & \ddots & \frac{1}{n-2} & 3 \\ 2 & & \ddots & \ddots & \ddots & 1 & \frac{1}{n-1} \\ \frac{1}{2} & \ddots & & 2 & 0 & 10 & 1 \\ 1 & \frac{1}{2} & 2 & \dots & 2 & 0 & 10 \end{pmatrix}$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the approximate solution of the system using appropriately Gauss's method and calculate the relative error. How many correct significant digits are there?
- Calculate the determinant of the matrix A.
- Calculate the inverse of the matrix A.

21. Consider the linear system $A\mathbf{x} = \mathbf{b}$ of order $n = 100$, where

$$A = \begin{pmatrix} 9 & 0 & 2 & 0 & \dots & 0 & 1 \\ 4 & 5 & 0 & 2 & 0 & \dots & 0 \\ 4 & 0 & 5 & \ddots & \ddots & \ddots & \vdots \\ \vdots & 0 & \ddots & \ddots & 0 & 2 & 0 \\ \vdots & \vdots & \ddots & \ddots & 5 & 0 & 2 \\ 4 & 0 & \dots & 0 & 0 & 5 & 0 \\ 4 & 0 & \dots & 0 & 0 & 0 & 5 \end{pmatrix}, \quad A = A * {}^tA$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Propose one or more numerical methods to calculate the solution of the system $Ax = b$ with the maximum possible precision. Calculate the relative error in infinity norm. How many correct significant digits are there in the calculated solution?
- Motivate the choice of the method made and comment on the obtained results.
- Calculate the determinant of A.
- Calculate the inverse of A.
- Verify that the growth factor ρ relative to the LU factorization is equal to 1.

22. Consider the linear system $A\mathbf{x} = \mathbf{b}$ of order $n = 100$, where

$$A(i,j) = \begin{cases} -7 & i = j \\ -2 & |i-j| = 1 \\ 1 & |i-j| = 4 \\ 0 & \text{otherwise} \end{cases}$$

and

$$b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\},$$

whose exact solution is $\mathbf{x} = {}^t(1, \dots, 1)$.

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Propose one or more numerical methods to calculate the solution of the system $Ax = b$ with the maximum possible precision. Calculate the relative error in infinity norm. How many correct significant digits are there in the calculated solution?
- Evaluate the stability of the proposed methods by calculating the growth factor ρ.
- Motivate the choice of the method made and comment on the obtained results.
- Calculate the determinant of A.
- Calculate the inverse of A.
- Verify that the growth factor ρ relative to the LU factorization is less than 2.

23. Consider the linear system $Ax = b$ of order $n = 80$, where

$$A = \begin{pmatrix} 15 & -3 & -3 & -3 & \dots & -3 \\ 2 & 15 & -3 & -3 & \ddots & \vdots \\ 0 & 2 & 15 & \ddots & \ddots & -3 \\ \vdots & & \ddots & \ddots & \ddots & -3 \\ \vdots & \ddots & & 2 & 15 & -3 \\ 0 & \dots & \dots & 0 & 2 & 4 \end{pmatrix}$$

and

$$\mathbf{b} = {}^t(1, \dots, 1).$$

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Calculate the approximate solution y of the system using appropriately Gauss's method. Report the first 2 components of the vector y with the digits that can be considered correct.
- Verify the stability of the Gauss method used, evaluating its growth factor ρ.
- Calculate the infinity norm of the residual $Ay - b$.
- Calculate the determinant of A.
- Calculate the inverse of A.

24. Consider the linear system $Ax = b$ of order $n = 60$, with A being the Wilkinson matrix

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 1 \\ -1 & 1 & 0 & 0 & & \vdots & 1 \\ -1 & -1 & 1 & \ddots & \ddots & \vdots & \vdots \\ \vdots & -1 & \ddots & \ddots & 0 & 0 & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 & 0 & 1 \\ -1 & -1 & \dots & -1 & -1 & 1 & 1 \\ -1 & -1 & \dots & -1 & -1 & -1 & 1 \end{pmatrix}$$

and $\mathbf{b}(i) = \sum_{j=1}^{n} a_{ij}, \quad i = 1 : n$

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Establish based on any properties of the matrix which variant of Gauss's method or factorization is more convenient to use to guarantee maximum stability at minimum cost.
- Calculate the growth factor ρ if GE is used.
- Calculate the solution and the relative error committed.
- Calculate the growth factor ρ if GEPP is used.
- Calculate the solution and the relative error committed.
- Make appropriate considerations on the obtained results.
- Calculate the solution with GETP and the relative error committed. Appropriately use the default MATLAB function lu.
- Make appropriate considerations on the obtained results.
- Repeat the same exercise with $n = 50$.

25. Write a MATLAB function that calculates the growth factor ρ for the evaluation of the algorithmic error in GE and GEPP.

26. Consider the linear system $Ax = b$ of order $n = 60$, with $\mathbf{b}(i) = i \quad i = 1, 2 \dots, n$ and $A = G * G'$, being

$$\begin{aligned} G(i, i+1) &= \cos(2i+1), \quad i = 1, 2, .., , n-1 \\ G(i, i) &= n+1, \quad i = 1, 2, \dots, n \\ G(i, 1) &= 1, \quad i \geq 2 \\ 0 & \quad \text{elsewhere} \end{aligned}$$

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.
- Establish based on any properties of the matrix which variant of Gauss's method or factorization is more convenient to use to guarantee maximum stability at minimum cost.
- Calculate the growth factor ρ if GEPP is used.
- Calculate the solution and the relative error committed.
- Make appropriate considerations on the obtained results.

27. Consider the linear systems

$$Gx = c$$

$$Gy = d$$

with $G = (g_{i,j})_{i,j=1,\dots,n}$ matrix from the previous exercise and with

$$c(i) = \sum_{j=1}^{n} g_{i,j}, \quad i = 1, \dots, n,$$

$$d(i) = 1, \quad i = 1 : n$$

- Calculate the condition number and the number of correct significant digits that can be expected in the calculation of the approximate solution.

- Establish based on any properties of the matrix which variant of Gauss's method or factorization is more convenient to use to guarantee maximum stability at minimum cost.
- Calculate the growth factor ρ if GEPP is used.
- Calculate the solution and the relative error committed.
- Make appropriate considerations on the obtained results.

28. Consider the linear system $Ax = b$ of order $n = 80$ with

$$A = \begin{pmatrix} 10 & 1 & 0 & \dots & 0 & 2 & 200 \\ 1 & 10 & 2 & \ddots & & \ddots & 2 \\ 0 & 1 & 10 & \ddots & \ddots & & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & & \ddots & \ddots & \ddots & n-2 & 0 \\ 2 & \ddots & & \ddots & 1 & 10 & n-1 \\ 100 & 2 & 0 & \dots & 0 & 1 & 10 \end{pmatrix}$$

and

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i = 1, \dots, n.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calculate the relative error. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Comment on the obtained results.

29. Determine the inverse matrix of the matrix given in the previous exercise. Indicate:

- the procedure used
- the most adequate factorization to guarantee stability
- the computational cost.

30. Consider the linear system $Ax = b$ of order $n = 100$ with

$$A = \begin{pmatrix} 2 & 0 & \dots & 0 & 1 & 3 \\ 0 & 4 & 0 & \dots & 0 & 1 \\ 2 & 0 & 6 & \ddots & \vdots & 0 \\ \vdots & 2 & \ddots & \ddots & 0 & \vdots \\ 2 & \vdots & \ddots & \ddots & 2n-2 & 0 \\ 2 & 2 & \dots & 2 & 0 & 2n \end{pmatrix}$$

and

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?

- Since it is known that the solution is $\mathbf{x} = {}^t(1, 1, \ldots, 1)$, calculate the relative error. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Comment on the obtained results.

31. Consider the linear system $AX = B$ of order $n = 80$ with

$$A = (a_{i,j})_{i,j=1,\ldots,n} = \left(\sqrt{\frac{2}{n+1}} \sin\left(\frac{\pi i j}{n+1}\right) \right)_{i,j=1,\ldots,n}$$

and

$$\mathbf{b} = \begin{pmatrix} \sum_{j=1}^{n} A_{1,j} & 1 & 2 \\ \sum_{j=1}^{n} A_{2,j} & 1 & 2 \\ \vdots & \vdots & \vdots \\ \sum_{j=1}^{n} A_{n,j} & 1 & 2 \end{pmatrix}.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 rows of the solution matrix with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Calculate the relative residual. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Comment on the obtained results.

32. Consider the linear system $Ax = b$ of order $n = 110$ with

$$A = \begin{pmatrix} 3 & 0 & 0 & 0 & \ldots & 0 & 2 \\ 2 & 1 & 0 & 0 & \ldots & 0 & 2 \\ 2 & 0 & 1 & \ddots & \ddots & \vdots & \vdots \\ \vdots & 0 & \ddots & \ddots & 0 & 0 & 2 \\ \vdots & \vdots & \ddots & \ddots & 1 & 0 & 2 \\ 2 & 0 & \ldots & 0 & 0 & 1 & 2 \\ 2 & 0 & \ldots & 0 & 0 & 0 & 3 \end{pmatrix}$$

and

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\ldots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \ldots, n\}.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1, 1, \ldots, 1)$, calculate the relative error. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Comment on the obtained results.

33. Consider the linear system $Ax = b$ of order $n = 120$ with

$$A = \begin{pmatrix} 2(n+1) & 2 & 0 & 0 & \ldots & 0 \\ -1 & 2(n+1) & 4 & 0 & \ldots & 0 \\ 0 & -1 & 2(n+1) & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & & \ddots & -1 & 2(n+1) & 2(n-1) \\ 0 & 0 & \ldots & 0 & -1 & 2(n+1) \end{pmatrix}.$$

and

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1,\dots,n\}.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1,1,\dots,1)$, calculate the relative error. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Calculate the inverse B of the matrix A with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the value of the components $B(3,4)$ and $B(4,4)$ with the digits that can be considered correct.
- Comment on the obtained results.

34. Given the matrix of order $n = 80$

$$A = \begin{pmatrix} \frac{1}{4} & -2 & -2 & \dots & -2 & 5 \\ 2 & \frac{1}{8} & -2 & \ddots & & -2 \\ 2 & 2 & \frac{1}{12} & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -2 & -2 \\ 2 & & \ddots & \ddots & \frac{1}{4n-4} & -2 \\ 9 & 2 & \dots & 2 & 2 & \frac{1}{4n} \end{pmatrix}.$$

- Report the MATLAB instructions used for its construction.
- Calculate the determinant of the matrix A with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the value of the determinant with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?

35. Consider the linear system $Ax = b$ of order $n = 80$ with

$$A = (a_{i,j})_{i,j=1,\dots,n} \quad \text{with} \quad a_{i,j} = \min\{i,j\}$$

and

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1,\dots,n\}.$$

- Report the MATLAB instructions used for the construction of A and b.
- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1,1,\dots,1)$, calculate the relative error. Report its value and the MATLAB instructions used. How many correct significant digits are there?
- Calculate the determinant of the matrix A with the maximum possible precision using an appropriate numerical procedure. Report the MATLAB instructions used and the value of the determinant with the digits that can be considered correct.
- Comment on the obtained results.

36. Consider the linear system $Ax = b$ of order $n = 90$ with

$$A = \begin{pmatrix} 4 & 0 & 0 & \dots & 0 & 3 & 2 \\ 0 & 5 & 0 & & & 3 & 3 \\ 0 & 0 & 6 & \ddots & \vdots & \vdots & \vdots \\ \vdots & & \ddots & \ddots & 0 & \vdots & \vdots \\ 0 & \dots & \dots & 0 & \ddots & 3 & n-1 \\ 2 & 2 & \dots & 2 & 2 & (n-1)+3 & n \\ 2 & 3 & \dots & \dots & n-1 & n & n+3 \end{pmatrix}$$

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calculate the relative error. How many correct significant digits are there?
- Comment on the obtained results.

37. Consider the linear system $Ax = b$ of order $n = 90$ with

$$A = \begin{pmatrix} 1 & 2 & 2 & \dots & 2 & 2 \\ 2 & 3 & 2 & 2 & & 2 \\ 2 & 2 & 5 & \ddots & \ddots & \vdots \\ \vdots & 2 & \ddots & \ddots & 2 & 2 \\ 2 & & \ddots & 2 & 2n-3 & 2 \\ 2 & 2 & \dots & 2 & 2 & 2n-1 \end{pmatrix}$$

$$\mathbf{b} = {}^t\{b_i\}_{i=1,\dots,n}, \quad b_i = \sum_{j=1}^{n} A_{i,j}, \quad i \in \{1, \dots, n\}.$$

- Calculate the approximate solution of the system with the maximum possible precision using an appropriate numerical procedure. Report the first 2 components of the solution vector with the digits that can be considered correct.
- Motivate the choice of the method.
- What is the computational cost of the numerical method used?
- Since it is known that the solution is $\mathbf{x} = {}^t(1, 1, \dots, 1)$, calculate the relative error. How many correct significant digits are there?
- Comment on the obtained results.

2 Numerical Methods for Solving a Rectangular Linear System in the Least Squares Sense

The Linear Least Squares Problem Consider a linear system of the type

$$A\mathbf{x} = \mathbf{b}$$

where A is a matrix of order $m \times n$, with $m \neq n$, $\mathbf{x}$ is a vector of length n, and $\mathbf{b}$ is a vector of length m. Many real-world situations, for example in statistical applications, signal theory problems, and astronomy, can be modeled using a linear system in which the coefficient matrix is rectangular (or square but singular). In such cases, a solution may not exist at all, or there may be infinite solutions.

- If $m > n$, i.e., if the number of equations is greater than the number of unknowns, the system is called **overdetermined** and does not admit exact solutions.
- If instead $m < n$, the system is called **underdetermined** and admits infinite solutions.

In these cases, instead of trying to determine the vector $\mathbf{x}$ that verifies the identity $A\mathbf{x} = \mathbf{b}$, we seek, if it exists, a vector $\mathbf{x}$ such that the distance of $A\mathbf{x}$ from $\mathbf{b}$ is the minimum possible. In other words, the problem is reduced to a minimization problem: determine an $\overline{\mathbf{x}}$ of length n such that, setting $r(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ (called the residual), it results that

$$\|r(\overline{\mathbf{x}})\| = \min_{\mathbf{x}} \|r(\mathbf{x})\| = \min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|$$

If the norm used is the Euclidean norm, i.e., the 2-norm,

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{{}^t\mathbf{x}\mathbf{x}}$$

the problem is called the **Linear Least Squares (LLS) problem**. The name stems from the fact that we seek the minimum of the quantity

$$\min_{\mathbf{x}=(x_1,\ldots,x_n)} \|A\mathbf{x} - \mathbf{b}\|_2 = \min_{\mathbf{x}=(x_1,\ldots,x_n)} \sqrt{\sum_{i=1}^{m} \left[b_i - \sum_{j=1}^{n} a_{ij} x_j \right]^2}$$

that is, we seek to minimize a sum of squares.

[Image of linear least squares regression]

The least squares problem can also have infinite solutions. In this case, the **minimum norm solution** is defined as the one for which

$$\|\mathbf{x}_{min}\|_2 \leq \|\mathbf{x}\|_2$$

where $\mathbf{x}$ is any solution. From now on, we will deal only with the case of overdetermined systems, i.e., with $m \geq n$, which is also the most significant from the point of view of applications. One of the most significant applications of the overdetermined least squares problem is related to data approximation, also known as **best fitting**.

Solvability of the Least Squares Problem Let us now see under which hypotheses the least squares problem admits a solution.

Teorema 2.1. *If A is an $m \times n$ matrix with $m > n$, then a solution to the LLS always exists. Furthermore, this solution is unique if and only if the rank of A is maximal, i.e., if rank$(A) = n$.*

We omit the proof of this theorem, but we highlight that such a proof is based on the following equivalence:

$$\mathbf{x} \text{ is a solution of the LLS} \iff \mathbf{x} \text{ is a solution of the system } {}^tAA\mathbf{x} = {}^tA\mathbf{b}.$$

The System of Normal Equations The system

$$^tAA\mathbf{x} = {}^tA\mathbf{b}$$

is called the **system of normal equations**. First, observe that the matrix ^tAA is a square matrix of order n. Furthermore, it is symmetric since

$$^t(^tAA) = {}^tA^t(^tA) = {}^tAA$$

that is, it coincides with its transpose. Finally, if A has full rank, it is positive definite because if $\mathbf{y}$ is any non-zero vector of length n, we have

$$^t\mathbf{y}(^tAA)\mathbf{y} = (^t\mathbf{y}^tA)(A\mathbf{y}) = {}^t(A\mathbf{y})(A\mathbf{y}) = \|A\mathbf{y}\|_2^2 > 0$$

(if the rank is not maximal, it may happen that $A\mathbf{y} = \mathbf{0}$, with $\mathbf{y} \neq \mathbf{0}$).

Geometric Interpretation We now want to give a geometric interpretation of the least squares problem. If A is of order $m \times n$, then it can be thought of as a linear map from $\mathbb{R}^n \to R(A) \subseteq \mathbb{R}^m$. The subspace $R(A)$ (the range or column space) is defined as follows:

$$R(A) = \{\mathbf{u} \in \mathbb{R}^m \mid \exists \mathbf{y} \in \mathbb{R}^n \,:\, \mathbf{u} = A\mathbf{y}\}.$$

Let $\mathbf{b} \in \mathbb{R}^m$. Since the 2-norm is the Euclidean norm, $\|\mathbf{b} - A\mathbf{x}\|_2$ represents the distance between the vectors $\mathbf{b}$ and $A\mathbf{x}$. It is evident then that for this distance to be minimal, it must happen that $\mathbf{b} - A\mathbf{x}$ is orthogonal to $R(A)$. Now, every vector in $R(A)$ can be seen as a linear combination of the columns of A. Indeed, if $\mathbf{u} \in R(A)$ is such that $\mathbf{u} = A\mathbf{y}$, this means that

$$\mathbf{u} = \begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix} = \begin{pmatrix} a_{11}y_1 + \cdots + a_{1n}y_n \\ \vdots \\ a_{m1}y_1 + \cdots + a_{mn}y_n \end{pmatrix}$$

that is

$$\mathbf{u} = y_1\mathbf{a}_1 + y_2\mathbf{a}_2 + \cdots + y_n\mathbf{a}_n$$

where $\mathbf{a}_j$ denotes the j-th column of A. Thus, for $\mathbf{b} - A\mathbf{x}$ to be orthogonal to $R(A)$, it suffices that it be orthogonal to all columns of A, i.e.,

$$^t\mathbf{a}_j(\mathbf{b} - A\mathbf{x}) = \mathbf{0}, \quad j \in \{1, \ldots, n\} \iff {}^tA(\mathbf{b} - A\mathbf{x}) = \mathbf{0} \iff {}^tAA\mathbf{x} = {}^tA\mathbf{b}$$

We thus arrive, also via a geometric route, at the equivalence between the solution of the least squares problem and that of the system of normal equations.

The Pseudoinverse

Definizione 2.1. Let A be an $m \times n$ matrix with $m \geq n$ and full rank ($\text{rank}(A) = n$). The Moore-Penrose **pseudoinverse** or **generalized inverse** of A is defined as the matrix

$$A^+ = (^tAA)^{-1}\,{}^tA.$$

The pseudoinverse generalizes the concept of the inverse of a matrix to the case of rectangular matrices. Indeed

$$A^+A = (^tAA)^{-1}\,{}^tAA = I.$$

On the other hand, if A is square and non-singular, we have

$$A^+ = (^tAA)^{-1}\,{}^tA = A^{-1}(^tA)^{-1}\,{}^tA = A^{-1},$$

that is, the inverse and pseudoinverse coincide. The link between the pseudoinverse and the least squares problem is provided by the following

Proposizione 2.2. *Let A be of order $m \times n$, with $m \geq n$ and full rank (rank$(A) = n$), and let $\mathbf{b}$ be a vector of length m. The unique solution to the least squares problem related to these data is given by*

$$\mathbf{x} = A^{+}\mathbf{b} \equiv ({}^{t}AA)^{-1}\,{}^{t}A\mathbf{b}.$$

The proposition is an immediate consequence of the existence and uniqueness theorem. In fact, we have already stated that the solution to the Linear Least Squares (LLS) problem is the solution to the system of normal equations

$${}^{t}AA\mathbf{x} = {}^{t}A\mathbf{b} \implies \mathbf{x} = ({}^{t}AA)^{-1}\,{}^{t}A\mathbf{b}.$$

The pseudoinverse matrix has several useful applications, not only in solving the least squares problem. Important information that can be deduced from the knowledge of A^{+} is the conditioning of the least squares problem. It is indeed possible to prove that if perturbations are introduced on $\mathbf{b}$ and A, then the corresponding perturbation on the vector $\mathbf{x}$, the solution to the LLS, is proportional to those introduced on the data via the quantity

$$pcond(A) = \|A\|\|A^{+}\|.$$

Solving the System of Normal Equations We have seen how the solution to the least squares problem coincides, in the overdetermined and full-rank case, with the solution to the system of normal equations:

$${}^{t}AA\mathbf{x} = {}^{t}A\mathbf{b}.$$

Therefore, to numerically calculate the solution of the LLS, it is possible to numerically solve the system of normal equations. Since the coefficient matrix is symmetric and positive definite, we can use the Gaussian elimination method without pivoting or, with halved computational cost, the Cholesky method. Thus, the least squares problem can be solved in the following way:

- form the matrix ${}^{t}AA$;
- factorize ${}^{t}AA$ using Cholesky factorization, i.e., compute L such that ${}^{t}AA = L^{t}L$;
- form the vector $\mathbf{c} = {}^{t}A\mathbf{b}$;
- solve the pair of triangular linear systems:

$$\begin{cases} L\mathbf{y} = \mathbf{c} \\ {}^{t}L\mathbf{x} = \mathbf{y} \end{cases}.$$

Computational Cost and Stability The overall computational cost of the procedure is approximately

$$m\frac{n^2}{2} + \frac{n^3}{6}$$

operations, as it takes about $m\frac{n^2}{2}$ to form ${}^{t}AA$ (constructed by symmetry) and $\frac{n^3}{6}$ for the Cholesky factorization. The backward and forward substitutions cost n^2 overall and are negligible. Therefore, the procedure is relatively inexpensive, as the most expensive operation (Cholesky factorization) is performed on the matrix of order n (which is generally much smaller than m). Unfortunately, the procedure just described is overall unstable. In fact, the first problem is the construction of the matrix ${}^{t}AA$. It may happen that there is a loss of significant digits in the machine representation of ${}^{t}AA$. In the most pathological cases, it may occur that the "machine" matrix is no longer positive definite or even turns out to be singular. It has been proven that if $pcond(A) > 10^{\frac{t}{2}}$, where t is the precision of the floating-point representation, it is no longer guaranteed that $fl({}^{t}AA)$ is non-singular.

Esempio 2.1. Consider the matrix

$$A = \begin{pmatrix} 1 & 1 \\ 10^{-4} & 0 \\ 0 & 10^{-4} \end{pmatrix}$$

and assume we are working with precision $t = 8$. The columns of A are linearly independent, and thus the rank is 2. Let us now compute ${}^tAA$.

$$ {}^tAA = \begin{pmatrix} 1+10^{-8} & 1 \\ 1 & 1+10^{-8} \end{pmatrix} $$

Since $t = 8$, then $fl({}^tAA) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, which is singular! Note that $pcond(A) = 1.4142... \cdot 10^4 > 10^4$.

Even if one does not encounter these "pathological" cases, the procedure is generally not very stable because, while the LLS has a conditioning regulated by $pcond(A)$, the system of normal equations has a conditioning that depends on $cond({}^tAA)$. It can be proven that (in 2-norm):

$$ cond({}^tAA) = pcond(A)^2. $$

In conclusion, we can say that solving via normal equations, although inexpensive, should be resorted to only when there is information regarding the good conditioning of the matrix ${}^tAA$. In cases where one is uncertain about the conditioning of the problem, it is better to use other methods, which are more expensive but stable.

Method via QR Factorization

The QR Factorization It is known that matrix factorizations constitute an effective tool in linear algebra. The most well-known are the LU factorization and the Cholesky L^tL factorization (for symmetric and positive definite matrices). Both of these factorizations are for square matrices. The computational cost for constructing L and U is approximately $\frac{m^3}{3}$ operations, while Cholesky factorization costs $\frac{m^3}{6}$ operations. Let us now illustrate another factorization called QR factorization, which can also be constructed for rectangular matrices. Given a matrix A of order $m \times n$, there exist two matrices, Q square of order m and orthogonal (i.e., such that $Q^{-1} = {}^tQ$), and R of order $m \times n$, of the type:

$$ R = \begin{pmatrix} R_1 \\ 0 \end{pmatrix} $$

with R_1 upper triangular of order n, if $m > n$, or of the type $(R_1 \quad 0)$ with R_1 upper triangular of order m, if $m < n$. In the case where A is square, the factors Q and R are both square and R is upper triangular. There are various algorithms for constructing the QR factorization, the most used of which (the one utilizing elementary Householder matrices) has a computational cost of $\frac{2}{3}m^3$ in the case of a full square matrix. In the rectangular case with $m > n$, however, the cost is of the order of:

$$ n^2 \left(m - \frac{n}{3} \right). $$

Comparing square matrices, the QR factorization "costs more" than the LU factorization. However, the added value lies in the fact that the factor Q is orthogonal. Orthogonal matrices are of significant importance due to their numerous properties:

- It is not necessary to construct inverses, which are available simply by transposition ($Q^{-1} \equiv {}^tQ$), which implies ${}^tQQ = Q^tQ = I$.
- If $\mathbf{y}$ is any vector of length m, we have:

  $$ \|Q\mathbf{y}\|_2 = \sqrt{{}^t(Q\mathbf{y})(Q\mathbf{y})} = \sqrt{{}^t\mathbf{y}{}^tQQ\mathbf{y}} = \sqrt{{}^t\mathbf{y}\mathbf{y}} = \|\mathbf{y}\|_2 $$

  that is, multiplication by orthogonal matrices leaves the vector norm invariant (the same applies to ${}^tQ$).
- From the above, we have $\|Q\|_2 = 1$, $\|{}^tQ\|_2 = 1$, from which $cond(Q) = 1$, meaning orthogonal matrices are perfectly conditioned.

The Method for LLS Based on QR Factorization Recall that the linear least squares problem consists of finding the minimum, over all vectors of length n, of the quantity $\|A\mathbf{x} - \mathbf{b}\|_2$ where A is a matrix of order $m \times n$, with $m \geq n$, $rank(A) = n$, and $\mathbf{b}$ has dimension m. An alternative numerical method is based on using the QR factorization of A. Indeed, let Q and R be the factors of A; we have:

$$^tQA = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$$

where R_1 is square, of order n, and upper triangular. Since ^tQ is orthogonal, and thus preserves the 2-norm of the vector, we have:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \|^tQ(A\mathbf{x} - \mathbf{b})\|_2^2 = \|^tQA\mathbf{x} - {}^tQ\mathbf{b}\|_2^2$$

Now setting $^tQ\mathbf{b} = \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix}$ with $\mathbf{c}$ being a vector of length n, we have:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \| \begin{pmatrix} R_1\mathbf{x} \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix} \|_2^2 = \| \begin{pmatrix} R_1\mathbf{x} - \mathbf{c} \\ -\mathbf{d} \end{pmatrix} \|_2^2$$

Let us now observe that if a vector $\mathbf{y}$ of length m is decomposed into two sub-vectors, for example $\mathbf{y} = {}^t(\mathbf{z}, \mathbf{t})$ with $\mathbf{z}$ of length n and $\mathbf{t}$ of length $m - n$, then $\|\mathbf{y}\|_2^2 = \|\mathbf{z}\|_2^2 + \|t\|_2^2$. Therefore:

$$\|A\mathbf{x} - \mathbf{b}\|_2^2 = \| \begin{pmatrix} R_1\mathbf{x} - \mathbf{c} \\ -\mathbf{d} \end{pmatrix} \|_2^2 = \|R_1\mathbf{x} - \mathbf{c}\|_2^2 + \|\mathbf{d}\|_2^2$$

Since the second term of the sum does not depend on $\mathbf{x}$ and is always positive, the minimum of the first member will be attained if $\|R_1\mathbf{x} - \mathbf{c}\|_2 = \mathbf{0}$, that is, if $R_1\mathbf{x} = \mathbf{c}$, or if $\mathbf{x}$ is the solution to the triangular system with R_1 as the coefficient matrix and $\mathbf{c}$ as the known term.

Procedure and Cost Thus, the proposed procedure can be summarized as follows:

- perform the QR factorization of the matrix A and thus, in particular, determine the matrix R_1;
- form the vector $\mathbf{c}$ consisting of the first n elements of the vector $^tQ\mathbf{b}$;
- solve the triangular system $R_1\mathbf{x} = \mathbf{c}$ (via backward substitution).

The quantity $\|\mathbf{d}\|_2$ represents the 2-norm of the residual. The algorithm has a computational cost which is essentially that of the QR factorization and is therefore $n^2(m - \frac{n}{3})$. The described procedure is furthermore stable.

Conclusions We have proposed two possible numerical strategies for solving the linear least squares problem, in the case where A has dimensions $m \times n$, with $m \geq n$ and $rank(A) = n$:

1. solving the system of normal equations $^tAA\mathbf{x} = {}^tA\mathbf{b}$;
2. the method utilizing the QR factorization of the matrix A.

Comparison of Schemes

Method 1 (Normal Equations):

- form the matrix ^tAA;
- calculate the Cholesky factorization $^tAA = L^tL$;
- solve the pair of triangular systems $\begin{cases} L\mathbf{y} = {}^tA\mathbf{b} \\ {}^tL\mathbf{x} = \mathbf{y} \end{cases}$;
- computational cost: $m\frac{n^2}{2} + \frac{n^3}{6}$;
- stability: guaranteed only if $cond(^tAA) = pcond(A)^2$ is not large.

Method 2 (QR Factorization):

- calculate the QR factorization of A;
- calculate ${}^tQ\mathbf{b} = \begin{pmatrix} \mathbf{c} \\ \mathbf{d} \end{pmatrix}$, where $\mathbf{c}$ has length n;
- setting $R = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$, solve the system $R_1\mathbf{x} = \mathbf{c}$;
- the norm of the residual is given by $\|\mathbf{d}\|_2$;
- computational cost: $n^2(m - \frac{n}{3})$;
- stability: always guaranteed.

MATLAB Functions The MATLAB functions involved in solving the LLS (Linear Least Squares) problem are the following:

- `A\b`: implements, in the case where A is rectangular, the method utilizing QR factorization, i.e., Method 2;
- `R = chol(B)`: calculates the Cholesky factor of the symmetric and positive definite matrix B ($B = {}^tRR$);
- `D = pinv(A)`: calculates the (left) pseudo-inverse of the matrix A;
- `[Q,R] = qr(A)`: calculates the QR factorization of the matrix A (square or rectangular) via Householder transformations.

2.1 Exercises

1. Consider the test matrix A defined by the MATLAB commands

$$B = gallery('kahan', 80);$$

$$A = B(:, 1:30).$$

   - Determine the dimensions of the matrix and establish whether it has full rank or not.
   - Consider the system $Ax = b$ with $b(i) = \sum_{j=1}^{n} a_{ij},\ i = 1, \ldots, m$ in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
   - Choose the most suitable method to apply and justify the choice.
   - It is known that in this case the solution is given by $\bar{x} = {}^t(1, 1, \ldots, 1) \in \mathbb{R}^n$. Calculate the error committed and compare the obtained result with the *a priori* estimate.

2. Consider the test matrix A defined by the MATLAB command

$$A = gallery('chebvand', 350, 60).$$

   - Determine the dimensions of the matrix and establish whether it has full rank or not.
   - Consider the system $Ax = b$ with $b(i) = \sum_{j=1}^{n} a_{ij},\ i = 1, \ldots, m$ in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
   - Choose the most suitable method to apply and justify the choice.
   - It is known that in this case the solution is given by $\bar{x} = {}^t(1, 1, \ldots, 1) \in \mathbb{R}^n$. Calculate the error committed and compare the obtained result with the *a priori* estimate.

3. Consider the test matrix A defined by the MATLAB commands

$$B = gallery('kms', 100, .1);$$

$$A = B(:, 1:30).$$

- Determine the dimensions of the matrix and establish whether it has full rank or not.
- Consider the system $Ax = b$ with $b(i) = \sum_{j=1}^{n} a_{ij},\ i = 1, \dots, m$ in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
- Choose the most suitable method to apply and justify the choice.
- It is known that in this case the solution is given by $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calculate the error committed and compare the obtained result with the *a priori* estimate.

4. Consider the test matrix A defined by the MATLAB commands

$$A = gallery('lauchli', 100) * gallery('lehmer', 100);$$

- Determine the dimensions of the matrix and establish whether it has full rank or not.
- Consider the system $Ax = b$ with $b(i) = \sum_{j=1}^{n} a_{ij},\ i = 1, \dots, m$ in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
- Choose the most suitable method to apply and justify the choice.
- Calculate the error committed by comparing the solution obtained with the one obtained using the MATLAB 'function. Compare this error with the *a priori* estimate.

5. Given the matrix of order $n = 100$

$$B = \begin{pmatrix} 1 & -2 & -2 & \dots & \dots & -1 \\ 4 & 2 & -2 & \dots & \dots & -2 \\ 4 & 4 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -2 & -2 \\ \vdots & & \ddots & \ddots & n-1 & -2 \\ 5 & 4 & \dots & 4 & 4 & n \end{pmatrix},$$

consider the matrix $A = B(:, 1:7)$.

- Determine the dimensions of the matrix and establish whether it has full rank or not.
- Consider the system $Ax = b$ with $b = 2{*}sum(A, 2)$, in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
- Choose the most suitable method to apply and justify the choice.
- Comment on the obtained results.

6. Consider the following test matrix

$$A = gallery('lauchli', 10).$$

- Determine the dimensions of the matrix and establish whether it has full rank or not.
- Consider the system $Ax = b$ with $b(i) = \sum_{j=1}^{n} a_{ij},\ i = 1, \dots, m$ in the least squares sense. Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
- Choose the most suitable method to apply and justify the choice. What is its computational cost?
- It is known that in this case the solution is given by $\bar{x} = {}^t(1, 1, \dots, 1) \in \mathbb{R}^n$. Calculate the error committed and compare the obtained result with the *a priori* estimate.

7. Let $m = 90$. Consider the system $Ax = b$ in the least squares sense, where

$$A = (a_{i,j})_{\substack{i=1,\dots,m \\ j=1,\dots,\frac{m}{3}}}, \quad a_{i,j} = \begin{cases} \sin(i + 2j) + 1, & i = j \\ \cos(2i + j - 5), & i \neq j \end{cases}$$

$$\mathbf{b} = \{b_i\}_{i=1,\dots,m}, \quad b_i = \sum_{j=1}^{\frac{m}{3}} a_{i,j}, \quad i = 1, \dots, m.$$

- Report the MATLAB instructions used to construct A and b.

- Determine the dimensions of the matrix and establish whether it has full rank or not.
- Study the existence and uniqueness of the system's solution, the conditioning of the problem, and determine the theoretical *a priori* error estimate.
- Choose the most suitable method to apply and justify the choice. What is its computational cost?
- It is known that in this case the solution is given by $\bar{x} = {}^t(1, 1, \ldots, 1) \in \mathbb{R}^{\frac{m}{3}}$. Calculate the error committed and compare the obtained result with the *a priori* estimate.

3 Numerical Methods for Solving Non-Linear Equations

3.1 Solving Non-Linear Equations

Introduction Let $F \in C^0([a,b])$, i.e., F is a continuous function on an interval $[a,b] \subset \mathbb{R}$, such that $F(a)F(b) < 0$. We want to find the roots of the non-linear equation

$$F(x) = 0$$

The condition $F(a)F(b) < 0$ is a sufficient condition for the existence of at least one root of F in $[a,b]$.

Bisection Method This method consists of constructing, starting from the interval $[a,b]$, a sequence of nested intervals

$$[a,b] = [a_0,b_0] \supset [a_1,b_1] \supset \cdots \supset [a_n,b_n]$$

all containing the root $\overline{x}$ of F, such that

$$b_n - a_n \to 0 \quad \text{for } n \to +\infty.$$

The intervals $[a_k,b_k]$, $k \in \{1,\dots,n\}$, of the sequence are determined as follows: given $[a_{k-1},b_{k-1}]$, we determine the midpoint

$$m_k = \frac{a_{k-1}+b_{k-1}}{2}$$

- if $F(m_k) = 0$ then $\overline{x} = m_k$ and we are finished;
- if $F(m_k) \neq 0$ we set

$$[a_k,b_k] = \begin{cases} [a_{k-1},m_k] & \text{if } F(a_{k-1})F(m_k) < 0 \\ [m_k,b_{k-1}] & \text{if } F(b_{k-1})F(m_k) < 0 \end{cases}$$

After n steps, we arrive at the interval $[a_n,b_n]$ containing the root $\overline{x}$ and having width

$$b_n - a_n = \frac{b_{n-1}-a_{n-1}}{2} = \frac{b_{n-2}-a_{n-2}}{2^2} = \cdots = \frac{b-a}{2^n}$$

We choose as an estimate for $\overline{x}$

$$x_{n+1} = m_{n+1} = \frac{a_n+b_n}{2}$$

so that

$$\overline{x} = x_{n+1} \pm \varepsilon_{n+1}$$

where

$$\varepsilon_{n+1} = |\overline{x} - x_{n+1}| < \frac{b-a}{2^{n+1}}$$

is the absolute error of the approximation of the root $\overline{x}$ of F. Given a tolerance TOLL, it is possible to determine the number of iterations k necessary to approximate the root $\overline{x}$ with precision TOLL by imposing that

$$|\overline{x} - x_n| < \frac{b-a}{2^n} < \text{TOLL}$$

In fact, we can write

$$\frac{b-a}{\text{TOLL}} < 2^n$$

$$\log\left(\frac{b-a}{\text{TOLL}}\right) < \log(2^n) = n\log 2$$

$$n > \frac{\log\left(\frac{b-a}{\text{TOLL}}\right)}{\log 2}$$

Therefore, we take

$$n = \left\lfloor \frac{\log\left(\frac{b-a}{\text{TOLL}}\right)}{\log 2} \right\rfloor + 1$$

where $\lfloor a \rfloor$ denotes the floor (integer part) of a.

```
function [x, i, itmax] = bisec(a, b, f, toll)
% BISEC Finds the root of f(x)=0 in [a,b] using the Bisection method.
%
%   Input:
%     a, b  - Interval endpoints
%     f     - Function handle of the function
%     toll  - Required tolerance (default 1e-6)
%
%   Output:
%     x     - Vector with the history of approximations
%     i     - Number of iterations performed
%     itmax - Maximum theoretical number of iterations expected

    %% 1. Input Validation
    arguments
        a     (1,1) double
        b     (1,1) double
        f     (1,1) function_handle
        toll  (1,1) double {mustBePositive} = 1e-6
    end

    %% 2. Zero Existence Check (Intermediate Value Theorem)
    fa = f(a);
    fb = f(b);

    % Use sign() to avoid overflow if f(a)*f(b) becomes too large
    if sign(fa) * sign(fb) > 0
        error('The function does not change sign at the endpoints of the interval [a,b].');
    end

    %% 3. Initialization
    % Theoretical calculation of the necessary steps
    itmax = 1 + floor(log((b-a)/toll)/log(2));

    i = 1;               % Current index
    x(i) = (a + b) / 2; % First midpoint
    fx = f(x(i));        % Evaluation at midpoint

    %% 4. Iterative Loop
    % Continue as long as: steps not finished AND error is high
    while (i < itmax) && (abs(fx) >= toll)

        % Sub-interval selection
        if sign(fx) * sign(fa) < 0
            b = x(i);   % The root is on the left (between a and x)
        else
            a = x(i);   % The root is on the right (between x and b)
            fa = fx;    % Crucial: update fa for the next comparison
        end

        % New step
        i = i + 1;
        x(i) = (a + b) / 2; % New midpoint
        fx = f(x(i));       % New evaluation
    end

end
```

Function bisec

Order of Convergence We now introduce a measure of the rate of convergence of a numerical sequence.

Definizione 3.1. Let $\{x_n\}_n$ be a sequence converging to the value $\overline{x}$. Let $\varepsilon_n = |\overline{x} - x_n|$. If there exist a real number $p \geq 1$ and a positive real constant C such that

$$\lim_{n \to +\infty} \frac{\varepsilon_{n+1}}{\varepsilon_n^p} = C$$

then we say that the sequence $\{x_n\}_n$ has an order of convergence p.

Since for the Bisection method we have

$$\frac{\varepsilon_{n+1}}{\varepsilon_n} \simeq \frac{1}{2}$$

meaning the error roughly halves at each step, its order of convergence is 1; that is, it converges very slowly. However, it must be observed that the method only requires the continuity of the function F and knowledge of the sign of F at the endpoints of the interval $[a, b]$.

Nota 6. For every $n \geq 0$, assume that the approximation x_n of $\overline{x}$ has $t_n \in \mathbb{R}$ correct decimal digits, i.e.,

$$\varepsilon_n = \frac{1}{2}10^{-t_n}$$

Then, since the error halves at each step,

$$\frac{1}{2}10^{-t_{n+1}} = \varepsilon_{n+1} = \frac{1}{2}\varepsilon_n = \frac{1}{4}10^{-t_n}$$

or

$$10^{-t_{n+1}} = \frac{1}{2}10^{-t_n}$$

$$t_{n+1} = t_n + \log_{10} 2 \approx t_n + 0.3010, \quad n \geq 0$$

Thus t_n increases by one unit after at least 4 iterations. 3 iterations are not sufficient to obtain an approximation of $\overline{x}$ with one additional correct decimal digit.

Iterative Methods Starting from the initial point x_0, we generate the values $x_1, x_2, \ldots, x_n$ in the following way:

- We draw a line with slope m_0 from the point $(x_0, F(x_0))$ and choose the intersection of this line with the x-axis as the approximation x_1;
- We draw a line with slope m_1 from the point $(x_1, F(x_1))$ and choose the intersection of this line with the x-axis as the approximation x_2;

Thus, at each step, we choose as the approximation of the root $\overline{x}$ of F, the root of the linear equation

$$F(x_n) + m_n(x - x_n) = 0$$

that is

$$x_{n+1} = x_n - \frac{F(x_n)}{m_n} \quad (1)$$

A formula of type (1) is called an **iterative formula**, and the sequence $x_1, x_2, \ldots, x_n, \ldots$ is said to be constructed via an iterative process. The function

$$g(x) = x - \frac{F(x)}{m_n}$$

such that $x_{n+1} = g(x_n)$ is called a **simple iteration function** because it uses only the point x_n to construct the sequence $\{x_n\}_n$. We speak of multiple iterative processes when

$$x_{n+k} = g(x_{n+k-1}, x_{n+k-2}, \ldots, x_n), \quad k > 1$$

meaning the sequence $\{x_n\}_n$ is constructed starting from the points $x_0, x_1, \ldots, x_{k-1}$.

Stopping Criteria When automatically implementing an iterative process of the type $x_{n+1} = g(x_n)$, $n \in \mathbb{N}$, one or more criteria are needed to stop the process. Given an arbitrarily small tolerance TOLL, the stopping criteria usually used are:

1. $|F(x_n)| <$ TOLL (the smaller TOLL is, the closer x_n is to $\overline{x}$);
2. $|x_{n+1} - x_n| <$ TOLL or $\frac{|x_{n+1}-x_n|}{|x_{n+1}|} <$ TOLL (the smaller TOLL is, the closer x_n is to the limit of the sequence $\overline{x}$);
3. number of iterations $<$ ITMAX, where ITMAX is a fixed integer variable.

Criterion 3 comes into play when the first two fail. For this reason, the ITMAX variable is also called a guard variable. In general, it is preferable to use all three criteria, because for particular functions, convergence may be too slow, or criteria 1 and/or 2 may never be numerically satisfied.

Newton's Method or Method of Tangents In the formula

$$x_{n+1} = x_n - \frac{F(x_n)}{m_n}$$

the directions m_n can be chosen in various ways. In this method, the direction chosen at each step is that of the tangent to the curve $y = F(x)$ at the point x_n, i.e.,

$$m_n = F'(x_n).$$

Thus the method is given by

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}, \quad n \in \mathbb{N}.$$

The iteration function is therefore

$$g(x) = x - \frac{F(x)}{F'(x)}.$$

It is evident that the method becomes undefined if $F'(x_n) = 0$ for some n. Therefore, generally, it is assumed that $F'(x) \neq 0$, $\forall x \in [a, b]$. The iterative formula allows us to calculate the sequence $x_1, x_2, \ldots$ starting from the point x_0. But how do we choose the point x_0?

```
function [x, i] = newton(x0, f, df, toll, itmax)
% NEWTON Finds the root of f(x)=0 using Newton's method.
%   [x, i] = NEWTON(x0, f, df, toll, itmax) calculates the approximation
%   of the root starting from an initial estimate x0.
%
%   Input:
%     x0    - (double) Initial estimate
%     f     - (function_handle) The function f(x)
%     df    - (function_handle) The first derivative f'(x)
%     toll  - (double, optional) Tolerance for stopping (default 1e-6)
%     itmax - (integer, optional) Maximum number of iterations (default 100)
%
%   Output:
%     x - (vector) Vector containing all calculated iterations
%     i - (integer) Number of iterations performed

    %% 1. Input Validation
    arguments
        x0    (1,1) double
        f     (1,1) function_handle
        df    (1,1) function_handle
        toll  (1,1) double {mustBePositive} = 1e-6
        itmax (1,1) double {mustBePositive, mustBeInteger} = 100
    end

    %% 2. Initialization
    i = 1;                % Position index (starts at 1)
    x(i) = x0;            % Insert initial estimate into vector

    fx = f(x(i));         % Function evaluation
    dfx = df(x(i));       % Derivative evaluation

    err = 1;              % Initialize error to enter the loop

    %% 3. Iterative Loop
    % Continue if: itmax not exceeded AND error is high AND residual is high
    while (i < itmax) && (err > toll) && (abs(fx) > toll)

        % Safety check: Derivative zero or nearly zero
        if abs(dfx) < eps
            warning('Derivative nearly zero at x = %f. Method stopped.', x(i));
            break;
        end

        % Newton Step
        % Formula: x(k+1) = x(k) - f(x)/f'(x)
        x(i + 1) = x(i) - fx / dfx; % Calculate and add new value to vector

        % Relative Error Calculation
        % Note: Add eps to denominator to avoid crash if x(i + 1) is 0
        err = abs(x(i + 1) - x(i)) / (abs(x(i + 1)) + eps);

```

```
        % Update indices and functions
        i = i + 1;
        fx = f(x(i));
        dfx = df(x(i));
    end

    %% 4. Final check
    % If we exited because i reached itmax (and error is still high)
    if (i == itmax) && (err > toll)
        warning('Reached maximum number of iterations (%d) without convergence.', itmax);
    end
end
```

Function newton

Convergence Theorem The following theorem establishes conditions for determining an interval $[a, b]$ for which Newton's method converges for every choice of $x_0 \in [a, b]$.

Teorema 3.1. *Let $F(x) \in C^2([a, b])$ with $[a, b]$ closed and bounded. If:*

- $F(a)F(b) < 0$;
- $F'(x) \neq 0$, $\forall x \in [a, b]$;
- $F''(x) \geq 0 \ \vee \ F''(x) \leq 0$, $\forall x \in [a, b]$;
- $|\frac{F(a)}{F'(a)}| < (b - a) \ \wedge \ |\frac{F(b)}{F'(b)}| < (b - a)$;

then Newton's Method converges to the unique solution $\overline{x} \in [a, b]$ for every choice of $x_0 \in [a, b]$.

Proof. From conditions 1-3, the uniqueness of the solution of the equation $F(x) = 0$ in the interval $[a, b]$ follows. Let us first show that condition 4 implies that the tangent to the curve of equation $y = F(x)$ at the interval endpoints intersects the x-axis inside $[a, b]$. Consequently, if we take a point belonging to the interval $[a, b]$ (a and b included) as the initial point x_0, all points of the sequence $\{x_k\}_{k\in\mathbb{N}}$ belong to the interval $[a, b]$. The tangent line at $(a, F(a))$ has the equation

$$y = F'(a)(x - a) + F(a)$$

and thus intersects the x-axis at

$$x = a - \frac{F(a)}{F'(a)}$$

Since $\frac{F(a)}{F'(a)}$ under conditions 1-3 is always negative and, using hypothesis 4, it results that

$$|x - a| = \left|\frac{F(a)}{F'(a)}\right| < b - a$$

we obtain $a < x < b$. Similar reasoning applies to the other endpoint. Let us now show that the sequence $\{x_k\}_{k=1,2,\ldots}$ constructed starting from $x_0 \in [a, b]$ is monotonic and bounded. We conduct the proof in one of the four possible situations that can occur under hypotheses 1-3 (the proof in the other cases is similar). Let us assume then that $F' > 0$, $F \leq 0$, $F(a) < 0$, $F(b) > 0$. We therefore prove that $x_k \leq \overline{x}$ and $x_{k+1} \geq x_k$. We start with $k = 0$ and then proceed by induction. Without loss of generality, let us take x_0 in the interval $[a, \overline{x}]$ and thus such that $F(x_0) \leq 0$. We then have

$$x_1 = x_0 - \frac{F(x_0)}{F'(x_0)} \geq x_0$$

Thus $x_0 \leq \overline{x}$ and $x_1 \geq x_0$. Since the property is true for $k = 0$, we demonstrate that, assuming it is true for k, it holds for $k + 1$. We have

$$x_{k+1} = x_k - \frac{F(x_k)}{F'(x_k)}$$

and, by the Mean Value Theorem (Lagrange's Theorem),

$$-F(x_k) = F(\overline{x}) - F(x_k) = F'(\xi_k)(\overline{x} - x_k), \quad x_k \leq \xi_k \leq \overline{x}$$

Since by hypothesis $F(x) \leq 0$, it follows that F' is decreasing and therefore $F'(\xi_k) \leq F'(x_k)$, from which

$$-F(x_k) \leq F'(x_k)(\overline{x} - x_k)$$

Consequently

$$x_{k+1} = x_k - \frac{F(x_k)}{F'(x_k)} \leq x_k + (\overline{x} - x_k) = \overline{x}$$

$$F(x_{k+1}) \leq 0 \implies x_{k+2} = x_{k+1} - \frac{F(x_{k+1})}{F'(x_{k+1})} \geq x_{k+1}$$

To complete the proof, it suffices to observe that the sequence $\{x_k\}_{k=1,2,\ldots}$, being monotonic and bounded, is convergent. Furthermore, passing to the limit in the iterative formula, we immediately deduce that it converges to $\overline{x}$. In fact, letting $\beta = \lim_{k\to\infty} x_k$, we have

$$\beta = \lim_{k\to\infty} x_{k+1} = \lim_{k\to\infty} x_k - \lim_{k\to\infty} \frac{F(x_k)}{F'(x_k)} = \beta - \frac{F(\beta)}{F'(\beta)}$$

from which $F(\beta) = 0$ and therefore $\beta = \overline{x}$ since, given our hypotheses, $\overline{x}$ is the unique zero of F in the interval $[a, b]$. □

Order of Convergence of Newton's Method To determine the order of convergence, we use the following theorem:

Teorema 3.2. *A necessary and sufficient condition for a simple iterative process converging to $\overline{x}$ to have an order of convergence p is that, if the iteration function g possesses a continuous p-th derivative for $x = \overline{x}$, it results that*

$$g'(\overline{x}) = g''(\overline{x}) = \cdots = g^{(p-1)}(\overline{x}) = 0 \quad \textit{and} \quad g^{(p)}(\overline{x}) \neq 0$$

For Newton's Method, we have:

$$g'(x) = 1 - \frac{(F'(x))^2 - F(x)F''(x)}{(F'(x))^2} = \frac{F(x)F''(x)}{(F'(x))^2}$$

but, since we assume that $F(\overline{x}) = 0$, we have

$$g'(\overline{x}) = 0$$

and therefore

$$g(x) = \frac{(F'(x)F''(x) + F(x)F'''(x))(F'(x))^2 - 2F'(x)F(x)(F''(x))^2}{(F'(x))^4}$$

$$g''(\overline{x}) = \frac{F''(\overline{x})}{F'(\overline{x})}$$

which, in general, is different from 0. Thus, in general, the method has an order of convergence of 2.

Question: What happens if $\overline{x}$ is a double zero, i.e., $F(\overline{x}) = F'(\overline{x}) = 0$ and $F''(\overline{x}) \neq 0$? In this case, it is not possible to calculate either g or g' at $\overline{x}$, but it is possible to define them by continuity:

$$g(\overline{x}) = \overline{x} - \lim_{x\to\overline{x}} \frac{F(x)}{F'(x)} = \overline{x} - \lim_{x\to\overline{x}} \frac{F'(x)}{F''(x)} = \overline{x}$$

$$g'(\overline{x}) = F''(\overline{x}) \lim_{x\to\overline{x}} \frac{F(x)}{(F'(x))^2} = F''(\overline{x}) \lim_{x\to\overline{x}} \frac{F'(x)}{2F'(x)F''(x)} = \frac{1}{2}$$

Thus, if $\overline{x}$ is a double zero, since $g'(\overline{x}) \neq 0$, Newton's method has an order of convergence of 1. More generally, if $\overline{x}$ is a zero of order r, i.e.,

$$F(\overline{x}) = F'(\overline{x}) = \cdots = F^{(r-1)}(\overline{x}) = 0 \quad \text{and} \quad F^{(r)}(\overline{x}) \neq 0$$

then

$$g'(\overline{x}) = 1 - \frac{1}{r} \neq 0$$

therefore, Newton's method still has an order of convergence of 1.

Newton's Method for Multiple Roots When $\overline{x}$ is a multiple zero, it is possible to slightly modify Newton's method to "recover" order 2. Consider the following iterative formula:

$$x_{n+1} = x_n - 2\frac{F(x_n)}{F'(x_n)} \quad n \in \mathbb{N}$$

whose iteration function is

$$g(x) = x - 2\frac{F(x)}{F'(x)}$$

The previous iterative formula has an order of convergence of 2; indeed:

$$g'(x) = 1 - 2\frac{(F'(x))^2 - F(x)F''(x)}{(F'(x))^2} = 2\frac{F(x)F''(x)}{(F'(x))^2} - 1$$

we obtain

$$g'(\overline{x}) = 2F''(\overline{x}) \lim_{x\to\overline{x}} \frac{F(x)}{(F'(x))^2} - 1 = 1 - 1 = 0$$

In general, if $\overline{x}$ is a multiple root of order r, the following iterative formula is used

$$x_{n+1} = x_n - r\frac{F(x_n)}{F'(x_n)}, \quad n \in \mathbb{N}$$

whose order of convergence is still 2. Appropriate modifications also exist for cases where the root multiplicity is > 1 but unknown, and cases where the multiplicity is infinite. There are variants of Newton's Method that have an order of convergence > 2.

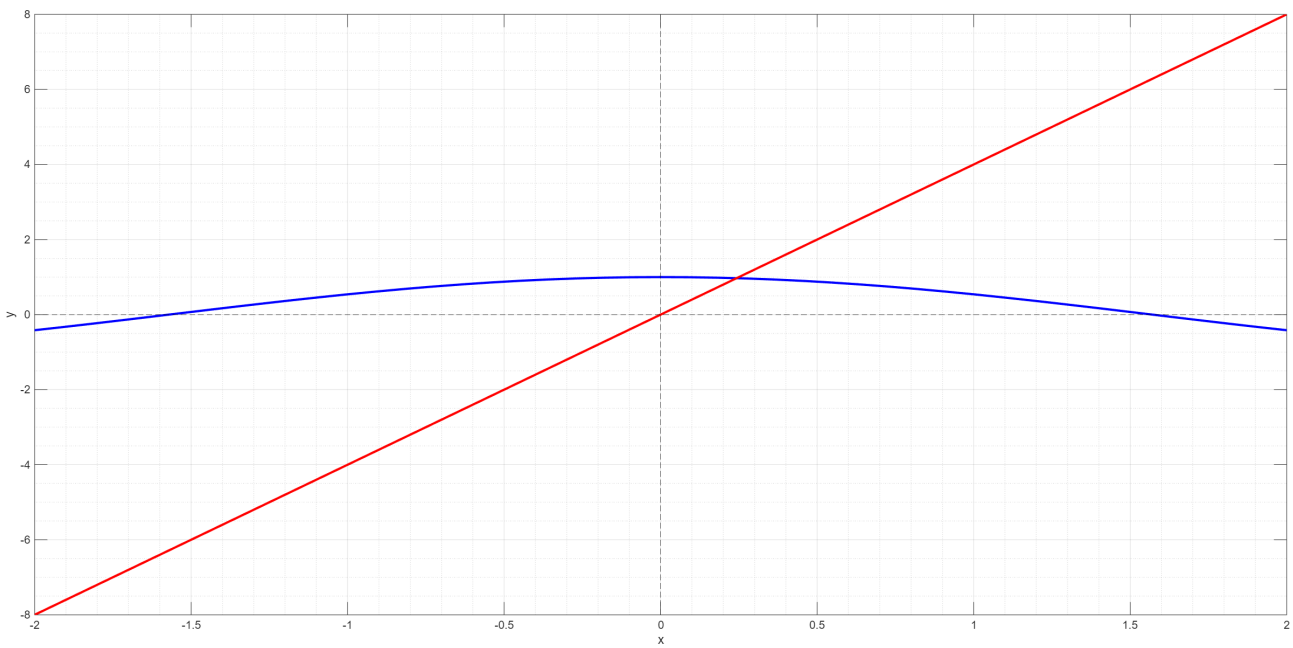
Nota 7. When the function F has multiple roots, we have seen that from a theoretical point of view, Newton's Method converges. However, it often happens that it does not converge numerically. The same thing sometimes happens when F has "quasi-multiple" roots.

Esempio 3.1. Given the equation

$$\cos x - 4x = 0$$

determine an interval on the real axis that contains the unique zero of this equation and is such that Newton's Method is convergent.

Solution: We graphically localize the zero of the equation by finding the intersection point of the curves $y = \cos x$ and $y = 4x$.



It can be seen that the zero $\overline{x}$ falls in the interval $\left[0, \frac{1}{2}\right]$. Let's see if this interval is the right candidate for the convergence of Newton's Method. First, we verify the conditions at the endpoints. We have

$$F(0)F(1/2) = 1 \cdot (-1.12241) < 0$$

Setting $F(x) = \cos x - 4x$, we have

$$F'(x) = -\sin x - 4 \implies F'(x) < 0 \quad \forall x \in \mathbb{R}$$

$$F''(x) = -\cos x \implies F''(x) \leq 0 \text{ if } \cos x \geq 0$$

Thus

$$F(x) \leq 0 \quad \forall x \in \left[0, \frac{1}{2}\right].$$

Furthermore

$$\left|\frac{F(0)}{F'(0)}\right| = \left|\frac{1}{-4}\right| = \frac{1}{4} < \frac{1}{2}$$

$$\left|\frac{F(1/2)}{F'(1/2)}\right| = \left|\frac{-1.12241}{-4.47942}\right| = 0.250571\cdots < \frac{1}{2}$$

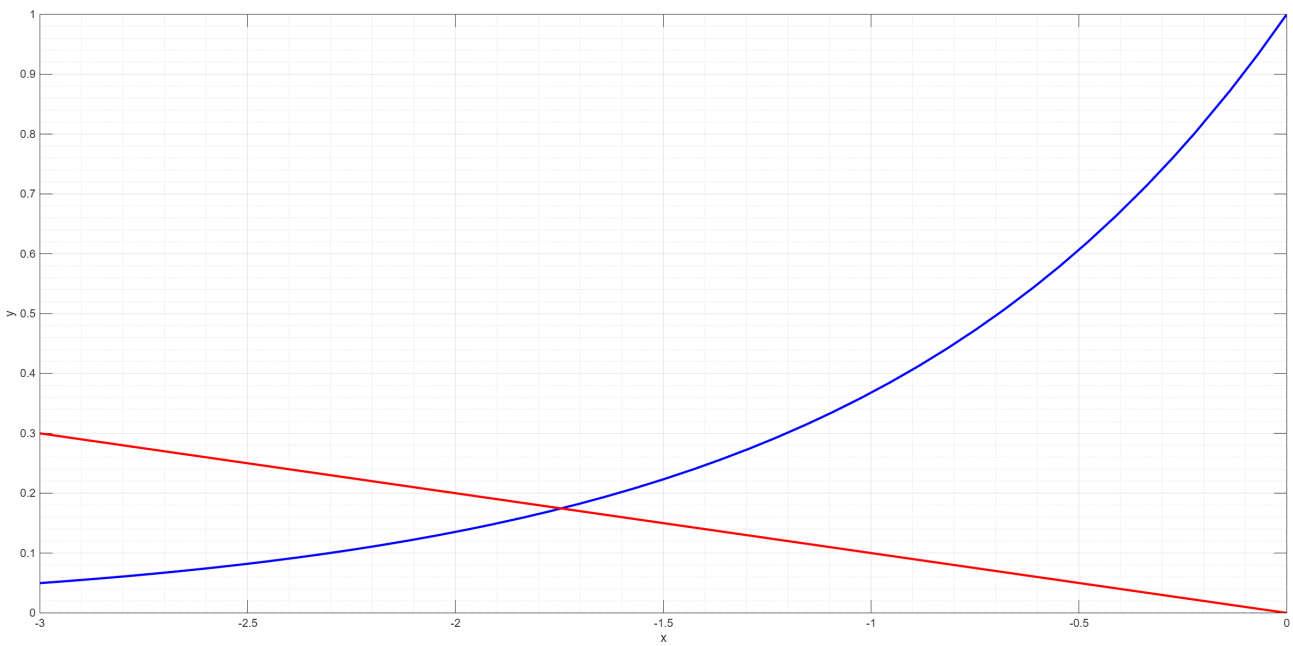
Thus Newton's Method converges $\forall x_0 \in \left[0, \frac{1}{2}\right]$.

Esempio 3.2. Given the equation

$$e^x + \frac{x}{10} = 0$$

determine an interval on the real axis that contains the unique zero of this equation and is such that Newton's Method is convergent.

Solution: We graphically localize the zero of the equation by finding the intersection point of the curves $y = e^x$ and $y = -x/10$. It can be seen that the zero $\overline{x}$ falls in the interval $\left[-2, -\frac{3}{2}\right]$.



Let's see if this interval is the right candidate for the convergence of Newton's Method. First, we verify the conditions at the endpoints. We have

$$F(-2)F(-3/2) = (-0.64665e-003) \cdot (0.73130e-003) < 0$$

Setting $F(x) = e^x + x/10$, we have

$$F'(x) = e^x + 1/10 \implies F'(x) > 0 \; \forall x \in \mathbb{R}$$

$$F''(x) = e^x \implies F''(x) > 0 \; \forall x \in \mathbb{R}$$

Furthermore

$$\left|\frac{F(-2)}{F'(-2)}\right| = \left|\frac{-0.64665e-003}{0.23534}\right| = 0.27478 < \frac{1}{2}$$

$$\left|\frac{F(-3/2)}{F'(-3/2)}\right| = \left|\frac{0.73130e-003}{0.32313e-002}\right| = 0.22632e-001\cdots < \frac{1}{2}$$

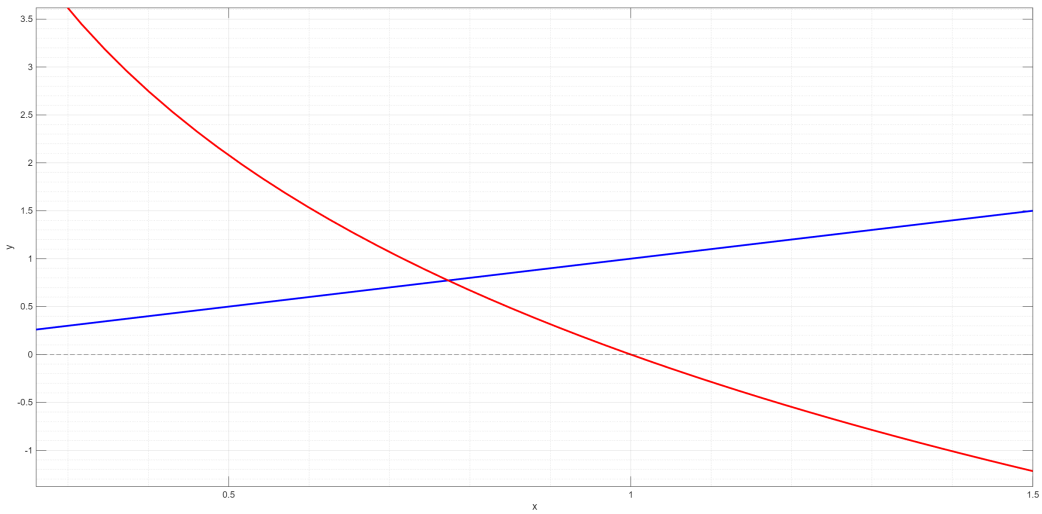
Thus Newton's Method converges $\forall x_0 \in \left[-2, -\frac{3}{2}\right]$.

Esempio 3.3. Given the equation

$$x + \log x^3 = 0$$

determine an interval on the real axis that contains the unique zero of this equation and is such that Newton's Method is convergent.

Solution: We graphically localize the zero of the equation by finding the intersection point of the curves $y = \log x$ and $y = -x/3$.



It can be seen that the zero $\overline{x}$ falls in the interval $\left[\frac{1}{2}, 1\right]$. Let's see if this interval is the right candidate for the convergence of Newton's Method. First, we verify the conditions at the endpoints. We have

$$F(1/2)F(1) = (-1.57944) \cdot 1 < 0$$

Setting $F(x) = x + 3\log x$, we have

$$F'(x) = 1 + \frac{3}{x} \implies F'(x) > 0 \; \forall x > 0$$

$$F''(x) = -\frac{3}{x^2} \implies F''(x) < 0 \; \forall x \in \left[\frac{1}{2}, 1\right]$$

Furthermore

$$\left|\frac{F(1/2)}{F'(1/2)}\right| = \left|\frac{-1.57944}{7}\right| = 0.22563 < \frac{1}{2}$$

$$\left|\frac{F(1)}{F'(1)}\right| = \left|\frac{1}{4}\right| < \frac{1}{2}$$

Thus Newton's Method converges $\forall x_0 \in \left[\frac{1}{2}, 1\right]$.

Nota 8. The convergence of Newton's Method is guaranteed when, assuming the theorem's hypotheses are met, the initial approximation x_0 is "sufficiently" close to the root $\overline{x}$. Therefore, this method often proves efficient especially for improving a sufficiently good approximation obtained with a lower-order method whose convergence is assured. In general, instead of verifying the theorem's hypotheses, it is preferable to use the Bisection Method to determine an approximation of the root with 1 or 2 correct decimal digits, and then apply Newton's Method to obtain the desired precision with very few iterations.

```
function [x_bis, i_bis, x_newt, i_newt] = bisnew(a, b, f, df, toll, itmax)
% BISNEW Hybrid method: Roughing with Bisection + Refinement with Newton.
%
%  Input:
%    a, b  - Initial interval endpoints
%    f, df - Function and its Derivative (handles)
%    toll  - Required tolerance for the final phase (Newton)
%    itmax - Max iterations for the final phase (Newton)
```

```
%
%  Output:
%    x_bis  - (vector)  History of approximations from the Bisection phase
%    i_bis  - (integer) Number of iterations from the Bisection phase
%    x_newt - (vector)  History of approximations from the Newton phase
%    i_newt - (integer) Number of iterations from the Newton phase

    %% 1. Input Validation
    arguments
        a     (1,1) double
        b     (1,1) double
        f     (1,1) function_handle
        df    (1,1) function_handle
        toll  (1,1) double {mustBePositive} = 1e-6
        itmax (1,1) double {mustBePositive, mustBeInteger} = 100
    end

    %% 2. Phase 1: Roughing (Bisection)
    % We use a fixed tolerance of 0.5e-1 to approach safely.
    toll_coarse = 0.5e-1;
    [x_bis, i_bis] = bisec(a, b, f, toll_coarse);

    %% 3. Phase 2: Refinement (Newton)
    % The trigger point for Newton is the last result from Bisection.
    start_point = x_bis(end);
    [x_newt, i_newt] = newton(start_point, f, df, toll, itmax);

end
```

Function bisnew

3.2 Solution of Algebraic Equations

Evaluation of a polynomial at a point Algebraic equations are equations of the type

$$P(x) = 0$$

where P is a polynomial of degree n, i.e.,

$$P(x) = a_1 x^n + a_2 x^{n-1} + \cdots + a_n x + a_{n+1}, \quad a_1 \neq 0, \quad a_k \in \mathbb{R}$$

The following properties are well known:

- A polynomial of degree n has exactly n roots, which can be real or complex, each counted with its multiplicity (Fundamental Theorem of Algebra);
- If a polynomial P has a complex root $\alpha = a + ib$, then its conjugate $\overline{\alpha} = a - ib$ is also a root of P;
- A polynomial of odd degree has at least one real root;
- For $n \geq 5$, there is no explicit form for the roots of P.

To determine an approximation of the real roots of an algebraic equation, it is possible to use the methods seen for non-linear equations, after identifying an interval containing it for each real root. When n is remarkably large, the problem of identifying such intervals may not be immediate. In general, preliminary operations must be performed:

- *root localization*: serves to determine a circle in the complex plane that contains all roots (both real and complex);
- *root counting*: consists of determining the number of real roots.

The following results are useful for this purpose.

Cauchy's Theorem

Teorema 3.3. *All zeros of*

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

are included in the circle of the plane centered at the origin with radius

$$r = 1 + \max_{2\le k\le n+1} \left|\frac{a_k}{a_1}\right|$$

Nota 9. Thus, in particular, the real roots are found in the interval $[-r, r]$.

Descartes' Rule of Signs Let

$$P(x) = a_1x^n + a_2x^{n-1} + \cdots + a_nx + a_{n+1}$$

Consider the ordered set of coefficients

$$A = \{a_1, a_2, \ldots, a_n, a_{n+1}\}$$

If we denote by ν the number of sign variations in the set A (any zeros are not counted), then the number k of positive real roots of P satisfies the following properties:

$$k \le \nu \quad \text{and} \quad \nu - k \text{ is an even number.}$$

If we apply Descartes' rule to the polynomial

$$Q(x) = P(-x)$$

we obtain the number of positive real roots of Q and therefore the number of negative real roots of P. By applying the previous results and plotting the graph of P on the interval $[-r, r]$, one can establish with certainty the number of real roots of P and determine as many intervals as there are roots of P. One can then proceed to the approximate calculation of the real roots of P using the methods seen for non-linear equations.

Nota 10. It is also possible to use Newton's method to approximate the complex roots of a polynomial by appropriately choosing the point x_0. There are variants of Newton's Method that allow approximating all roots of the polynomial simultaneously, both real and complex ones.

Esempio 3.4. Let

$$P(x) = x^6 - x - 1$$

Since

$$\max_{2\le k\le n+1} \left|\frac{a_k}{a_1}\right| = 1$$

applying Cauchy's Theorem, all real roots of P belong to the interval $[-2, 2]$. Since

$$A = \{1, 0, 0, 0, 0, -1, -1\}$$

applying Descartes' rule gives $\nu = 1$. Therefore k must necessarily be 1, i.e., P has only one positive root. Finally, setting

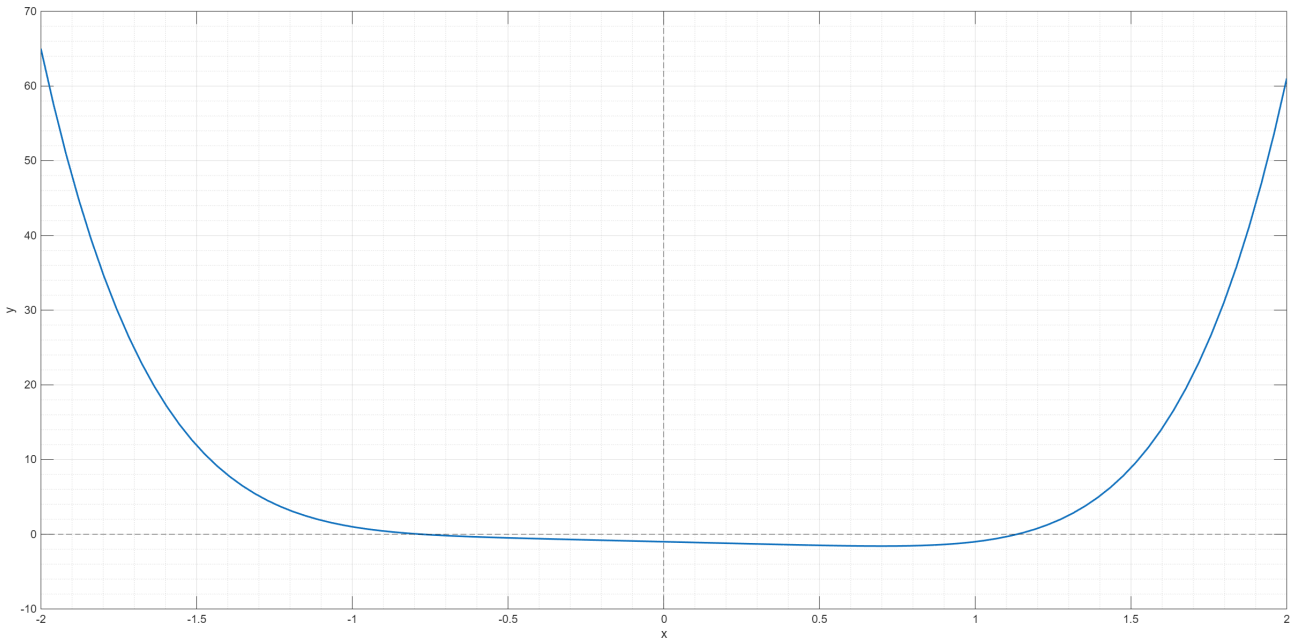
$$Q(x) := P(-x) = x^6 + x - 1$$

we have

$$A = \{1, 0, 0, 0, 0, 1, -1\}$$

and, applying Descartes' rule again, we also obtain $\nu = 1$ in this case, i.e., the number of negative roots of P is $k = 1$. We can conclude that $P(x) = x^6 - x - 1$ has two real roots and two pairs of complex conjugate roots belonging to the circumference of the complex plane centered at 0 with radius 2. In particular:

- the positive root belongs to the interval $[0, 2]$;
- the negative root belongs to the interval $[-2, 0]$.



Esempio 3.5. Let

$$P(x) = x^9 + 2x^8 - 3x^7 + x^6 + x^4 - 2x^2 + x - 1$$

Since

$$\max_{2 \le k \le n+1} \left| \frac{a_k}{a_1} \right| = 3$$

applying Cauchy's Theorem, all real roots of P belong to the interval $[-4, 4]$. Since the set of coefficients is

$$A = \{1, 2, -3, 1, 0, 1, 0, -2, 1, -1\}$$

applying Descartes' rule gives $\nu = 5$. Therefore k, the number of positive roots, can be 1, 3, or 5; i.e., P has 1, 3, or 5 positive roots. Finally, setting

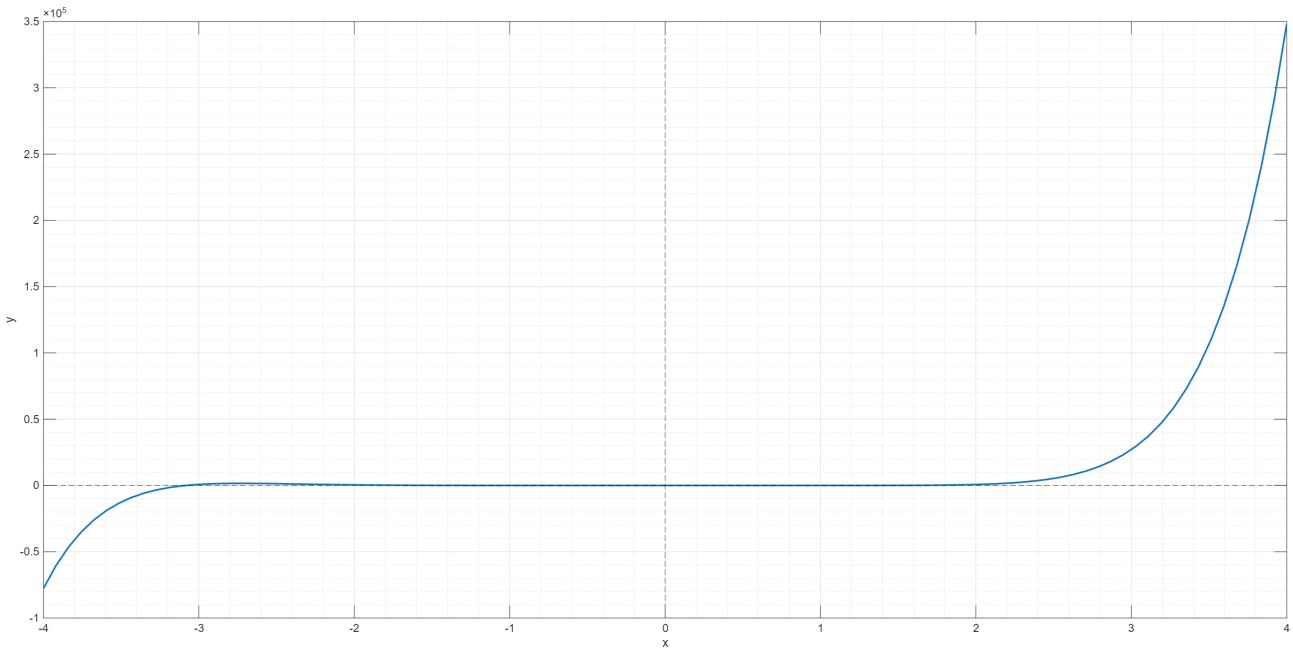
$$Q(x) := P(-x) = -x^9 + 2x^8 + 3x^7 + x^6 + x^4 - 2x^2 - x - 1$$

we have the set of coefficients of Q:

$$A = \{-1, 2, 3, 1, 0, 1, 0, -2, -1, -1\}$$

and, applying Descartes' rule again, we obtain $\nu = 2$. That is, the number of negative roots of P is $k = 0$ or $k = 2$. We can conclude that all roots of $P(x)$ belong to the circle in the complex plane centered at 0 with radius 4. In particular, the following cases are possible:

- P has 5 positive roots belonging to the interval $[0, 4]$ and two pairs of complex conjugate roots;
- P has 3 positive roots belonging to the interval $[0, 4]$ and three pairs of complex conjugate roots;
- P has 1 positive root belonging to the interval $[0, 4]$ and four pairs of complex conjugate roots;
- P has 2 negative roots belonging to the interval $[-4, 0]$, 5 positive roots belonging to the interval $[0, 4]$ and one pair of complex conjugate roots;
- P has 2 negative roots belonging to the interval $[-4, 0]$, 3 positive roots belonging to the interval $[0, 4]$ and two pairs of complex conjugate roots;
- P has 2 negative roots belonging to the interval $[-4, 0]$, 1 positive root belonging to the interval $[0, 4]$ and three pairs of complex conjugate roots.



By zooming in, one can see that P has 3 real roots, of which:

- 1 negative root belonging to the interval $[-\frac{7}{2}, -3]$;
- 1 negative root belonging to the interval $[-1, -\frac{1}{2}]$;
- 1 positive root belonging to the interval $[\frac{1}{2}, \frac{3}{2}]$.

Evaluation of a polynomial at a point The Bisection and Newton methods for the approximate calculation of the roots of an algebraic equation $P(x) = 0$ require the evaluation of the polynomial P and its first derivative P' at successive approximations $x_0, x_1, \ldots, x_n, \ldots$ of the root $\overline{x}$. Consider a polynomial of degree n with real coefficients:

$$P(x) = a_1 x^n + a_2 x^{n-1} + \cdots + a_n x + a_{n+1}$$

Our goal is to build the most efficient algorithm to evaluate P at a fixed point $\overline{x}$.

Algorithm 1: Direct Method The most direct approach consists of calculating each term of the summation individually:

$$P(x) = \sum_{i=0}^{n} a_{i+1} x^{n-i}$$

In this method, each power x^k is calculated from scratch by multiplying x by itself $k-1$ times. This entails a high computational cost for large degrees n.

- *Multiplicative operations:* $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- *Additive operations:* n

Algorithm 2: Method with Power Accumulation Efficiency can be improved by avoiding the recalculation of powers each time. Let's rewrite the polynomial as:

$$P(x) = a_{n+1} + \sum_{i=0}^{n-1} a_{n-i} x^{i+1}$$

The key idea is to accumulate the power of x: instead of calculating x^{i+1} from scratch, use the value of the previous power x^i and multiply it by x (i.e., $x^{i+1} = x \cdot x^i$). In this way, the number of required multiplications decreases drastically, moving from quadratic to linear growth.

- *Multiplicative operations:* $2n$ (one to update the power and one for the coefficient at each step)
- *Additive operations:* n

There exists an algorithm, called **Horner's Algorithm**, which allows evaluating P at a point with a computational cost of n multiplicative operations and n additive operations.

Horner's Algorithm To simplify, let's assume P is of degree 3:

$$P(x) = a_4 + a_3 x + a_2 x^2 + a_1 x^3$$

Factoring out x, we obtain:

$$P(x) = a_4 + x(a_3 + x(a_2 + x a_1))$$

Summarizing, by setting

$$b_1 := a_1$$

$$b_2 := a_2 + \overline{x} b_1$$

$$b_3 := a_3 + \overline{x} b_2$$

$$b_4 := a_4 + \overline{x} b_3$$

we obtain $P(\overline{x}) = b_4$. In general, to evaluate the polynomial $P(x)$ at a point $\overline{x}$, by setting:

$$b_1 := a_1$$

$$b_i := a_i + \overline{x} b_{i-1}, \quad i \in \{2, 3, \ldots, n+1\}$$

we have

$$P(\overline{x}) = b_{n+1}.$$

Nota 11. The numbers $b_i, i \in \{1, \ldots, n\}$ are nothing but the coefficients of the polynomial $Q(x)$ obtained as the quotient of the division of the polynomial $P(x)$ by the binomial $(x - \overline{x})$, and $P(\overline{x})$ is nothing but the remainder, i.e.,

$$P(x) = Q(x)(x - \overline{x}) + P(\overline{x})$$

with

$$Q(x) = b_1 x^{n-1} + b_2 x^{n-2} + \cdots + b_{n-1} x + b_n$$

Therefore, the formulas of Horner's algorithm allow calculating, without performing division, the coefficients of $Q(x)$ as well as the remainder $P(\overline{x})$ according to Ruffini's scheme:

$$\begin{array}{c|cccccc|c}
 & a_1 & a_2 & a_3 & \ldots & a_{n-1} & a_n & a_{n+1} \\
\overline{x} & & \overline{x} b_1 & \overline{x} b_2 & \ldots & \overline{x} b_{n-2} & \overline{x} b_{n-1} & \overline{x} b_n \\
\hline
 & b_1 & b_2 & b_3 & \ldots & b_{n-1} & b_n & b_{n+1}
\end{array}$$

Algorithm Scheme

```
START main
in n, a, x
b(1) = a(1)
for i = 2; i <= n; i++
    b(i) = a(i) + x * b(i-1)
px = a(n+1) + x * b(n)
out px, b
END
```

Computational Cost n multiplicative operations and n additive operations.

```
function [p, b] = hornerP(a, t)
% HORNERP Performs synthetic division of P(x) by (x-t).
%  [p, b] = HORNERP(a, t)
%
%  Input:
%    a - Vector of coefficients ordered from highest to lowest degree
%    t - Point at which to evaluate the polynomial (scalar).
%
%  Output:
%    p - Remainder of the division (i.e., P(t)).
%    b - Coefficients of the quotient polynomial Q(x) of degree n-1

    % Number of coefficients
    m = length(a);

    % Pre-allocation of b for speed (degree n-1)
    b = zeros(1, m-1);

    % The first coefficient of the quotient is equal to the first of P(x)
    b(1) = a(1);

    % Calculation of quotient coefficients (Ruffini's Rule)
    for i = 2:m-1
        b(i) = b(i-1) .* t + a(i);
    end

    % Calculation of the final remainder
    p = b(m-1) .* t + a(m);

end
```

Function hornerP

Esempio 3.6. Evaluate the polynomial

$$P(x) = 2x^9 + 8x^4 - x^3 - 1$$

at the point $\overline{x} = 1$. Using Ruffini's scheme:

$$\begin{array}{c|ccccccccc|c}
 & 2 & 0 & 0 & 0 & 0 & 8 & -1 & 0 & 0 & -1 \\
1 & & 2 & 2 & 2 & 2 & 2 & 10 & 9 & 9 & 9 \\
\hline
 & 2 & 2 & 2 & 2 & 2 & 10 & 9 & 9 & 9 & 8
\end{array}$$

Thus $P(1) = 8$ and the quotient is:

$$Q(x) = 2x^8 + 2x^7 + 2x^6 + 2x^5 + 2x^4 + 10x^3 + 9x^2 + 9x + 9.$$

In MATLAB we have

```
% Definition of the polynomial: P(x) = 2x^9 + 8x^4 - x^3 - 1
a = [2 0 0 0 0 8 -1 0 0 -1];

% Evaluation point
t = 1;

% Performs synthetic division (Ruffini)
[p, b] = hornerP(a, t);

fprintf("Remainder (Value of the polynomial at %d): %d\n", t, p)
fprintf("Coefficients of the Quotient polynomial: \n")
disp(b);
```

```
Remainder (Value of the polynomial at 1): 8
Coefficients of the Quotient polynomial:
     2     2     2     2     2    10     9     9     9
```

Calculation of the derivative We now see that, simultaneously with the calculation of $P(\xi)$, it is also possible to calculate $P'(\xi)$. It can be easily shown that

$$P(x) = Q(x)(x - \xi) + P(\xi)$$

where

$$Q(x) = b_1 x^{n-1} + b_2 x^{n-2} + \cdots + b_{n-1} x + b_n$$

Recalling Taylor's formula:

$$P(x) = P(\xi) + P'(\xi)(x - \xi) + \frac{(x-\xi)^2}{2!} P''(\xi) + \cdots + \frac{(x-\xi)^n}{n!} P^{(n)}(\xi)$$

we can write

$$Q(x) = \frac{P(x) - P(\xi)}{x - \xi} = P'(\xi) + \frac{(x-\xi)}{2!} P''(\xi) + \dots$$

Taking the limit as $x \to \xi$ we obtain

$$Q(\xi) = \lim_{x \to \xi} \frac{P(x) - P(\xi)}{x - \xi} = P'(\xi)$$

Therefore, to calculate $P'(\xi)$ it is sufficient to calculate $Q(\xi)$. We can do this by using Horner's algorithm again on the coefficients b_i. In fact, recalling the definition of $Q(x)$ and setting:

$$\begin{cases} c_1 := b_1 \\ c_i = c_{i-1}\xi + b_i, \quad i \in \{2, \dots, n\} \end{cases}$$

we have

$$P'(\xi) = Q(\xi) = c_n.$$

```
function p = hornerR(c, r, x)
% HORNERR Calculates the first r derivatives of the polynomial P(x) at point x.
%   p = HORNERR(c, r, x) returns a vector with the derivative values.
%
%   Input:
%       c : Polynomial coefficients (descending degree).
%       r : Maximum derivative order (e.g., 2 for second derivative).
%       x : Evaluation point.
%
%   Output:
%       p : Result vector where p(k+1) is the k-th derivative.
%           p(1) = P(x), p(2) = P'(x), ..., p(r+1) = P^(r)(x).

    for i = 0 : r
        % Executes Ruffini's rule: obtains remainder (val) and updated quotient (c)
        % Note: 'c' is overwritten with the quotient coeffs for the next loop
        [val, c] = hornerP(c, x);

        % Inverse Taylor formula: P^(i)(x) = Remainder * i!
        p(i+1) = val * factorial(i);
    end

end
```

Function hornerR

Algorithm Scheme The algorithm to calculate $P(\xi)$ and $P'(\xi)$ is therefore:

```
p = a(1);
dp = p;
for i = 2:n
    p = p * x + a(i);
    dp = dp * x + p;
end
p = p * x + a(n+1);
```

Computational Cost Its total computational cost is $2n$.

```
function [p, dp] = horner(a, t)
% HORNER Evaluates a polynomial and its first derivative using Horner's algorithm.
%  [p, dp] = HORNER(a, t) calculates the value of the polynomial P(x) and its
%  derivative P'(x) at the point (or points) t.
%
%  Input:
%    a - Vector of coefficients ordered from highest to lowest degree
%      P(x) = a(1)*x^n + a(2)*x^(n-1) + ... + a(n)*x + a(n+1)
%    t - Point(s) where to evaluate the polynomial (scalar or vector).
%
%  Output:
%    p  - Value of the polynomial calculated at t.
%    dp - Value of the first derivative calculated at t.

    % Total number of coefficients (polynomial degree n = m - 1)
    m = length(a);

    % Initialization with the highest degree coefficient
    p = a(1);
    dp = p;

    % Horner loop: iteratively updates p and dp
    % Note: The loop stops at the second-to-last coefficient
    for i = 2 : m-1
        p = p .* t + a(i);       % Updates polynomial value
        dp = dp .* t + p;        % Updates derivative using the new p
    end

    % Last step for the polynomial (adding the constant term)
    % The derivative is already complete upon exiting the loop
    p = p .* t + a(m);

end
```

Function horner

Conditioning of polynomial roots Let

$$P(x) = a_1 x^n + a_2 x^{n-1} + \cdots + a_n x + a_{n+1}$$

be a polynomial of degree n and let $\alpha_1, \ldots, \alpha_n$ be its roots. Due to finite representation, the input data of the problem (i.e., the coefficients) are affected by an error which is at most machine epsilon; consequently, the actually calculated roots will be different. The goal of the conditioning analysis is to measure, a posteriori, the sensitivity of the posed problem. In the same polynomial, there may be well-conditioned roots and other ill-conditioned ones; that is, the study is not global (all roots) but pointwise (one root at a time).

Conditioning of a simple root Let α be a generic simple root of the polynomial $P(x)$ and let β be the corresponding root of the polynomial obtained by perturbing the k-th coefficient. It can be shown that:

$$\frac{|\alpha - \beta|}{|\alpha|} \leq \frac{\varepsilon}{|\alpha|} \max_{k \in \{1,\ldots,n+1\}} \left| \frac{a_k \alpha^{n-k+1}}{P'(\alpha)} \right|$$

The quantity

$$\frac{1}{|\alpha|} \max_{k \in \{1,\ldots,n+1\}} \left| \frac{a_k \alpha^{n-k+1}}{P'(\alpha)} \right|$$

represents the condition number of the root. It is evident that the error amplifies either because the first derivative of the computed polynomial at the root is very small (nearly double roots) or because the polynomial coefficients are very large.

Conditioning of a multiple root Let α be a generic root with multiplicity $r > 1$. It can be shown that:

$$\frac{|\alpha - \beta|}{|\alpha|} \leq \frac{\varepsilon^{1/r}}{|\alpha|} \max_{k \in \{1,\ldots,n+1\}} \left| \frac{r! a_k \alpha^{n-k+1}}{P^{(r)}(\alpha)} \right|^{1/r}$$

Therefore, the problem is generally ill-conditioned.

```
function cond = condRoot(c, r, x)
% CONDROOT Calculates the condition number of a root.
%  cond = CONDROOT(c, r, x) estimates how sensitive root x (with multiplicity r)
%  is to perturbations on the polynomial coefficients.
%
%  Input:
%    c - Vector of coefficients (from highest degree to constant).
%    r - Multiplicity of root x (r=1 for simple roots).
%    x - The root value.
%
%  Output:
%    cond - Condition number.

    n = length(c) - 1;   % Degree of the polynomial

    % Calculates derivatives up to order r using Generalized Horner
    p = HornerR(c, r, x);

    y = p(end);          % Value of the r-th derivative: P^(r)(x)

    % Calculates individual polynomial terms: a_k * x^power
    % (Needed to find the maximum term in the numerator)
    for k = 1 : n+1
        fatt(k) = c(k) * x^(n - k + 1);
    end

    % Condition number formula for multiple roots:
    % K = (1/|x|) * [ (r! * max|a_k * x^k|) / |P^(r)(x)| ] ^ (1/r)
    cond = max(abs(fatt * factorial(r) / y).^(1/r)) / abs(x);

end
```

Function condRoot

Problem of simultaneous calculation of all zeros There are several approaches:

1. **Deflation:** A root x_1 is calculated, and the quotient $P_1(x) = P(x)/(x - x_1)$ is constructed. The process is iterated. *Drawbacks:* Round-off error accumulates unacceptably, providing spurious results after the first few roots.

2. **Companion Matrix:** The calculation of the zeros of P is reduced to calculating the eigenvalues of the companion matrix of P, which has P as its characteristic polynomial.

$$P_n(x) = a_1 x^n + \cdots + a_n x + a_{n+1}$$

$$A = \begin{pmatrix} -\frac{a_2}{a_1} & -\frac{a_3}{a_1} & \dots & -\frac{a_n}{a_1} & -\frac{a_{n+1}}{a_1} \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{pmatrix}$$

This is what the MATLAB function `roots` does. *Drawbacks:* The method accurately calculates the matrix eigenvalues, but this does not imply a satisfactory approximation of the zeros of P, since the two problems have different conditioning.

Implementation of Algorithms for Algebraic Equations The analysis carried out in the previous sections regarding the stability of polynomial evaluation (Horner's Algorithm) and derivative calculation (Generalized Horner) finds its natural application in root-finding. Unlike generic routines for arbitrary non-linear functions $f(x)$ (seen in previous chapters), the following implementations are specifically optimized for algebraic equations of the form $P_n(x) = 0$. The use of Horner's scheme to evaluate both $P(x)$ and $P'(x)$ ensures not only greater numerical robustness against round-off errors but also reduced computational cost ($2n$ operations to obtain both value and derivative). Three algorithmic variants are presented below:

`bisecAlg` A specialization of the *Bisection* method that uses Horner's algorithm for pointwise residual evaluation. This approach ensures global convergence even for high-degree polynomials, where direct evaluation of powers x^n could introduce significant errors.

```
function [iter, x_vect, itmax] = bisecAlg(a, b, c, TOLL)
% BISECALG Finds the root of a polynomial in [a,b].
%  [iter, x_vect, itmax] = bisecAlg(a, b, c, TOLL)
%
%  Input:
%    a, b - Search interval endpoints.
%    c    - Polynomial coefficients (for Horner).
%    TOLL - Required tolerance for stopping.
%
%  Output:
%    iter   - Number of iterations performed.
%    x_vect - Vector containing the sequence of approximations.
%    itmax  - Maximum number of theoretical iterations expected.

    % Calculation of endpoint values using Horner
    fa = Horner(c, a);
    fb = Horner(c, b);

    % Zero existence check (Intermediate Value Theorem)
    if fa * fb >= 0
        disp('bisecAlg:ZeroThm','The function does not change sign at the interval endpoints.');
        iter = [];
        x_vect = [];
        itmax = [];
        return;
    end

    % Calculation of maximum theoretical iterations
    itmax = 1 + floor(log((b - a) / TOLL) / log(2));

    % Pre-allocation of vector x for efficiency
    x_vect = zeros(itmax, 1);

    % First iteration
    x_vect(1) = (a + b) / 2;
    fx = Horner(c, x_vect(1));
    iter = 1;

    % Bisection loop
    while iter < itmax && abs(fx) >= TOLL
        iter = iter + 1;

        % Sub-interval selection
        if fa * fx < 0
            b = x_vect(iter-1);
        else
            a = x_vect(iter-1);
            fa = fx;
        end

        % New estimate
        x_vect(iter) = (a + b) / 2;
        fx = Horner(c, x_vect(iter));
    end

    % Removes excess zeros if the loop finishes before itmax
    x_vect = x_vect(1:iter);

end
```

Function bisecAlg

`newtonAlg` Implementation of the *Modified Newton* method, designed to handle roots with multiplicity $m \geq 1$. This routine leverages repeated synthetic division to simultaneously evaluate $P(x)$ and $P'(x)$ at the same point, making the calculation of the update step $x_{k+1} = x_k - m\frac{P(x_k)}{P'(x_k)}$ particularly efficient.

```matlab
function [x_vect, iter] = newtonAlg(x0, c, m, TOLL, itmax)
% NEWTONALG Finds the root of a polynomial using Newton's method.
%  [x_vect, iter] = newtonAlg(x0, c, m, TOLL, itmax)
%
%  Input:
%    x0    : Initial root estimate.
%    c     : Vector of polynomial coefficients (for Horner).
%    m     : Expected root multiplicity (m=1 for simple roots).
%    TOLL  : Required tolerance for stopping (on increment and residual).
%    itmax : Maximum number of allowed iterations.
%
%  Output:
%    x_vect : Vector containing the sequence of approximations.
%    iter   : Number of iterations performed.

    % Pre-allocation of vector x for efficiency
    x_vect = zeros(itmax, 1);

    % Initialization
    x_vect(1) = x0;
    iter = 1;
    err = 1; % Arbitrary initial relative error > TOLL

    % Calculation of value and derivative at the initial point using Horner
    [fx, dfx] = Horner(c, x_vect(1));

    % Newton loop
    % Stopping criteria: max iterations, relative error, residual
    while (iter < itmax) && (err > TOLL) && (abs(fx) > TOLL)

        % Check to avoid division by zero (zero derivative)
        if dfx == 0
            disp('Stop: The derivative has vanished.');
            break;
        end

        % Modified Newton step for multiplicity m
        % x_{k+1} = x_k - m * (f(x_k) / f'(x_k))
        x_vect(iter + 1) = x_vect(iter) - m * (fx / dfx);

        % Calculation of estimated relative error
        if x_vect(iter + 1) ~= 0
            err = abs(x_vect(iter + 1) - x_vect(iter)) / abs(x_vect(iter + 1));
        else
            err = abs(x_vect(iter + 1) - x_vect(iter)); % Absolute error if x is 0
        end

        % Update for next iteration
        iter = iter + 1;
        [fx, dfx] = Horner(c, x_vect(iter));

    end

    % Resize vector to actual number of iterations performed
    x_vect = x_vect(1:iter);

end
```

Function newtonAlg

`bisnewAlg` A *hybrid* strategy that combines the strengths of the two previous methods. The algorithm starts a "roughing" phase using Bisection to get closer to the root (entering the basin of attraction), and then automatically switches to Newton's method ("refinement" phase) to exploit its quadratic convergence.

```matlab
function [x_bis, i_bis, x_newt, i_newt] = bisnewAlg(a, b, c, m, TOLL, itmax)
% BISNEWALG Hybrid method for polynomials.
%
%   The function combines the robustness of Bisection (to approach the
%   root) with the speed of Newton (to refine the result), specifically
%   for algebraic equations P(x)=0.
%
%   Input:
```

```
%       a, b  : Initial interval endpoints.
%       c     : Vector of polynomial coefficients.
%       m     : Root multiplicity (m=1 for simple roots).
%       TOLL  : Required tolerance for the final phase (Newton).
%       itmax : Max iterations for the final phase (Newton).
%
%   Output:
%       x_bis  : (vector) History of approximations (Bisection).
%       i_bis  : (int)    Number of iterations (Bisection).
%       x_newt : (vector) History of approximations (Newton).
%       i_newt : (int)    Number of iterations (Newton).

    %% 1. Input Validation
    arguments
        a     (1,1) double
        b     (1,1) double
        c     (1,:) double % Row vector of coefficients
        m     (1,1) double {mustBePositive} = 1
        TOLL  (1,1) double {mustBePositive} = 1e-6
        itmax (1,1) double {mustBeInteger, mustBePositive} = 100
    end

    %% 2. Phase 1: Roughing (Bisection)
    % We use a fixed tolerance of 0.5e-1 to approach safely.
    toll_coarse = 0.5e-1;
    [i_bis, x_bis, ~] = bisecAlg(a, b, c, toll_coarse);

    %% 3. Phase 2: Refinement (Newton)
    % The trigger point for Newton is the last approximation from Bisection
    start_point = x_bis(end);
    [x_newt, i_newt] = newtonAlg(start_point, c, m, TOLL, itmax);

end
```

Function bisnewAlg

3.3 Exercises

1. Write a MATLAB function that implements the Bisection method.
2. Write a MATLAB function that implements Newton's method.
3. Write a MATLAB function that implements the combined Bisection-Newton method.
4. Given the equation
$$F(x) = \cos(x) - 4x.$$
   - Find an interval that contains the zero of the function.
   - Approximate the zero with machine precision using the Bisection method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using Newton's method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using the combined Bisection-Newton method. Report the approximated value of the zero and the number of iterations performed by the methods.
5. Given the equation
$$F(x) = e^x + \frac{x}{10}.$$
   - Find an interval that contains the zero of the function.
   - Approximate the zero with machine precision using the Bisection method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using Newton's method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using the combined Bisection-Newton method. Report the approximated value of the zero and the number of iterations performed by the methods.

6. Given the equation
$$F(x) = x + \log(x^3).$$
   - Find an interval that contains the zero of the function.
   - Approximate the zero with machine precision using the Bisection method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using Newton's method. Report the approximated value of the zero and the number of iterations performed by the method.
   - Approximate the zero with machine precision using the combined Bisection-Newton method. Report the approximated value of the zero and the number of iterations performed by the methods.

7. Suppose that a chemical reaction generates, at a certain instant t, a concentration of a particular ion given by the law:
$$c(t) = 7e^{-5t} + 3e^{-2t}.$$
   If at the initial instant the concentration is $c(0) = 10$, at what instant $\bar{t}$ will the initial concentration be halved, i.e.,
$$c(\bar{t}) = 5?$$
   - Considering that the problem is equivalent to determining the zero of the equation
$$F(t) = 7e^{-5t} + 3e^{-2t} - 5 = 0,$$
     find an interval on the positive semi-axis that contains the zero of the function F.
   - Approximate the zero with machine precision using the combined Bisection-Newton method. Report the approximated value of the zero and the number of iterations performed by the methods.

8. Calculate $\sqrt{33}$ with machine precision using the combined Bisection-Newton method.

9. Calculate $1/43$ with machine precision using the combined Bisection-Newton method.

10. Write a MATLAB function that implements the Bisection method for algebraic equations.

11. Write a MATLAB function that implements Newton's method for algebraic equations.

12. Write a MATLAB function that implements the combined Bisection-Newton method for algebraic equations.

13. Write a MATLAB function that implements Horner's algorithm for calculating the value of a polynomial and its derivative at a point.

14. Write a MATLAB function that calculates the condition numbers of simple and multiple roots.

15. Let
$$P(x) = x^6 - x - 1.$$
   - Approximate the real roots of P with machine precision using the combined Bisection-Newton method. What is the number of iterations of the Bisection method? What is the number of iterations of Newton's method?
   - Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

16. Let
$$P(x) = x^9 + 2x^8 - 3x^7 + x^6 + x^4 - 2x^2 + x - 1$$
   - Approximate the real roots of P with machine precision using the combined Bisection-Newton method. What is the number of iterations of the Bisection method? What is the number of iterations of Newton's method?
   - Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

17. Let
$$P(x) = 2x^9 - 3x^8 + 4x^5 + \frac{1}{2}x^4 - x^3 + x - \frac{1}{2}$$
   - Find the interval that contains all real roots.

- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the combined Bisection-Newton method. What is the number of iterations of the Bisection method? What is the number of iterations of Newton's method?
- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

18. Let
$$P(x) = x^7 - 3x^6 + 2.25x^5 - x^3 + 3.5x^2 - 3.75x + 1.125.$$
- The polynomial P has $x = \frac{3}{2}$ as a double root. Calculate its conditioning.
- Approximate it with machine precision using Newton's method. What is the number of iterations?
- Approximate it with machine precision using the suitably modified Newton's method. What is the number of iterations?
- Approximate it using the MATLAB 'roots' function.

19. Let
$$P(x) = x^5 + 0.631x^4 + 0.676387x^3 - 0.325473867x^2 + 0.04352613299999995x - 0.001860867$$
- Find the interval that contains all real roots of P.
- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the studied methods appropriately. What is the number of iterations of the method used?
- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

20. Let
$$P(x) = x^8 - 4.01x^7 + 4.02x^6 + x^3 - 3.01x^2 + 0.01x + 4.02.$$
- Find the interval that contains all real roots of P.
- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the studied methods appropriately. What is the number of iterations of the method used?
- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

21. Let
$$P(x) = 2x^8 - 8.02x^7 + 8.04x^6 + x^3 - 3.01x^2 + 0.001x + 4.02$$
- Find the interval that contains all real roots.
- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the studied methods appropriately. What is the number of iterations of the method used?
- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

22. Let
$$P(x) = x^{10} - 55x^9 + 1320x^8 - 18150x^7 + 157773x^6 - 902055x^5 + 3416930x^4 - 8409500x^3 + 12753576x^2 - 10628640x + 3628800.$$
- Find the interval that contains all real roots of P.
- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the studied methods appropriately. What is the number of iterations of the method used?

- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.

23. Let

$$P(x) = x^6 - 2x^5 - 4x^4 + 6x^3 + 7x^2 - 4x - 4$$

- Find the interval that contains all real roots of P.
- How many real roots are there? What are their multiplicities? Find an interval containing each real root.
- Approximate the real roots of P with machine precision using the studied methods appropriately. What is the number of iterations of the method used?
- Analyze the conditioning of the real roots of P. Report the value of the roots with the digits that can be considered correct.